Project report

Matura thesis

01.11.2026

ABRAXAS

anonyme-sjf-26

CONTENTS

1 Introduction 5
Origin of the project 5
Reminder of the research question 5
Form of the project 6
General approach 6
Main technical choices 6
Success criteria 6
2 Notes 7
2.1 Use of AI 7
2.2 Sources 7
Links 7
Photographs 7
3 Glossary and sailing theory 9
3.1 Units 9
3.2 Reference frames 9
Earth frame 9
Boat frame 9
3.3 Convention 9
3.4 Movement over ground 10
SOG 10
COG 10
3.5 True wind 10
3.6 Induced wind 11
3.7 Apparent wind 12
3.8 Calculation of the true wind 12
3.9 Gain strategy 13
Upwind gain 13
Downwind gain 13
3.10 Calculation of the VMG 14
3.11 Pitch and roll 14
4 Intentions 15
4.1 Identification of needs 15
4.2 Existing solutions 15
4.3 Requirements for the solution 17
4.4 MIT License 18
4.5 Name of the device 19
4.6 Logo of the device 20
5 First prototype 21
5.1 Hardware 21
Order 1 21
Order 2 21
Order 3 22
Order 4 22
5.2 Assembly 23

5.3 Installing the OS ... 24
5.4 SSH with public keys ... 24
Asymmetric encryption ... 25
NAT loopback and port mapping ... 25
Change of the Raspberry's public key ... 26
5.5 File transfer ... 27
In practice ... 27
5.6 Packages and libraries ... 28
5.7 Testing the GPIO ... 29
5.8 Serial Port setting ... 30
5.9 Using the IMU ... 31
5.10 Using the GPS ... 32
Installing Blinka ... 33
Special precaution ... 35
Installing the GPS library ... 36
Testing the GPS ... 36
5.11 Using the camera ... 38
5.12 Using the anemometer ... 39
Choice of anemometer ... 39
Connection to the prototype ... 41
5.13 SQL database ... 42
6 Software of the first prototype ... 45
6.1 Class file — classes.py ... 45
Logging library ... 46
6.2 Button monitoring — interface.py ... 46
Automatic execution ... 46
6.3 Session management — master.py ... 47
6.4 IMU and GPS management — imu.py & gps.py ... 48
6.5 Video recording — vid.py ... 49
6.6 Finalising the video recording — grp.py ... 50
6.7 Automatic installation ... 51
6.8 Configuration files ... 51
progress_bar.sh ... 51
config.sh ... 52
auto_install.exp ... 53
config.sql ... 53
6.9 Moving to GitHub ... 53
7 Second prototype ... 55
7.1 Autonomous power supply ... 55
Battery and autonomy ... 55
PowerBoost ... 55
ON/OFF SHIM ... 55
7.2 Motherboard ... 56
Electrical schematic ... 56
7.3 PCB footprint ... 61
8 3D work ... 63

8.1 Modelling ... 63
Box ... 63
Lid ... 66
Removable standoffs ... 66
GoPro-compatible mounts ... 66
Anemometer mount ... 66
8.2 Printing ... 67
8.3 Assembly ... 67
Additional parts for the assembly ... 67
Soldering ... 68
Wiring plan ... 69
Assembly steps ... 69
9 Software of the second prototype ... 71
9.1 Usage ... 71
Setting up the device ... 71
Switching the device on and off ... 71
Indicators ... 71
Starting and stopping a recording ... 72
Displaying data in real time ... 72
Accessing the debrief ... 73
9.2 Automatic installation ... 74
config.sh ... 75
config.sql ... 75
progress_bar.sh ... 75
auto_install.exp ... 75
9.3 Recording system ... 75
MQTT ... 75
db_recording.py ... 75
anemo.py, imu.py, gps.py ... 75
interface.py ... 76
master.py ... 76
vid.py ... 76
grp.py ... 76
classes.py ... 76
gpio_daemon.py ... 76
9.4 Hotspot ... 77
server.js ... 77
client.html ... 77
client.css ... 77
horizon.png ... 77
client.js ... 77
export.html ... 78
export.css ... 78
export.js ... 78
9.5 Remote server ... 78
Server system ... 79

Operation and purpose of the server 79
AI debrief 80
Role of the server within the project 81
Server code 82
9.6 Abraxas OS 82
10 Trial 83
10.1 Installing the device 83
10.2 Course of the training session 83
10.3 Technical operation of the device 85
10.4 Quality of the analysis and of the visualisation 86
10.5 Drawbacks of using the device for the crew 86
10.6 Benefits of using the device for the crew 86
11 Conclusion 91
11.1 Summary 91
On the technical level 92
On the level of the benefit for the team 92
11.2 Improvements and outlook 92
11.3 Final thoughts 93
12 Acknowledgements 95
13 Bibliography 97
13.1 Books 97
13.2 Interviews and oral sources 97
13.3 Existing systems 97
13.4 Technical documentation and code repositories 97
13.5 Articles and web pages 98
13.6 Regatta results 98
14 Appendices 99
14.1 Photos of the prototype 99
14.2 Access to the report PDF and the GitHub repositories 113
14.3 Access to the CAD model 114
14.4 Access to the server 114
14.5 Access to the hotspot 115
14.6 Websites of the existing solutions 115
14.7 Hardware 115
14.8 Technical documentation 117
14.9 Articles and web pages 118
14.10 Access to the code 118

1 INTRODUCTION

Origin of the project

In the spring of 2025, my team and I took part in the Critérium Longtze, a stage of the European circuit of the Longtze, a high-performance racing monohull sailboat.

We finished twelfth out of fourteen[1]. This ranking was the result of a full season of diligent training. Clearly, something in the way we trained needed to be reconsidered.

In our club, we train with three crews on three boats. During sessions, we sail together, comparing our speeds, our manoeuvres and our strategies. The cornerstone of a good performance was, and still is, for us the reproducibility and the consistency of our performances. Indeed, the crew that wins a regatta is not the one that is best in its actions or decisions, but the one that makes the fewest mistakes.

As such, when the training regattas gave satisfactory results, that is to say when the different crews were sailing at an equivalent level without showing any major weakness in their technique, we moved on to refining elements of lesser importance.

However, during the critérium, we realised that the fundamentals we thought we had mastered needed to be thoroughly reworked. Side by side, our competitors were systematically faster than us and pointed higher. Had we realised this during training, we could have worked on these points and thus remained competitive at the critérium.

The Abraxas project was therefore born out of frustration, and out of the desire not to remain stuck on it, but on the contrary to move forward and do everything possible to prevent such a situation from happening again.

It was only natural that I chose the development of a solution to this problem as the subject of my Matura thesis.

Reminder of the research question

The research question formulated at the beginning of the year for my thesis is the following:

> How would it be possible, using a computer device, to collect data during the training sessions of an inshore[2] racing sailing team in order to provide additional material for debriefings and, ultimately, give the team the opportunity to improve its performance?
>
> After answering this question, I will endeavour to build such a device, while keeping in mind that such devices already exist, but are reserved for the elite because of

[1] "Classement du Criterium Longtze YCG 2025", *Manage2Sail*, 04.05.2025, https://www.manage2sail.com/fr/event/Longtze2025 (accessed on 21.10.2026).

[2] Inshore refers to competitive sailing on sheltered waters, such as lakes or coastal areas. The regattas sailed are short, technical and dynamic. The boats used are generally light, fast and particularly demanding.

> their price and complexity. This project therefore also aims to make such systems accessible to all.

This research question has not changed and still applies to the project.

Form of the project

The project is called Abraxas, in reference to the novel *Demian* by Hermann Hesse[3]. An explanation of the choice of this name is given in Section 4.5. In the end, the result of the project consists of a case that can be attached to the foot of the mast, an anemometer connected to the case by a cable, and an external GPS antenna. The case is the centre of the device; it is from there that it is controlled.

The measurements concern the wind, the speed, the position of the boat and its orientation in space. A video is also recorded. The device itself has no screen, but any device that can connect to a WiFi network can be used to interact with it.

General approach

The first step in carrying out this project was to define precise specifications by identifying the intentions behind it. I then started the design and then the assembly of a first prototype. The goal was not to be able to use it in real conditions, but to prove the concept by building a working assembly of all the components together. Once this first prototype was completed, I was able to move on to building a second one which, this time, could be taken on board a boat. This brought a new set of difficulties, related in particular to robustness and waterproofing constraints. It was only after being able to build a usable dataset with this prototype that I started creating two data visualisation systems. One displays the data in real time, the other is designed to be used during the debrief.

Main technical choices

The device is built around a Raspberry Pi Zero. Running a full Linux (without a graphical interface), it offers all the power needed for massive data processing and for video recording. The case is 3D printed and contains a motherboard on which all the components are mounted.

Success criteria

The main condition for considering my project complete is to have a device capable of recording, processing and presenting data about an inshore sailing training session while respecting the principles of affordability, adaptability to clubs, open source, no-nonsense and multifactorial approach. These principles are developed in Section 4.3. The device should eventually be included in training sessions and contribute to their improvement, so as to avoid ending up again in the situation of the 2025 critérium.

[3]HESSE Hermann, *Demian. Die Geschichte von Emil Sinclairs Jugend*, Berlin, Suhrkamp, 2012.

2 NOTES

2.1 Use of AI

As we will see, the use of AI in the broad sense is an intrinsic part of my work, in particular in the analysis of the data. The use of large language models (LLMs) is a different matter. For me, even outside the scope of this work, it is a difficult question. I believe I am fully aware of their limits and of the dangers inherent in their use; I am nevertheless not blind to the advantages they have.[4] I did not decide to set them aside completely, but rather to use them thoughtfully. In this approach, I was fortunate to receive the insights of a philosopher specialising in issues related to the uses of AI.

As part of this project, no text was written or rephrased by AI. I did not use AI to organise or develop my ideas, nor to proofread the substance. It is only in the computing part that I resorted to LLMs. The AI-generated content present in the training debrief is marked as such. In particular, I used AI processes to:

- Get an opinion on my code architecture ideas.
- Identify and fix errors.
- Perform completions.
- Discover, learn and understand new languages and new syntaxes.
- Receive suggestions on complex cases, such as real-time data exchange between two different languages.
- Automatically generate commit messages and release notes on GitHub.

In some cases, I also delegated the writing of code to an LLM. When this is the case:

- I do not consider the code to be an integral part of my work, but a tool that I have chosen and that I use, just as I would use a library that I had imported.
- I state it explicitly in my report.
- I explain the reason and give a justification.

2.2 Sources

Links

All links and files mentioned in this document are provided as QR codes in the appendices (Section 14), including the PDF version of this report.

Photographs

Unless otherwise stated, all photographs were taken by the author in 2026.

[4] Philosopher specialising in AI-related issues, discussion of 09.06.2026.



Figure 1: Photo taken during SailGP 2026 in Geneva. In the foreground is the German F50. Behind it, the American F50 can be seen.

3 GLOSSARY AND SAILING THEORY

3.1 Units

The units used in sailing are the following[5]:

Quantity	Unit	Equivalence
Distances in nautical miles	NM	1 NM = 1852 m
Speed in knots	knt	1 knt = 1.852 km/h
Angles in degrees, minutes and seconds	00°00'00''	1° = 60' = 3600''

3.2 Reference frames

Two reference frames are defined: the Boat frame and the Earth frame. The offset between the directions of the axes of these frames is the true course of the boat (COG). To convert a direction from the Boat frame to the Earth frame, the COG is added.

Earth frame

This frame is oriented with the x axis pointing towards true North and the y axis towards the East. The z axis points downwards, so the positive direction of rotation is clockwise.

Boat frame

This frame is oriented with the x axis along the longitudinal axis (from stern to bow) and the y axis along the transverse axis of the boat (from port to starboard). The z axis also points downwards, so the positive direction of rotation is also clockwise.

Leeway and currents are neglected: the true heading of the boat (HDG), i.e. the orientation of the boat's x axis relative to true North, is equal to the angle of its track relative to true North (COG).

3.3 Convention

Strictly speaking, wind vectors point in the direction in which the wind blows. However, by convention, wind angle data is displayed in the opposite direction. Upwind, the wind blows at about 220° from the boat's x axis. We then say that the wind is at 40°. By a misuse of language, we say that the apparent wind is at 40°, when it is in fact at 220°.

For the sake of clarity, in this document the direction of a wind vector is taken to be the direction towards which it blows. Only the final display will follow the convention. We will simply subtract 180° from it.

[5] Coach of the racing team of the Yacht Club de Genève, *Theory lessons, briefing and debriefing*, unpublished oral knowledge.

3.4 Movement over ground

The movement over ground is the displacement vector of the Boat frame relative to the Earth frame. It is denoted $\vec{B}$ (from Bewegung, German for movement).

In the Earth frame:

$$\vec{B} = \begin{pmatrix} \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

In the Boat frame:

$$\vec{B} = \begin{pmatrix} \mathrm{SOG} \\ 0 \end{pmatrix}$$

SOG

The speed over the ground is called Speed Over Ground (SOG). It is calculated from the position measurements of a GPS: it is the norm of the derivative of the track.

It may differ from the speed through the water in the presence of currents. On the lake, this phenomenon is negligible; it can however become essential to take it into account in regions with strong currents such as northern Brittany, where currents can reach 3knt[6].

COG

The course over the ground is called Course Over Ground (COG). It is calculated from the position measurements of a GPS: it is the angle of the derivative of the track relative to true north.

It may differ from the course through the water because of currents. It also differs from the course relative to magnetic north (magnetic declination), but this phenomenon is negligible at our latitudes (a few minutes of arc). It becomes significant further north: the delta can reach 15° in Lapland[7].

3.5 True wind

The true wind is the displacement vector of the air mass relative to the Earth frame at a given point. It is the wind felt by an observer standing still on the shore. It is denoted $\overrightarrow{V_R}$.

The norm of this vector is called True Wind Speed (**TWS**) and its angle relative to the boat's axis True Wind Angle (**TWA**).

[6] Ibid.
[7] Ibid.

In the Earth frame:

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{TWS} \cdot \cos(\mathrm{COG} + \mathrm{TWA}) \\ \mathrm{TWS} \cdot \sin(\mathrm{COG} + \mathrm{TWA}) \end{pmatrix}$$

In the Boat frame:

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{TWS} \cdot \cos(\mathrm{TWA}) \\ \mathrm{TWS} \cdot \sin(\mathrm{TWA}) \end{pmatrix}$$

The TWA is different from the meteorological True Wind Direction (**TWD**): the TWD indicates the direction from which the wind blows relative to true north, whereas the TWA indicates the direction from which the wind blows relative to the boat's axis. In other words, the TWD is nothing other than the TWA expressed in the Earth frame.

For example: the bise is a wind blowing from the north. Its TWD is 0°. A boat sailing upwind at 30° to the true wind will observe a TWA of 30°.

3.6 Induced wind

The induced wind is the displacement vector of the Earth frame relative to the Boat frame. It is the wind felt by an observer standing still on the moving boat if the true wind is equal to $\vec{0}$. It is denoted $\overrightarrow{V_V}$.

The norm of this vector is called Induced Wind Speed (**IWS**) and its angle relative to the boat's axis Induced Wind Angle (**IWA**).

Since the induced wind is created by the movement of the boat, the IWA is always equal to 180°, unless the boat makes leeway or the current moves it sideways. These effects are considered negligible on the lake.

Once this approximation is made, the induced wind has the property of being equal to the opposite of the movement over ground vector $\vec{B}$.

In the Earth frame:

$$\overrightarrow{V_V} = \begin{pmatrix} \mathrm{IWS} \cdot \cos(\mathrm{COG} + \mathrm{IWA}) \\ \mathrm{IWS} \cdot \sin(\mathrm{COG} + \mathrm{IWA}) \end{pmatrix} = -\vec{B} = \begin{pmatrix} -\mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ -\mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

In the Boat frame:

$$\overrightarrow{V_V} = \begin{pmatrix} \mathrm{IWS} \cdot \cos(\mathrm{IWA}) \\ \mathrm{IWS} \cdot \sin(\mathrm{IWA}) \end{pmatrix} = -\vec{B} = \begin{pmatrix} -\mathrm{SOG} \\ 0 \end{pmatrix}$$

with $\mathrm{IWS} = \mathrm{SOG}$ and $\mathrm{IWA} = 180°$ (the induced wind blows towards the stern).

3.7 Apparent wind

The apparent wind is the displacement vector of the air mass relative to the Boat frame. It is also the resultant of the true wind and the induced wind. It is the wind observed by an observer standing still on a moving boat. It is denoted $\overrightarrow{V_A}$.

The norm of this vector is called Apparent Wind Speed (**AWS**) and its angle relative to the boat's axis Apparent Wind Angle (**AWA**).

In the Earth frame:

$$\overrightarrow{V_A} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) \end{pmatrix}$$

In the Boat frame:

$$\overrightarrow{V_A} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{AWA}) \end{pmatrix}$$

3.8 Calculation of the true wind

The apparent wind $\overrightarrow{V_A}$ is easily measured with an anemometer, and the induced wind $\overrightarrow{V_V}$ is calculated from $\vec{B}$, measured with the GPS. From there, we want to obtain the true wind $\overrightarrow{V_R}$.

The calculation is carried out in the Earth frame.

$$\overrightarrow{V_A} = \overrightarrow{V_R} + \overrightarrow{V_V} \Rightarrow \overrightarrow{V_R} = \overrightarrow{V_A} - \overrightarrow{V_V} = \overrightarrow{V_A} + \vec{B}$$

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) \end{pmatrix} + \begin{pmatrix} \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix} = \begin{pmatrix} V_{R_x} \\ V_{R_y} \end{pmatrix}$$

$$\mathrm{TWS} = \sqrt{V_{R_x}^2 + V_{R_y}^2}$$

$$\mathrm{TWD} = \mathrm{atan2}\left(V_{R_y}, V_{R_x}\right)$$

We switch to the Boat frame:

$$\mathrm{TWA} = \mathrm{TWD} - \mathrm{COG}$$

Or, according to the convention of displaying the direction the wind comes from:

$$\mathrm{TWA}_{\mathrm{convention}} = \mathrm{TWA} + 180°$$

In their expanded forms:

$$\mathrm{TWS} = \sqrt{\begin{aligned}&(\mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}))^2 \\ &+(\mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}))^2\end{aligned}}$$

$$\begin{aligned}\mathrm{TWA}_{\mathrm{convention}} = \operatorname{atan2}(&\mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}), \\ &\mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}) + 180)\end{aligned}$$

These are the true wind values displayed in the dashboard; they come from the calculations above. The only difference is that the code works in radians.

3.9 Gain strategy

Upwind gain

It is impossible for a sailboat to head in the direction the wind is coming from. If it wants to reach a target located upwind, it must sail at an angle of about 45° to its target, then tack (turn so that the wind hits the other side of the boat by passing the bow through the wind) and continue sailing on that tack. This process is called beating.

There is an optimal upwind angle: the more you luff up (head closer to the wind), the more you slow down, but the more you go in the right direction; the more you bear away (head further from the wind), the more you speed up, but the less you go in the right direction. The goal is to maximise the component of the boat's speed along the wind axis. This component is called Velocity Made Good (**VMG**), in French *vitesse capitalisée*[8].

Downwind gain

If you want to reach a target downwind, the most efficient way is not to go there in a straight line either. The problem here is that the boat's speed is created by the apparent wind, not by the true wind. If it goes in the same direction as the wind, its speed will be subtracted from the true wind and the apparent wind will decrease. To avoid this phenomenon, one sails towards a downwind target by alternating legs at about 135° to the wind. This way, the apparent wind is in no danger of dropping towards zero. On the contrary: if enough induced wind is built up, the apparent wind will become considerably stronger than the true wind, and its direction will change, so that the AWA can be 45° while the TWA is 135°. It is this phenomenon that makes it possible to reach the highest speeds on high-performance boats.

Once again, there is an optimum here: the more you luff up, the more you speed up, but the less you go in the right direction; the more you bear away, the more you slow down, but the more you go in the right direction. The goal is to minimise the VMG (maximise its absolute value)[9].

[8] Ibid.
[9] Ibid.

3.10 Calculation of the VMG

Once again, strictly speaking, the VMG should be positive when going in the direction in which the wind blows, but by convention it is positive when sailing upwind. Its sign must therefore be artificially reversed. The calculation of the VMG is thus the calculation of the opposite of the component of the SOG along the TWA.

$$\mathrm{VMG} = -\ \mathrm{SOG} \cdot \cos(\mathrm{TWA})$$

If the TWA is taken according to the convention, the sign of the VMG is consistent:

$$\mathrm{VMG} = \mathrm{SOG} \cdot \cos(\mathrm{TWA}_{\mathrm{convention}})$$

3.11 Pitch and roll

Pitch, *tangage* in French, is the rotational movement around the boat's y axis: it is the movement made when the boat dips its nose. On displacement boats, this value is often close to zero and therefore difficult to measure. It is on foiling boats that it takes on its full meaning: foils are optimised for a pitch value that can sometimes reach –10°[10].

Roll, *roulis*, or, more generally, heel, is the rotational movement around the boat's x axis: it is the movement made when the boat leans under the effect of the wind, for example. Each boat has, depending on the shape of its hull, a roll value for which its performance is optimised. The crew's goal is therefore, as far as possible, to keep the boat in this position[11].

[10] Ibid.
[11] Ibid.

4 INTENTIONS

4.1 Identification of needs

Following the identification of the problem with our training method described in the introduction, the question of how to solve it quickly arose. What strategies should we put in place to remedy this problem? What means should we have at our disposal in order to do so?

The problem did not lie in the quality of the training itself, but in the lack of objective data against which to compare ourselves. Indeed, it is possible not to realise that a major weakness exists, simply because all our points of comparison at the local level share it.

We therefore decided to look for a way to collect this data, not only to be able to detect this kind of problem in the future, but also to identify the mechanisms that lead to it. To do this, I imagined a device capable of measuring various variables that are significant for the functioning of a boat, and software capable of analysing them and highlighting areas for improvement. Among these variables, we wanted to have the SOG, the COG, the TWS, the TWA, the AWS, the AWA and above all the VMG. We also wanted a video of the crew while sailing to link our actions with their consequences on the measured data. Finally, this data was not to be analysed individually, but put into perspective over several training sessions in order to identify a real evolution in performance.

4.2 Existing solutions

Our team is of course not the first to encounter this need, and as such there are several systems offering a similar service. We identified four:

- Garmin-type smartwatches

With a price range from about 250 to 1500CHF[12], this type of watch represents a significant investment for an individual, but a feasible one for a club. It is a tool we already use to know our speed (SOG). It plays an important role in the way we practise our sport. Nevertheless, it is more focused on individual performance (heart rate, number of calories burned, etc.) than on team performance. Moreover, it offers neither the possibility of taking the wind into account nor any real performance tracking between training sessions. Yet, without wind data, speed data is unusable after the end of the training session.

[12] "Wearables smartwatches", *Garmin*, n.d., https://www.garmin.com/fr-CH/c/wearables-smartwatches/ (accessed on 11.10.2026).



Figure 2: Photograph of our crew during the Critérium Longtze 2025.
Untitled, 04.05.2025, private collection, with the photographer’s permission.

- Vakaros

This is an American training monitoring system specialised in sailing sports. As such, it is perfectly suited to a team's need to optimise its sailing during training. Nevertheless, it does not offer the possibility of taking the wind into account either, which does not allow effective tracking. Moreover, it is sold at the prohibitive price of 1085CHF[13] without accessories.

- Orca

This Norwegian system is designed for keelboats sailing at sea. I had the opportunity to test it during a crossing of several days in the Baltic Sea and to observe its formidable efficiency. However, it is focused on assisting long-distance navigation and not on optimising performance during dinghy training sessions. It is nevertheless able to produce a perfectly usable performance report, and at a price of 599CHF[14], it would probably have been the best option on the market. Moreover, it takes the wind into account. Unfortunately, installing it on the boat is complex: it is not a removable device but a permanent one. This does not suit the needs of a club at all. Such a tool should be usable on different boats or be completely removed during sailing lessons for beginners.

- Custom systems reserved for the elite

For elite professional teams such as those taking part in the Sail Grand Prix (SailGP) or the America's Cup (AC), extremely precise monitoring systems directly integrated into the design of the boat exist. Extremely little information can be found on this subject, because they represent a considerable competitive advantage. Each organisation keeps the exact workings of its system secret. These devices are obviously the best, being the most specialised and focused on racing. They take into account a gigantic number of parameters, down to the internal forces and moments in the boat's structure as well as the micro-vibrations to which it is subjected. Such systems are moreover by nature impossible to remove[15].

4.3 Requirements for the solution

After identifying the problem, the needs to remedy it and the existing solutions, we came to the conclusion that no existing system fully met our needs. It was only after this reflection that I decided to design a system matching our needs myself.

[13] "Atlas 2", *Vakaros*, n.d., https://www.vakaros.com/en-eu/collections/atlas-family/products/atlas2 (accessed on 11.10.2026).
[14] "Get Orca", *Orca*, n.d., https://getorca.com/product-configuration/ (accessed on 11.10.2026).
[15] *Lessons, briefing and debriefing*, op. cit., p.9.

The requirements set for this device are the following:

- Affordability

The device must not be reserved for an elite or for high-level competitive teams. Every club should have the opportunity to offer a quality means of improving.

- Adaptability to clubs

Boats in clubs are not used only by the racing team. They are also used for a range of lessons and activities, which must not be restricted by this device. Moreover, a team can train on different boats. The device must therefore be easy to move so that it can be used on different boats.

- Open Source

This is related to the goal of affordability. The device must be a means of progress, not a potential object of speculation. The opportunity to progress should belong to the athlete, not to a third party likely to monetise it. Placing this project in the public domain is the expression of a desire to make it accessible to all, as well as a guarantee of reproducibility and quality.

- No-nonsense

The focus is on clear and usable results for the debrief, understandable by the coach as well as by the athletes. Its implementation should also be easy. The device must save the team time, not cost it time.

- Multifactorial approach

The measurements must not focus on a single aspect, but on the contrary take into account many aspects of sailing to obtain a representative overall picture. In particular, we want at least data on the wind, on the speed, on the geographical position of the boat and on its attitude in space (linear acceleration, rotational acceleration and orientation along the three axes).

4.4 MIT License

The project is placed in the Open Source domain through the use of the MIT license. It is available alongside my work in the GitHub repository where it is publicly accessible. It is reproduced below.

> MIT License
>
> Copyright (c) 2026 anonyme-sjf-26
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

4.5 Name of the device

In *Demian* by Hermann Hesse, Abraxas is a symbol of the unification of opposites. It does not deny the very principle of opposites, but proposes to overcome the contradictions between them by ceasing to systematically oppose them in one’s thinking, so as to arrive at a more holistic view of a situation. In other words, opposites are not only inseparable, but deeply intertwined. Thus, considering one element without the one opposing it would be nonsense, and the mere act of proposing a boundary between two antagonistic concepts would be arbitrary[16].

In the field of competitive sailing, I have personally often been confronted with an implicit boundary between “classic” racing and “high-level” racing, which is often partly professionalised. While this distinction first seemed rational to me, I later realised that it did not always correspond to a difference in level, but rather to a difference in means. In my experience, moving on to a higher level of racing has often been more a matter of connections and of knowing how to sell oneself, sometimes at the expense of others, than of proving one’s actual level. Initiatives in the field of racing are often taken with only one of these groups in mind, even though neither can function without the other. The high level relies on a pool of athletes in training, and this pool cannot sustain itself without a goal to reach.

For my part, I was lucky enough to join a semi-professional crew (Mercedes-AMG Team Joker 2.0) thanks to the interest the team owner takes in young people. Unfortunately, this kind of opportunity is an exception: the step up to the high level is often made difficult by

[16] *Demian*, op. cit., p.6.

the need to have connections to get into a crew. More than a question of ability, the high level is a social milieu, and, barring exceptions, you practically have to be born into it to be part of it. I do not deny that there are two different worlds, but I do not recognise as legitimate the way in which they are separated. As such, I do not wish to produce a work that treats them as watertight compartments.

It is in reference to the refusal of the way in which these groups are defined, and therefore of considering one independently of the other, and of providing a solution that can only address one of them, that I named my project Abraxas. Both because of its design and its price, it is of interest to competitors of all levels.

Demian being a book that deeply affected me, I am happy to be able to pay tribute to it through my work. It is a way for me to combine two of my passions: sailing and literature.

4.6 Logo of the device

The logo consists mainly of a black circle crossed at 45° by white capsules of varying size. It represents at the same time drops of water on a screen or a lens, the scrolling of data, or boats gliding on the surface of the lake. This circle is accompanied on the right by the word “Abraxas”, written in stylised letters. The letters are exactly the height of the circle. The “X” echoes the style of the capsules in the black circle with its interrupted stroke. The logo was vectorised with the VectorCraft software. It comes in two variants: one with the circle only, and one with the name “Abraxas”.



Figure 3: Logo with the circle only



Figure 4: Logo with the name “Abraxas”

5 FIRST PROTOTYPE

5.1 Hardware

Choosing the hardware was a complicated step. Indeed, I had almost no knowledge of electronics, and choosing components requires thinking about the compatibility of protocols, voltages and pinouts. Moreover, online shops offer hundreds of extremely similar components that are nevertheless not interchangeable. Telling them apart was sometimes a challenge in itself. Not all components are available from the same seller, which forced me to place orders with several different vendors.

I had to reorder hardware several times, because I had not picked the right part or because I was missing some. I expected the main difficulty to lie in the software part, but choosing the hardware turned out to be particularly time-consuming.

Order 1

The first order includes the basic hardware, such as the Raspberry itself, the IMU, the GPS and the camera. The anemometer will only be ordered later.

Component type	Link	Component name
IMU	link	ICM20948, Pimoroni Breakout
Microcomputer	link	Raspberry Pi Zero 2 WH
GPS	link	MTK 3339, Adafruit Ultimate GPS Breakout
Camera	link	Original Raspberry Pi Kamera Module V2
Power boost	link	Adafruit PowerBoost 1000C
Battery	link	LiPO 8000 mAh 2C

Order 2

When placing the first order, I did not know that there were two CSI cable standards. The cable supplied with the camera is a 15-pin CSI, but the input on the Raspberry is a 22-pin mini-CSI. It was therefore necessary to buy another ribbon cable to connect the two components. The detail of this ribbon cable can be seen in Figure 12.

Component type	Link	Component name
CSI ribbon cable	link	CSI (15 pins) ↔ Mini-CSI (22 pins)



Figure 5: Photo of the hardware from the first order before assembly.

Order 3

The purpose of the third order was to add the anemometer. It can be seen in Figure 14. This order was also focused on building a second prototype usable in real conditions. As such, the use of some of the components mentioned is not described in this document.

Component type	Link	Component name
Anemometer	link	HY-WDC2E
Panel-mount USB socket	link	micro-B ↔ C
Panel-mount USB socket	link	A ↔ C
Shutdown management	link	ON/OFF SHIM
USB ↔ RS485	link	WaveShare
USB ↔ RS485	link	SparkFun
Connector	link	Male
Connector	link	Female
Button	link	Metal IP67
USB hub	link	Zero4U

Order 4

Component type	Link	Component name
Active GPS antenna	link	3-5V 28dB 5 Meter SMA
Antenna connector	link	SMA to uFL/u.FL/IPX/IPEX RF
USB cable	link	20cm (cable) + B3 (mini-B) + C1 (micro-B)

Micro-B plugs	link	Bare, two contacts
Converter	link	USB ↔ RS485

5.2 Assembly

All the wiring diagrams in this section were created by me using the Fritzing software.

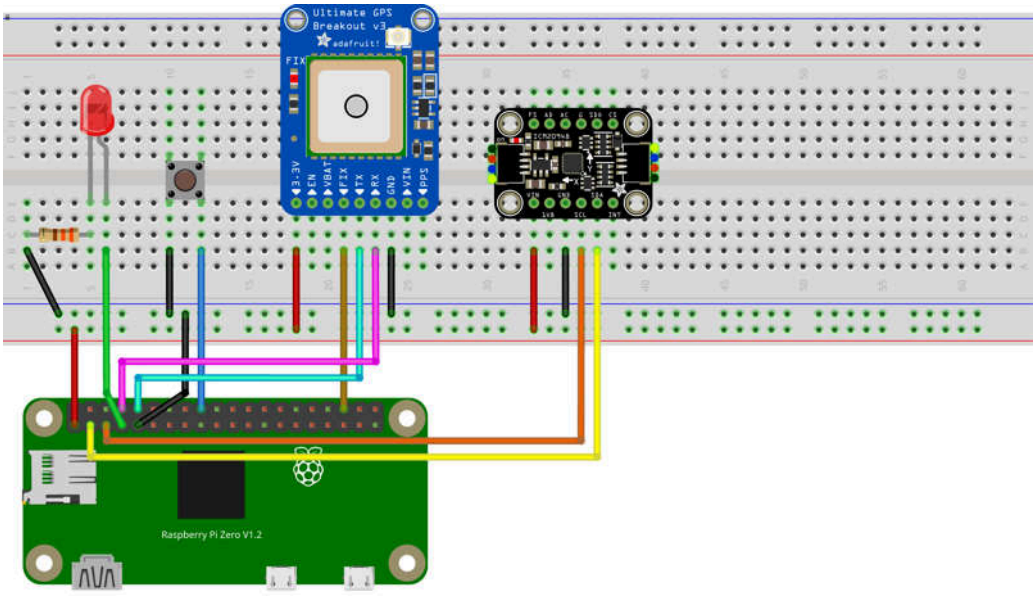


Figure 6: Representation of the assembly carried out.

In this representation, an Adafruit breakout of the ICM20948 was used for the IMU, whereas the breakout actually used is a Pimoroni one. However, the chip used and their pinout are identical, so the only difference is visual. This choice is due to the fact that Pimoroni has not created a graphical representation of its breakout usable in Fritzing.

To carry out the assembly, every pin of the IMU and of the GPS had to be soldered, 14 in total. This meticulous exercise was a major difficulty, because the pins are small and extremely close to each other (2.54mm pitch), yet the solder joints must under no circumstances touch each other. Moreover, the soldering iron must not stay close to the component for too long, otherwise the plastic of the header will melt.

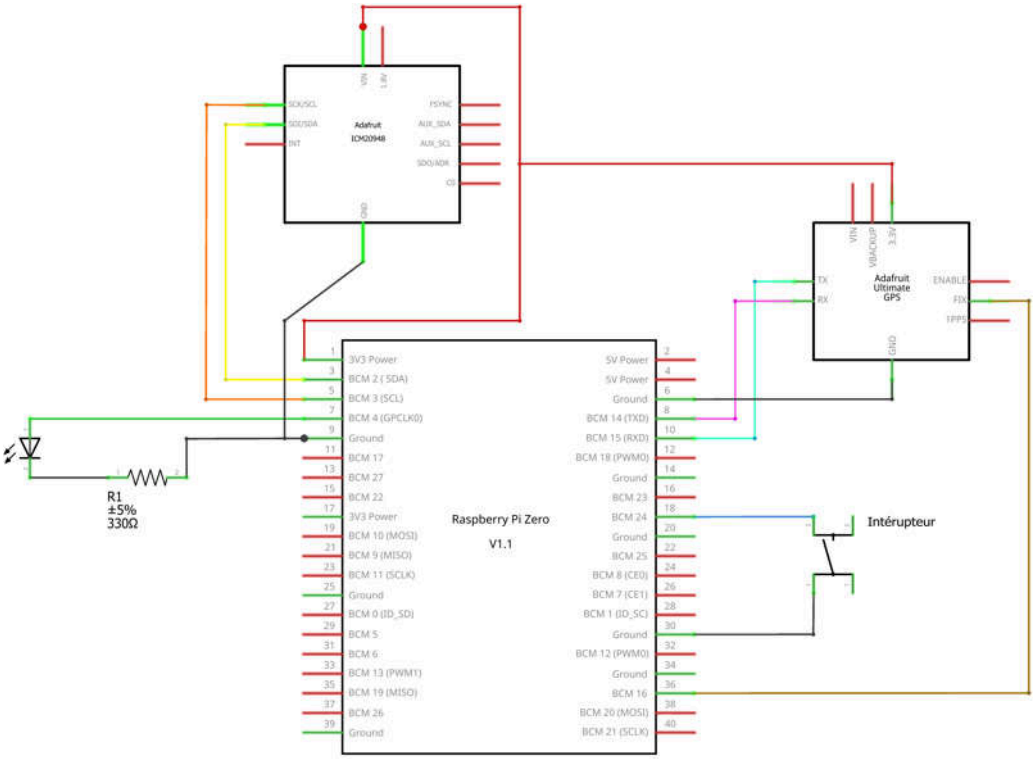


Figure 7: Schematic representation of the assembly carried out.

5.3 Installing the OS

The installed OS is *Raspberry Pi OS Lite (64-bit)*. Without a graphical interface, it was chosen because of the small amount of disk space it uses, its ease of use and installation, the extensive documentation about it, but above all for the low processor load it generates. Indeed, managing resources, which are limited on a *Raspberry Pi Zero 2WH*, is a major challenge of this project.

For its installation, the *Raspberry Pi Imager* software was used. With this software, the OS is downloaded to the computer before being flashed onto the SD card that will serve as the Raspberry's disk. During installation, the user account, the user password, the *root* password, the WiFi (SSID and password) are configured, as well as the use of *SSH* with asymmetric-key authentication (detailed in Section 5.4). Thus, once the SD card is inserted into the Raspberry, everything works immediately; it is not even necessary to connect it to a screen or a keyboard to use it.

5.4 SSH with public keys

SSH is short for *Secure Shell*. This protocol makes it possible to access the Raspberry's console from another computer with the following syntax:

```
batch
ssh user@ip_raspberry
```

Where `user` is the username and `ip_raspberry` is the IP address of the Raspberry (here `192.168.1.13`). By default, when an incoming connection request is detected, the Raspberry asks for the password of the `user` account to grant access to the console. However, this authentication method is not the most secure, because a password can be cracked by brute force. For this reason, asymmetric encryption was chosen.

The reason for using SSH for remote access instead of more intuitive higher-level systems is its low resource usage, its broad compatibility, its optimal security and its proven stability, precisely because it is a low-level process. Indeed, these processes are much more permissive and configurable in the case of development projects, and have the advantage of not requiring any additional download[17].

Asymmetric encryption

This method can be enabled after installing the OS, but for the sake of simplicity, it was configured directly with *Raspberry Pi Imager*. To connect via *SSH* in this way, a public key and a private key are needed on each device. These keys are generated using

```
batch
ssh-keygen
```

on the computer from which we want to access the Raspberry. This command works on Linux as well as on MacOS and Windows. In all three cases, we follow the instructions given by the console / terminal without entering any further options, so as to keep the default configuration. At the end of this operation, we obtain a public key and a private key, stored respectively in `.ssh/id_rsa.pub` and in `.ssh/id_rsa`. When configuring the OS with *Raspberry Pi Imager*, the public key is entered. The keys on the Raspberry are created automatically. Thus, from the Raspberry's very first *boot*, it can be accessed remotely without entering a password, provided that we are connected to the same network.

NAT loopback and port mapping

In order to access the Raspberry from outside the network it is connected to, an operation must be carried out on the router (often `192.168.1.1`). The default SSH port is 22. Consequently, the goal is to open port 22 of the router and redirect the information arriving on it to the Raspberry. The procedure depends on the router, but it can usually be accessed from a browser by typing `192.168.1.1` in the search bar.

[17] FAULKNER Andrew and BLANC Florian, "Avantages et faiblesses du protocole SSH", *Hakin9*, n°2, 2008, https://chk.me/qm9L8t2 (accessed on 21.10.2026).

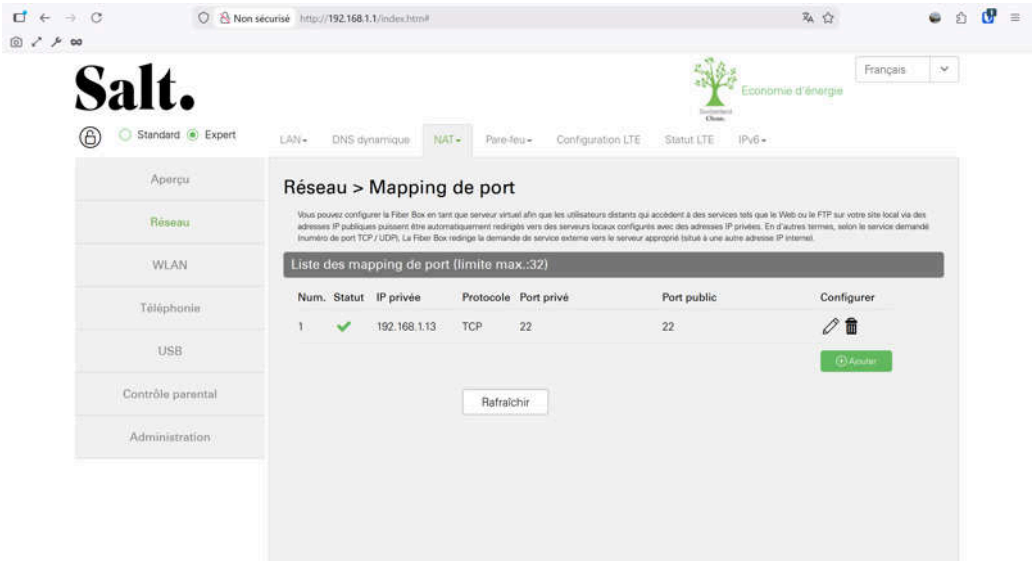


Figure 8: Port mapping configuration page on a Salt router.

To access the Raspberry from outside once port mapping is set up, simply run

```
batch
ssh user@public_ip_box
```

The `public_ip_box` is the IP address of the network as seen from outside. It can easily be found with websites such as https://www.myip.com/.

Next, NAT loopback must be enabled so that the computer can connect to the Raspberry from the same network, even though the preferred entry point faces outwards. NAT loopback allows the router to interpret requests coming from the network it generates as requests coming from outside. That is to say, a connection request addressed to its public IP from inside the network cannot be interpreted correctly without NAT loopback.

Change of the Raspberry's public key

This key sometimes changes automatically. The new key will therefore differ from the one saved in `.ssh/known_hosts`. This difference raises the following alarming error:

```
bash
@@@
```

```
Host key for 192.168.1.13 has changed and you have requested strict checking.
Host key verification failed.
```

To get rid of this error, run:

```
batch
ssh-keygen -R 192.168.1.13
```

This command removes all the public keys in `.ssh/known_hosts` related to this IP address. This allows a new connection, which must be confirmed by typing `y` when prompted. WARNING! Only run this command if you are certain that no one is conducting a MITM attack, as this would open the door of the network to them.

5.5 File transfer

Throughout this process, it will be necessary several times to transfer Python code to the Raspberry. There are several methods for doing so; we will present two of them:

- With SCP:

The Python file is saved on the remote computer. It is then copied to the Raspberry from the remote computer using SCP, an SSH protocol, with the following syntax:

```
batch
scp  absolute\path\to\source\script.py  user@192.168.1.13:/home/user/destination/
script.py
```

Be careful to work with absolute paths and not relative ones. On Windows, the "`/`" in paths are reversed.

- With `nano`:

Directly on the Raspberry, run:

```
bash
cd /home/user/destination
nano script.py
```

Then paste the code into the text editor that opens. Such a copy-paste is only possible if you are connected to the Raspberry via SSH; if not, it is better to use SCP. To save, press `Ctrl+X`, then `y` to confirm, and `Enter` to exit `nano`.

In practice

As part of this thesis, I created a `batch` script using SCP to directly transfer all the software files (presented in Section 6). Thus, after each modification, I simply had to run it to automatically update every file on my Raspberry.

```batch
scp -r "C:\Users\user\Documents\GitHub\Abraxas\*.py"
        user@51.154.XXX.XXX:/home/user
```

5.6 Packages and libraries

We start by doing the standard updates and installing `tree`, `pip` and `python3-full`:

```bash
sudo apt-get update
sudo apt-get -y upgrade
sudo apt install tree
sudo apt install python3-pip
sudo apt install python3-full
```

All other packages that may be needed later can simply be installed in the same way.

To install Python libraries, `pip` is used. For example, we need to install `time`:

```bash
pip3 install time
```

If `pip` returns the following error:

```bash
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    For more information visit http://rptl.io/venv

note: If you believe this is a mistake, please contact your Python installation
or OS distribution provider. You can override this, at the risk of breaking your
Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

An option can be added at the end of the line to force the installation:

```bash
pip3 install time --break-system-packages
```

5.7 Testing the GPIO

To carry out a first test, we want to make an LED connected to GPIO 7, i.e. BCM 4, blink. We create the file `blink.py` with the following code on the Raspberry (for example in `/home/user`):

```python
import RPi.GPIO as GPIO
import time

LED_PIN = 4  # GPIO 7 = BCM 4

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(LED_PIN, GPIO.LOW)
        time.sleep(0.5)

except KeyboardInterrupt:
    pass

finally:
    GPIO.cleanup()
```

We then install the `RPi.GPIO` library on the Raspberry:

```bash
pip3 install RPi.GPIO
```

Finally, we run the script on the Raspberry with:

```bash
python3 blink.py
```

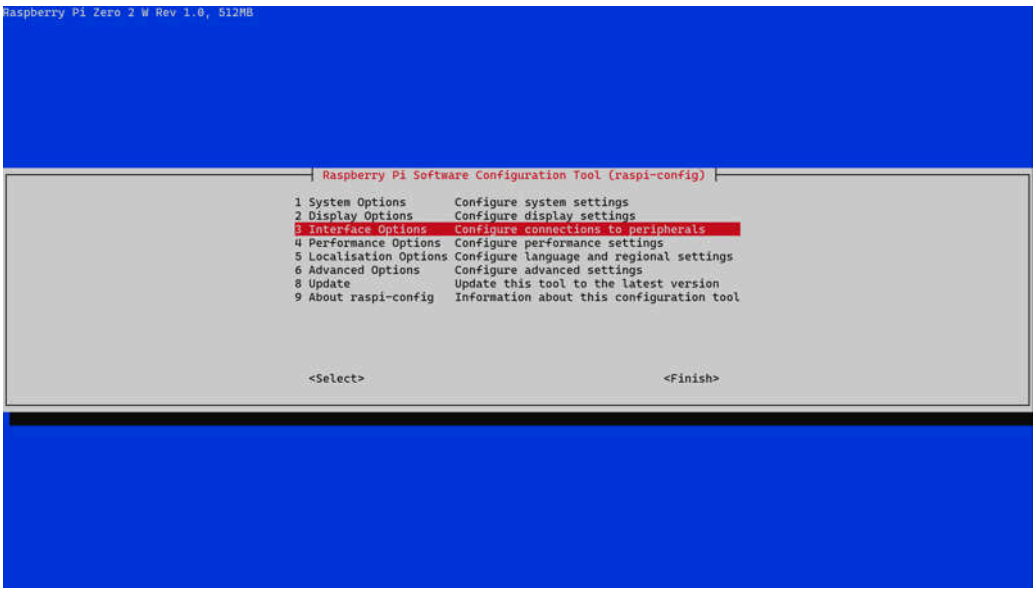
We observe the LED blinking with a period of half a second; it can be stopped with a `KeyboardInterrupt` (`Ctrl+C`).

5.8 Serial Port setting

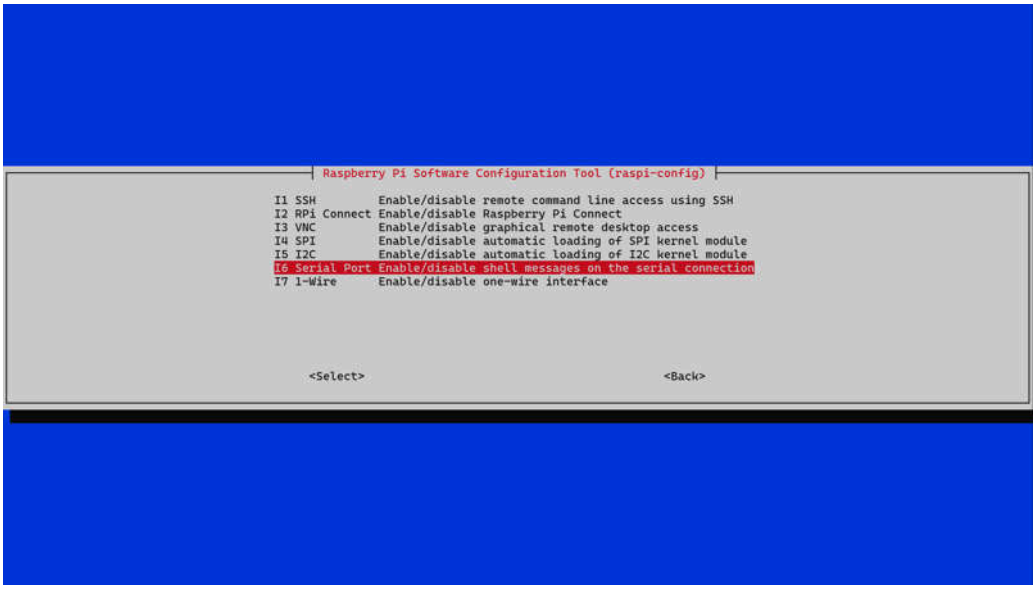
To allow the UART port (TxD and RxD) to work properly, the serial port must be configured: we want to disable the `serial login shell` and enable the `serial interface`. We run:

```
bash
sudo raspi-config
```

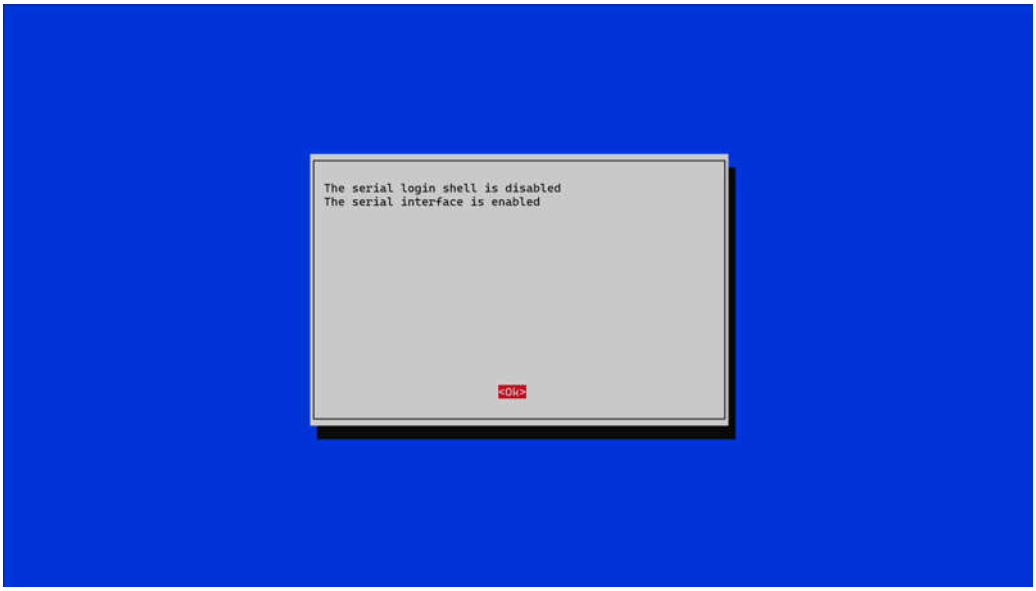
Next, select Interface Option



Then Serial Port



Choose `no` for the `serial login shell` and `yes` for the `serial interface`. You should get a window like this:



Confirm, then select `finish`, and for the changes to take effect (if a reboot is not already offered when exiting `raspi-config`):

```bash
sudo reboot
```

The Raspberry will restart and the SSH connection will close, but it will be possible to reconnect after a few minutes.

5.9 Using the IMU

The 9DoF IMU works over an I²C bus, i.e. with the SDA (data) and SCL (clock) ports[18]. To initialise it, we install the libraries, packages and scripts from the GitHub repository[19]. Then, we run the initialisation script `install.sh`. To do this, `git` needs to be installed:

```bash
sudo apt install git
```

We then run the following commands:

```bash
cd /home/user
git clone https://github.com/pimoroni/icm20948-python
cd /home/user/icm20948-python
./install.sh
```

[18] HOWARD Philip *et al.*, "icm20948-python", *GitHub*, 28.06.2024 (last commit), https://github.com/pimoroni/icm20948-python (accessed on 21.10.2026).
[19] Ibid.

During the installation, we are asked whether we want to download examples. This can be useful to test that the IMU works properly. We confirm with `y`. However, we decline the documentation with `n`: we do not want to clutter the Raspberry with it, and it is available in the repository. We still need to install the `icm20948` library from PyPi:

```bash
pip3 install icm20948
```

We test the IMU by running the example script `read-all.py`:

```bash
python3 /home/user/icm20948-python/examples/read-all.py
```

You should see blocks of information like this scrolling by:

```python
Accel: -0.16 00.00 01.04
Gyro:  -1.00 01.84 -0.75
Mag:   29.85 -41.25 128.85
```

To stop it, press `KeyboardInterrupt` (`Ctrl+C`).

5.10 Using the GPS

Before starting to initialise the GPS, make sure that the Serial Port setting (Section 5.8) has been carried out. The GPS works over asynchronous UART, i.e. with the RxD and TxD ports[20]. Unlike the I²C bus, this is not a master/slave architecture, but a point-to-point process, allowing more complex interactions between the Raspberry and the GPS.

To initialise the GPS, the following guides were used:

- GPS user manual (PDF)[21]
- Running CircuitPython code without a CircuitPython interpreter[22]
- Installing Blinka on a Raspberry Pi[23]

Some of the libraries, packages and test scripts come from the GPS repository[24]. The exact installation procedure had to be adapted to the specific needs of the project. It is therefore transcribed here in full.

[20] HERRADA Eva *et al.*, “Adafruit_CircuitPython_GPS”, *GitHub*, 09.09.2026 (last commit), https://github.com/adafruit/Adafruit_CircuitPython_GPS (accessed on 21.10.2026).

[21] HERRADA Eva, “Adafruit Ultimate GPS”, *Adafruit*, 20.08.2026 (last updated), https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps.pdf (accessed on 21.10.2026).

[22] HERRADA Eva and LEBLANC Williams, “Running CircuitPython Code without CircuitPython”, *Adafruit*, 05.07.2023 (last major update), https://chk.me/O2xfxk0 (accessed on 21.10.2026).

[23] HERRADA Eva and LEBLANC Williams, “Installing Blinka on Raspberry Pi”, *Adafruit*, 05.07.2023 (last major update), https://chk.me/Y40lkS7 (accessed on 21.10.2026).

[24] “Adafruit_CircuitPython_GPS”, op. cit., p.33.

The *Adafruit Ultimate GPS Breakout* works according to a CircuitPython logic, a lightweight version of Python. To be able to run it on a Raspberry (which natively lacks CircuitPython), an interpreter must be installed: Blinka.

Installing Blinka

We start by upgrading the `setuptools`:

```bash
sudo apt install --upgrade python3-setuptools
```

At this point, the Raspberry may ask for a reboot. We do it if necessary, then install a virtual environment to run Blinka in.

```bash
cd
sudo apt install python3-venv
python3 -m venv env --system-site-packages
```

Then we activate it:

```
bash
source env/bin/activate
```

We then proceed with the automatic installation of Blinka using a script provided by Adafruit:

```
bash
cd
pip3 install --upgrade adafruit-python-shell
wget https://raw.githubusercontent.com/adafruit/Raspberry-Pi-Installer-Scripts/
                                                        master/raspi-blinka.py
sudo -E env PATH=$PATH python3 raspi-blinka.py
```

The process will ask for confirmation, which we give by typing `y`. The installation takes a few minutes, at the end of which the Raspberry asks for a reboot. We accept by typing `y` again.

We create the file `blinkatest.py` (for example in `/home/user`) containing the following code:

```
python
import board
import digitalio
import busio

print("Hello, blinka!")

# Try to create a Digital input
pin = digitalio.DigitalInOut(board.D4)
print("Digital IO ok!")

# Try to create an I2C device
i2c = busio.I2C(board.SCL, board.SDA)
print("I2C ok!")

# Try to create an SPI device
spi = busio.SPI(board.SCLK, board.MOSI, board.MISO)
print("SPI ok!")

print("done!")
```

We then run the Python script to test Blinka:

```
bash
source env/bin/activate
python3 blinkatest.py
```

We should get:

```
bash
Hello, blinka!
Digital IO ok!
I2C ok!
SPI ok!
done!
```

Special precaution

CircuitPython contains a library named `board`, yet Python has a library with the same name. Trying to install the Python library system-wide would break Blinka irreparably, and a blank OS would have to be flashed onto the SD card again to restart the whole installation. I learned this the hard way. If the following error is raised:

```
bash
Traceback (most recent call last):
  File "/home/user/blinkatest.py", line 1, in <module>
    import board
ModuleNotFoundError: No module named 'board'
```

Do not under any circumstances try to fix it by importing `board`. The problem is not that this library is not imported: it is indeed present, but only in CircuitPython in Blinka's `venv`. To get rid of this error, check that you are in the intended virtual environment, and if necessary enter it again with

```
bash
source env/bin/activate
```

If Python's `board` is needed later (although it is preferable to find another way), make sure to confine its installation to a virtual environment separate from Blinka.

Installing the GPS library

First, `adafruit-circuitpython-gps` must be installed (some `DEPRECATION` warnings may be raised, but they are harmless):

```bash
sudo pip3 install adafruit-circuitpython-gps --break-system-packages
                                             --root-user-action=ignore
```

The `--root-user-action` option with the `ignore` attribute avoids the following error when running `pip` with `sudo`:

```bash
WARNING: Running pip as the 'root' user can result in broken permissions and
conflicting behaviour with the system package manager, possibly rendering your
system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.
```

Testing the GPS

We create the file `GPS-test.py` (for example in `/home/user`) with the following code[25]:

```python
# SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
# SPDX-License-Identifier: MIT
# Will wait for a fix and print a message every second with the current location
and other details.
import time
import board
import busio
import adafruit_gps
import serial

# Create a serial connection for the GPS connection using default speed
rx = board.RX
tx = board.TX
uart = serial.Serial("/dev/serial0", baudrate=9600, timeout=10)

# Create a GPS module instance.
gps = adafruit_gps.GPS(uart, debug=False)

# Turn on the basic GGA and RMC info
gps.send_command(b"PMTK314,0,1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0")

# Set update rate to once a second (1hz)
gps.send_command(b"PMTK220,1000")
```

[25] Modified version of /exemples/gps_simpletest.py in Adafruit_CircuitPython_GPS, op. cit., p.33.

```
# Main loop runs forever printing the location, etc. every second.
last_print = time.monotonic()
while True:
    gps.update()
    # Every second print out current location details if there's a fix.
    current = time.monotonic()
    if current - last_print >= 1.0:
        last_print = current
        if not gps.has_fix:
            # Try again if we don't have a fix yet.
            print("Waiting for fix...")
            continue
        # If we have a fix (gps.has_fix is true)
        print("=" * 40)
        print(
            "Fix timestamp: {}/{}/{} {:02}:{:02}:{:02}".format( # noqa: UP032
            gps.timestamp_utc.tm_mon,
            gps.timestamp_utc.tm_mday,
            gps.timestamp_utc.tm_year,
            gps.timestamp_utc.tm_hour,
            gps.timestamp_utc.tm_min,
            gps.timestamp_utc.tm_sec,
            )
          )
# gps.send_command(b'PMTK220,500')
print(f"Latitude: {gps.latitude:.6f} degrees")
        print(f"Longitude: {gps.longitude:.6f} degrees")
                      print(f"Precise   Latitude:   {gps.latitude_degrees}   degs,
{gps.latitude_minutes:2.4f} mins")
                    print(f"Precise   Longitude:   {gps.longitude_degrees}   degs,
{gps.longitude_minutes:2.4f} mins")
        print(f"Fix quality: {gps.fix_quality}")
        if gps.satellites is not None:
            print(f"# satellites: {gps.satellites}")
        if gps.altitude_m is not None:
            print(f"Altitude: {gps.altitude_m} meters")
        if gps.speed_knots is not None:
            print(f"Speed: {gps.speed_knots} knots")
        if gps.speed_kmh is not None:
            print(f"Speed: {gps.speed_kmh} km/h")
        if gps.track_angle_deg is not None:
            print(f"Track angle: {gps.track_angle_deg} degrees")
        if gps.horizontal_dilution is not None:
            print(f"Horizontal dilution: {gps.horizontal_dilution}")
        if gps.height_geoid is not None:
            print(f"Height geoid: {gps.height_geoid} meters")
```

To run it:

```bash
python3 GPS-test.py
```

For the GPS to acquire a first fix, it may be necessary to leave it for several minutes (or even hours) near a window or outside, with the antenna pointing towards the sky. The time required varies depending on the physical environment, the weather, and the presence of disruptive factors such as a high density of radio waves. Once the GPS is connected, you should see blocks of information like this scrolling by:

```python
Fix timestamp: 2/9/2026 20:26:21
Latitude: 46.188553 degrees
Longitude: 6.145146 degrees
Precise Latitude: 46 degs, 11.3132 mins
Precise Longitude: 6 degs, 8.7088 mins
Fix quality: 1
# satellites: 4
Altitude: 107.4 meters
Speed: 0.93 knots
Speed: 1.79 km/h
Track angle: 355.59 degrees
Horizontal dilution: 3.43
Height geoid: 48.2 meters
```

The version published by Adafruit on GitHub[26] from which this code was adapted was originally intended for microcontrollers with CircuitPython natively installed. The fact that the code is interpreted by Blinka required a few adjustments to create a match at the pinout level.

These adjustments were a major difficulty, because the question of virtual environments, of the difference between CircuitPython and Python, of communication protocols and of physical ports is very new to me. Yet these modifications brought all these issues together. Moreover, while Blinka and CircuitPython are well documented, the question of the correspondence at the physical level remains vaguer, and the information available, from amateur sources, is often unclear, outdated, or simply wrong.

5.11 Using the camera

Initialising the camera is a step I had poorly anticipated. To connect it to the Raspberry, I had planned a standard CSI cable, but it turned out that the CSI ribbon connector of the *Raspberry Pi Zero 2* is special. It does not have 15 pins (like most CSI connectors), but 22. A new specific cable therefore had to be obtained. Had I researched further at the time of the initial prototyping, I could have avoided this.

[26] Ibid.

Once the appropriate hardware is available, the physical installation is easy. No soldering is required. Simply open the CSI ports of the camera and of the Raspberry by gently pulling the black latch, then insert the ribbon cable into the port. The contacts of the ribbon cable must face those of the port. The latches are then closed, and the cable is locked.

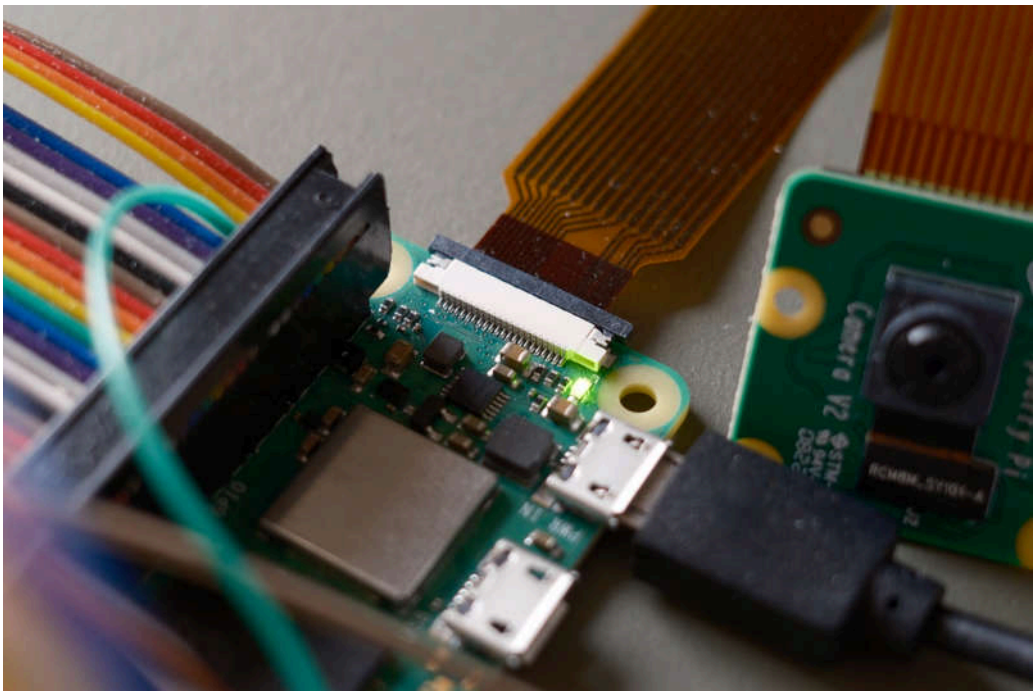


Figure 12: The golden CSI ribbon cable goes into the port on the Raspberry

Once the connection is made, no additional software adjustment is needed. The camera can be used right away. We test it with:

```bash
rpicam-still -o test.jpg --nopreview
```

After a five-second delay during which the image would be displayed if we had a graphical interface, the image is captured and saved in the current directory as `test.jpg`. We delete it so as not to clutter the storage.

```bash
rm test.jpg
```

5.12 Using the anemometer

Choice of anemometer

Before choosing the anemometer, I had to find out about the different types of devices available. The most common are cup anemometers, known as Robinson anemometers[27]. They consist of three cups mounted on a central rotating shaft. Their operation relies on

[27] "Anémomètre" in *Wikipedia*, n.d., https://chk.me/XaOHDIC (accessed on 21.10.2026).

extrapolating the measured rotational speed of the cups to determine the wind speed. In order to measure the wind direction, a wind vane must be added[28]. This sensor system has the advantage of being widely available commercially and inexpensive.

However, it consists of several moving parts, which makes it more complex to install on a boat and less durable over time[29]. Moreover, because of the inertia of its parts, it is not able to record brief peaks in the values: the resolution of the measured values is low by design.



Figure 13: “Cup anemometer”, *Wikipedia*[30].

To overcome these two problems, I chose an ultrasonic anemometer. It measures the speed of the air mass it is in, along two axes, by analysing the difference between the outbound and return travel times of sound. This type of anemometer has the advantage of having no moving parts and of working even in turbulent wind, thanks to the high resolution its design allows[31].

[28] Ibid.
[29] Ibid.
[30] Ibid.
[31] Ibid.

However, it is less common outside professional or industrial applications, and is therefore harder to find. As a result, it is also more expensive. Nevertheless, this is the type of anemometer I chose.

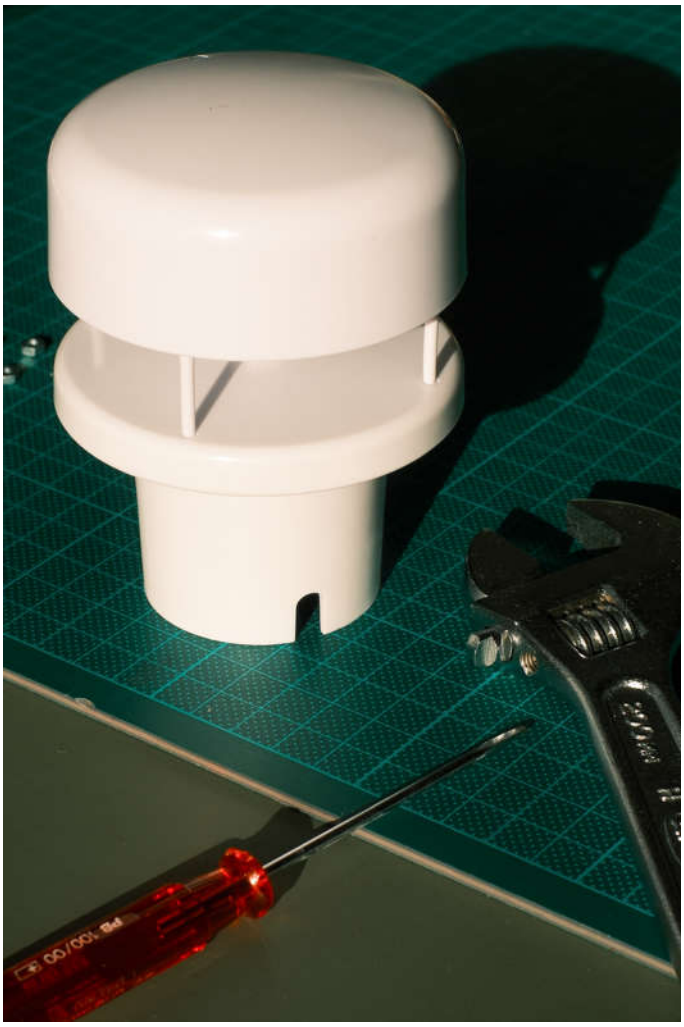


Figure 14: HY-WDC2E ultrasonic anemometer

Connection to the prototype

Since this type of anemometer is more common in the industrial field, the communication interfaces used are different from those used in small projects. The anemometer I chose can communicate either over RS232 or over RS485, two interfaces working with 12V voltage levels. They are incompatible with the GPIO of the Raspberry and of other development boards, which work with 3.3V logic. Connecting them directly would immediately destroy the board. I learned this the hard way: shortly before the start of the school year, I lost my Raspberry by connecting the anemometer directly to it. I had to order a new one and start the whole configuration process all over again.

To make communication possible, I use an RS485 to USB conversion module, a SparkFun breakout. The anemometer's wires are soldered onto the breakout, then the breakout itself is connected to the Raspberry by a USB cable. Once the datasheets have been read properly, the installation is easy.

The information is transmitted in NMEA 0183 sentences[32], the same protocol used by the GPS. The anemometer does not need any particular configuration: it sends unsolicited outputs on an open USB port (`ttyUSB0` in my project). To test it, we simply read the input of this port:

```bash
cat /dev/ttyUSB0
```

Outputs following this pattern should appear:

```bash
$WIMWV,180,R,5.12,M,A*06<CR><LF>
```

The 180 corresponds to the wind direction in degrees, and the 5.12 to its speed in $m \cdot s^{-1}$. The rest of the sentence contains the parameters (here the factory ones) and the checksum. To use this data in Python code, all that remains is to parse it.

5.13 SQL database

To record the IMU and GPS data, an SQLite database is chosen. It makes it possible to save information quickly and simply from Python. Moreover, such a database has the advantage of being very robust while enforcing an organisation of the data (which a JSON file, for example, would not do)[33].

Three tables are defined: `sessions`, `imu_data` and `gps_data`. Each table has an `id` primary key in addition to the other fields. Moreover, the `imu_data` and `gps_data` tables have a foreign key linking each of their records to a session (a record of the `sessions` table). This way, it will be easy to separate the measurements of the different training sessions.

`gps_data` and `imu_data` are separate because their records are asynchronous. Indeed, the IMU and GPS data cannot be recorded in the same table if the information received from them does not arrive at the same time.

[32] "User Manual", *Hongyuv*, 30.04.2024, https://www.hongyuv.com/download/WDC2E_Manual.pdf (accessed on 21.10.2026).

[33] EYGAY Erwan, "JSON ou SQL : Quel format choisir pour gérer vos données ?", *DataBird*, 19.03.2026 (last updated), www.data-bird.co/blog/json-ou-sql (accessed on 21.10.2026).

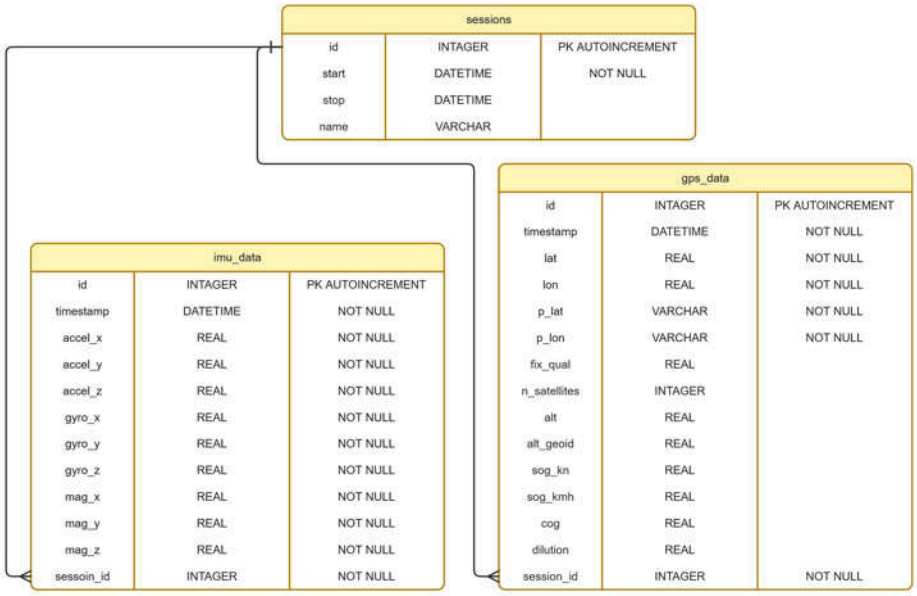


Figure 15: Diagram of the relational database `sys.db`.
Personal diagram made with Miro.

Before even starting to work with a local database, `sqlite` must be installed.

```bash
sudo apt install sqlite3
```

To build the database whose diagram is shown above, we start by creating the file `sys.db`. We then open it with `sqlite`.

```bash
touch sys.db
sqlite3 sys.db
```

We then build the first table by sending the following SQL query:

```SQL
CREATE TABLE sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

    start DATETIME NOT NULL,
    stop DATETIME,

    name VARCHAR);
```

For each field, the name, the type and a possible constraint have been specified. We do the same for the other tables.

```SQL
CREATE TABLE imu_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME NOT NULL,

    accel_x REAL NOT NULL,
    accel_y REAL NOT NULL,
    accel_z REAL NOT NULL,
    gyro_x REAL NOT NULL,
    gyro_y REAL NOT NULL,
    gyro_z REAL NOT NULL,
    mag_x REAL NOT NULL,
    mag_y REAL NOT NULL,
    mag_z REAL NOT NULL,

    session_id INTEGER NOT NULL);

CREATE TABLE gps_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME NOT NULL,

    lat REAL NOT NULL,
    lon REAL NOT NULL,
    p_lat VARCHAR NOT NULL,
    p_lon VARCHAR NOT NULL,
    fix_qual REAL,
    n_satellites INTEGER,
    alt REAL,
    alt_geoid REAL,
    sog_kn REAL NOT NULL,
    sog_kmh REAL,
    cog REAL NOT NULL,

    dilution REAL,
    session_id INTEGER NOT NULL);
```

Finally, we exit the database (saving is automatic).

```SQL
.exit
```

To view the contents of a table (e.g. `imu_data`) visually, we can run:

```bash
sqlite3 sys.db -header -column "SELECT * FROM imu_data;"
```

6 SOFTWARE OF THE FIRST PROTOTYPE

The code itself can be found in the appendix. The commentary is given here.

The prototype works as follows: a button is pressed to start a session. Recording from the IMU, the GPS and the camera begins. The same button is pressed a second time to stop the session. The recordings are finalised and the prototype remains on standby, ready for a new session.

To do this, we proceed as follows: a Python script `interface.py` monitors the button and, on each press, either starts the `master.py` script or stops it cleanly. This "master" script is the one that manages the whole course of the session. It launches `gps.py`, `imu.py` and `vid.py`, which manage the GPS, IMU and camera recordings respectively. When it shuts down, it stops these three processes and launches `grp.py`, whose purpose is to finalise the video recording and turn the raw information into a readable `.mp4` file.

All these files rely on a common class file, `classes.py`. The logs are saved in a `sys.log` file.

For the overall structure, I chose to separate the different tasks into distinct processes rather than threads of a single process. I do not think there is a "right" or "wrong" answer to this question, but after carefully weighing the pros and cons, I chose this option because of the excellent isolation it offers, which is useful if one of the processes were to crash. However, this method greatly complicates communication between the different tasks, as processes force the use of pipes.

6.1 Class file — classes.py

Many functions and much code are common to several of the files used. To avoid rewriting these parts in each file, a class file is used. It simply needs to be imported in every other file with `from classes import *`

In this file, logging is configured in particular. Thus, every time a log is made, whatever the file, it will be formatted properly, then printed in the console with the appropriate colour and saved in the log file `sys.log`.

The `SysDataBase` class, which will be used to interact with the `sys.db` database, is also defined. It makes it possible to use `cursor` without having to initialise it manually, to easily access the identifier of the current session, and to cleanly finalise a recording.

Finally, there is also the definition of the decorator function `non_blocking`. It takes a function as an argument and returns its non-blocking version (running in a separate thread). It is used to modify the emergency reboot function, also defined in this file.

I did not create this file from the start, but as I was coding and seeing the repetitions pile up, it became obvious that it was necessary.

Logging library

In order to produce our logs, we need `colorlog`. It can be installed as follows:

```
bash
sudo apt install python3-colorlog
```

6.2 Button monitoring — interface.py

The button is connected between pin 18 (BCM24) and ground. There is no need to add a resistor because GPIO 24 is set to internal pull-up.

We start by creating an instance of the `Button` class, which will be used to monitor the pin. The `when_pressed` event of this instance is defined as a daemon. It waits in the background. When it wakes up, it calls the `bouton_pressed` callback and then dies immediately.

When the `bouton_pressed` function runs, the global flag `witness_button` is set to `True` and the callback created by the daemon dies quickly (no blocking action can be executed inside it).

The flipping of the flag is detected by the infinite loop and triggers, depending on the state of the boolean `session_is_running`, either the launch of `master.py` or the sending of the `SIGUSR1` signal, which will be intercepted by `master.py` and cause it to shut down. The boolean `session_is_running` is then inverted so that the action executed on the next press is correct, a `time.sleep(1)` is run to avoid bouncing, and then the flag `witness_button` goes back to `False`.

The lines following the loop handle errors. A reboot is triggered in the event of an unexpected interruption of the script, because the prototype is of no use if it is no longer running.

In a previous version, the entire content of the loop was contained in the `bouton_pressed()` event, but the callback created to run it is not designed for blocking tasks and would terminate before the program it contained had finished running. This resulted in unpredictable behaviour that was particularly complex to debug because it stemmed from side effects, such as booleans being inverted for no apparent reason.

Automatic execution

In order for `interface.py` to run automatically when the Raspberry starts up, `cron` is used. It would have been more usual to use `systemctl`, but such a service is much more complex to configure, and being run in an outer ring at boot is not a problem in our case.

We must start by making our file executable, then open the `cron` editor:

```
bash
sudo chmod +x interface.py
crontab -e
```

When opened for the first time, `cron` will ask which editor we want to use. We choose `nano` by typing 1; it is the simplest in our situation. Then, we create the following job:

```
cron
@reboot python3 /home/user/interface.py
```

We confirm and exit with `Ctrl+X`, then `y` to confirm, and `Enter` to exit `nano`.

6.3 Session management — master.py

The interception of the `SIGUSR1` signal is handled on line 12. It triggers the `handle_usr1` callback, which sets the global flag `ending_session` to `True`. This way, the callback remains short and non-blocking, which is generally good practice.

This script will first create an instance of the `SysDataBase` class in order to interact with the database (in particular to retrieve the identifier of the last session). The first concrete action is to create the new session. Its identifier is determined automatically with `AUTOINCREMENT` directly in SQL. The class function `actualise_session_id()` adds 1 to the counter in order to go from the identifier of the previous session to that of the one that has just been created.

The creation of the session is directly followed by the launch of the scripts `imu.py`, `gps.py` and `vid.py`, which will read and record the sensor data. The session identifier is passed as an argument to `vid.py` because, since it takes a little longer to start, the program might retrieve the wrong identifier if sessions are very close together. Giving it the same session identifier as the master avoids any confusion.

Once the processes are launched, we wait to receive a signal with `signal.pause` before continuing. To avoid terminating the master upon receiving a signal other than the one sent by `interface.py`, we only continue if the global flag `ending_session` is true (i.e. if the `handle_usr1` callback has been triggered by the receipt of `SIGUSR1`, the signal sent by `interface.py`).

To shut down the subprocesses, the `shutdown()` function is used. It sends each of them a `SIGTERM` to terminate them. If this is not enough and they are still alive after 5 seconds, it kills them with `SIGKILL`.

We then finish recording the session by completing the SQL table with the end time, then close it using the class function provided for this purpose.

Finally, `grp.py` is launched so that it properly finalises the video recording. Its execution may be slow, but it runs in the background and a new master can be launched directly by `interface.py` to start a new session.

As with `interface.py`, this script was only needed to coordinate the others, and for this reason it was one of the last to be written. None of the syntax in this script is particularly complex, but it is the cornerstone of the code's logic and architecture. As such, it required considerable prior thought.

6.4 IMU and GPS management — imu.py & gps.py

These two scripts, although distinct, work in exactly the same way. They are both based on the code provided by the manufacturer and presented in Section 5.9 and Section 5.10.

First of all, there is the handling of stop signals. When a `SIGTERM` or a `SIGINT` is received, the `stop_handler` callback is called to set the global flag `running` to `False`, which will cleanly interrupt the loop that reads and records the data.

Next comes the `acquisition()` function, containing the manufacturer's code for retrieving the data. We then define an instance of the `SysDataBase` class for recording the data, the frequency and the period, as well as other variables as in the manufacturer's code.

In the loop, we start by acquiring the data with the function provided for this purpose, then make a record in the database. We then wait for the time remaining before the next record (frequency - time spent on this record).

The write is only committed once per second so as not to overload the Raspberry with unnecessary operations. To do this, a counter is used. When it reaches the frequency, one second has elapsed, the commit is made and the counter is reset.

Finally, we try to commit the last records and then close the connection. These two operations are carried out by the class function `terminate_database_connexion()`.

These two scripts are the result of reworking other code. They are the first ones I wrote for my thesis and have evolved a great deal. From a simple reading of data printed in the console, they have become a recording system interweaving Python and SQL queries.

Although less complex than some of the other scripts, they served as laboratories for syntax that was unknown to me, such as logging, SQL queries, callbacks, signals and the try/except/finally structure. I had never done exception handling before, and working with physical sensors is demanding in this respect, since the inputs are by definition not controlled.

6.5 Video recording — vid.py

We start by retrieving the session identifier that was passed with `subprocess.Popen`. If the recording were launched independently of the master, the identifier would be retrieved from the database.

We create the folders in which the videos will be saved. The destination folder is named after the session identifier and is located in the `sessions` folder.

Before starting to record, we create a thread whose role is to monitor the creation of the first video segments. This way, we get the most precise timestamp possible for the start of the video. A slight offset may remain because what is monitored is not the start of the shooting but the start of the recording. Unfortunately, it is not possible to obtain a more precise indication of the timestamp of the start of the video. The thread is defined as a daemon so that it does not survive the death of its parent program.

We then use `subprocess.Popen` to run the `bash` command that starts the video recording. This command is taken from the `rpicam-vid` documentation[34]. The video is recorded in two-second segments in raw H264. This way, the risk of obtaining a damaged video is at its lowest. Indeed, complex formats such as MP4 require final encoding, whereas H264 remains readable even if the recording is abruptly interrupted without proper finalisation.

The code then waits to receive the shutdown signal. Once received, it tries to end the video recording by sending `SIGTERM`. If the process is still alive after four seconds, it sends `SIGKILL` to kill it.

If a connection to the database had been opened, it is finally closed.

The video recording code was probably the most technically complex for me to write.

Indeed, it was disorienting for me not to be able to start a video without specifying its duration in advance. It took me a while to understand that I actually had to start several short ones of a defined duration to make up a long one.

Moreover, I initially tried to record in MP4 format, but the files I obtained were systematically corrupted. Even after switching to raw H264, I still had the problem that the first frame of each sequence did not coincide with the last frame of the previous sequence. This problem was solved by adding `--inline`, which clearly marks the start and the end of each sequence.

[34] "Camera software", *Raspberry Pi*, n.d., https://www.raspberrypi.com/documentation/computers/camera_software.html (accessed on 21.10.2026).



Figure 16: Photo taken on the lake during the Psaros 2026 critérium in Vidy with a resolution close to that of the camera used

Finally, I had a lot of difficulty obtaining a precise timestamp for the start of the video, since `rpicam-vid` does not provide one by itself and there is almost a two-second delay between the execution of the command and the actual start of the video. The solution I adopted is not perfect, but I do not have a better one for the moment.

6.6 Finalising the video recording — grp.py

As with `vid.py`, we try to retrieve the session identifier via a system argument passed from `master.py`. If this fails, the identifier of the most recent session in the database is used. We then define the absolute and relative paths of the folder in which the H264 videos to be assembled are saved.

We create a text file for FFmpeg containing the list of the absolute paths of each H264 clip, sorted in chronological order. We also retrieve the timestamp of the start of the video recording.

Once this is done, we can run the command using FFmpeg to merge the videos listed in the text file. To save processor time, the videos are not re-encoded; we simply concatenate them and then wrap the result in an MP4. The creation time is passed as metadata. The final video is called `final.mp4` and is saved next to the clips it comes from.

Finally, we delete the text file used by FFmpeg, then close the connection to the database if it has been opened.

Once the file `vid.py` had been created, most of the problems concerning the video had already been cleared up. Writing this file was not particularly complex, since it follows from solving the problems that had been encountered. The only surprise was finding that it was not possible to merge the videos directly via Python, but that a `bash` command had to be used.

6.7 Automatic installation

Throughout this document, we have seen how to manually configure a *Raspberry Pi Zero 2 WH* for the needs of this project, and then how to transfer the Python code files for data collection to it.

In order to simplify the building of a second prototype and, more generally, to make this configuration reproducible, an automatic configuration system was created.

The automatic installation works as follows: we curl the file `config.sh` onto the Raspberry, make it executable, then run it with

```
bash
cd
curl -LO https://raw.githubusercontent.com/anonyme-sjf-26/
                                            Abraxas/refs/heads/main/config.sh
sudo chmod +x config.sh
sudo ./config.sh
```

A progress bar appears, then, after a few minutes, a message announcing the end of the installation should be displayed.

6.8 Configuration files

progress_bar.sh

The code handling the display of the progress bar is adapted from open source code[35]. After checking it carefully and then adapting it to the needs of the project, I integrated it. It uses concepts that are unknown to me, so its workings are not described exactly: I use it as a plug-in. The progress bar looks like this:

```
plain text UTF-8
Configuring the Raspberry 6/30 ▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 20%
```

The principle is that if another script, while running a function, starts `progress_bar.sh` as a child process and redirects its outputs (`echo`) to it, `progress_bar.sh` displays a progress bar for this function. At launch, the child process receives as arguments the title of the bar and the number of steps required for its completion. To make the bar fill up, the child

[35] AVONTURE Christophe, “Linux - Using a progression bar in your script”, *avonture.be*, 07.09.2024, https://www.avonture.be/blog/bash-progression-bar/ (accessed on 21.10.2026).

process relies on the outputs of the parent script. It intercepts the expressions containing `Progress=` and updates the bar according to the number that follows.

config.sh

We start by defining four logging functions according to the level of the alerts: success, information, warning and critical error.

We then define an identity check function: the script should only be run with `root` permissions. The check is carried out just before the main function.

We initialise a counter to zero for the progress bar, as well as a function to increment it. It will be called after each step successfully completed in the main function.

Next comes the definition of the main function itself. It contains all the steps needed to configure the Raspberry, except SSH, as described in this document. The only steps that required adaptation are those relying on an interactive interface and not just on command lines. Only these adaptations are detailed below.

- Configuring `serial`

Instead of using the interactive interface of `raspi-config`, the desired commands are executed directly with the non-interactive built-in of this program.

- Configuring the virtual environment for the IMU

When it runs, the IMU configuration script asks three questions, which must be answered yes, no and no in succession. To do this, another file is needed, `auto_install.exp`. How it works will be detailed later.

- Initialising the database

Since the code needed to create the database is not `bash` but `sql`, it must be run from another file, `config.sql`.

- Configuring `cron`

The configuration of this service is interactive par excellence. As such, making it possible solely via a file executed linearly was a major challenge.

We start by retrieving the existing `cron` rules and copying them into a temporary file (`/tmp/cron_config`). We then add at the end of this file the `cron` rule we are interested in, namely the execution of `interface.py` (made executable beforehand) at system startup. We then replace the current `cron` rules with the temporary file. Finally, we delete the temporary file.

The main function ends by cleaning up the files needed for the installation that have become useless.

auto_install.exp

During the initialisation of the IMU, a bash configuration file provided by Pimoroni is used. This file asks for confirmation for three actions. It is easy to provide a single answer to a bash script when it is launched thanks to the following syntax:

```
bash
./script.sh <<< 'answer'
```

It is in fact used in the non-interactive configuration of the GPS. But here, the case is different, because the answers to be given are not always the same. We therefore need an expect file: `auto_install.exp`. This format is designed to launch the execution of a bash file, then to wait for certain outputs and answer them according to what has been configured beforehand. This is exactly what happens with `auto_install.exp`. It launches Pimoroni's `install.sh` file with `spawn`, then waits for the three questions and answers them according to what is described in the file, i.e. yes, no and no.

config.sql

This sql file contains exactly the commands described in Section 5.13. They have simply been gathered into a single file run from `config.sh`.

6.9 Moving to GitHub

In order to facilitate both the traceability of the different versions of the code written and its transfer to the Raspberry, I decided to move it to a GitHub repository. It is publicly accessible. This move was made roughly halfway through writing the code. Later, other GitHub repositories were added. In total, there are four:

- For the device code: https://github.com/anonyme-sjf-26/Abraxas
- For the server code: https://github.com/anonyme-sjf-26/Abraxas_server
- For the 3D files: https://github.com/anonyme-sjf-26/Abraxas_3D-files
- For the PCB-related files: https://github.com/anonyme-sjf-26/Abraxas_PCB

Indeed, this service makes it possible to keep editing the files via PyCharm as I did before, but instead of saving a copy at each major change, it allows me to keep track of the changes in the form of a tree in a single file, as visible in Figure 17. This makes it easier to go back to a previous version and allows better tracking of improvements, making it easy to understand again how a complex piece of code works, even after a long break.



Figure 17: View of the tree of changes
in the file `config.sh` in PyCharm

Moreover, making the code publicly accessible makes it easy to ask for help or to share the progress of the project. But above all, it makes it possible to download the code from GitHub's servers onto any device with a simple command line:

```bash
git clone https://github.com/anonyme-sjf-26/abraxas.git
```

To organise it, the usual shell commands such as `mv`, `rm`, `cp`, etc. are then used. In my case, this makes it possible to transfer the latest version of my files to the Raspberry simply and efficiently.

The GitHub repository also contains the instructions for initialising the prototype, this documentation and the MIT license, which places the entire project in the public domain (open source) while limiting my liability in the event of reuse by others.

Each change to the code (commit) must be accompanied by a very short description of the changes. I chose to generate them automatically (without a prompt) by AI, because although their existence is essential, writing them is trivial. Similarly, the release notes and the READMEs of all the repositories except the main one were generated automatically by AI. All the information they contain is already written in this report. Writing yet another summary of it myself was of little interest.

7 SECOND PROTOTYPE

After obtaining a first working prototype including a GPS, an IMU and a camera, I still had to add the anemometer and place everything in a waterproof box, so that the prototype could be taken on board for tests.

7.1 Autonomous power supply

Until now, the prototype was powered from the mains. In order to be used on a boat, it had to be autonomous. To do this, I use a battery and a PowerBoost.

Battery and autonomy

I chose an almost flat lithium-polymer (LiPo) model providing a voltage of 3.7V, with a capacity of 8000mAh. It offers about 4h of recording with video, and more than 8h of simple data recording. The autonomy without real-time data display has not been tested, but is estimated at more than 12h.

PowerBoost

In order to raise the voltage from 3.7V to 5V, I use Adafruit's PowerBoost 1000C. It also manages the charging of the battery via USB and has an indicator for full charge (green), for charging in progress (orange), and for low voltage (red). The latter is connected to the Low Battery Output (LBO) port, which goes HIGH at 5V when the voltage drops below 3.2V[36].

ON/OFF SHIM

Switching the device off and on is handled by Pimoroni's ON/OFF SHIM. It is a latching switch made up of an NPN and a PNP transistor.

By default, the PNP is closed: the 5V does not pass. A short press on the button opens the PNP, which lets the 5V through to the circuit, switches on the device and activates the NPN (it becomes conductive). After the button is released, the conducting NPN holds the PNP open by simulating the button press. When, from the software, the base of the NPN is set to LOW, it stops conducting and therefore stops holding the PNP open. The PNP closes again and the 5V is cut off[37].

[36] "Adafruit Powerboost 1000C", *Adafruit*, 08.03.2024, https://learn.adafruit.com/adafruit-powerboost-1000c-load-share-usb-charge-boost (accessed on 21.10.2026).

[37] "OnOff SHIM", *Pimoroni*, n.d., https://shop.pimoroni.com/en-ch/products/onoff-shim (accessed on 21.10.2026).

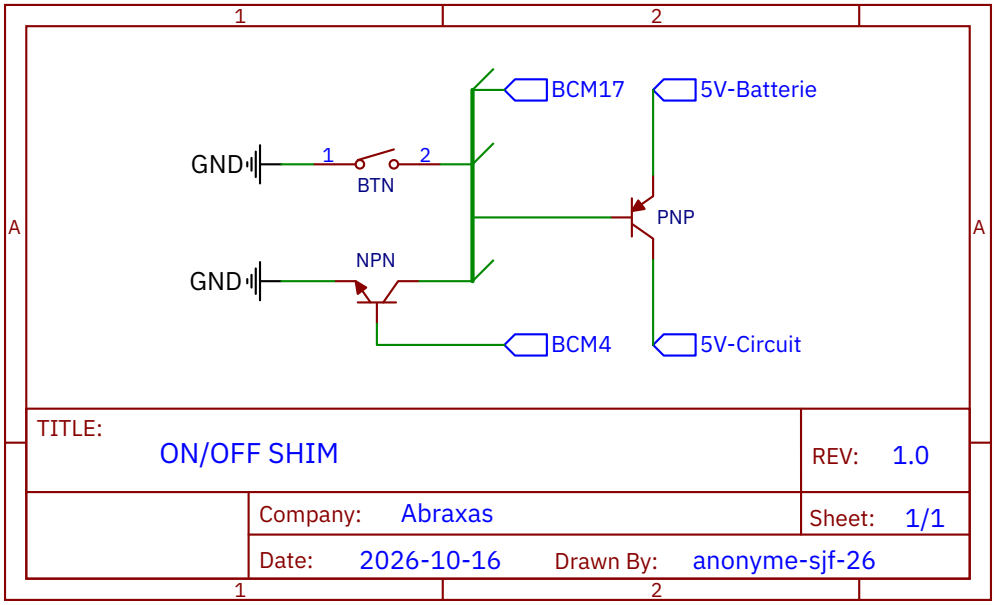


Figure 18: Electrical schematic of the ON/OFF SHIM.
Personal schematic created with EasyEDA.

To understand it better, let us take an analogy: the PNP is a door that is closed by default. Pressing the button opens it, and the NPN wedges the door so that it does not close again, even when the button has let go of it. Thus, the 5V can pass. If the Raspberry cuts the power to the base of the NPN, the latter falls and the door closes again. The 5V can no longer pass.

7.2 Motherboard

In order to make my device more compact, I developed, on the advice of my supervising teacher, a motherboard, or printed circuit board (PCB). The advantage of this solution also lies in the neatness and robustness of the final result. Indeed, it does away with a large number of wires and makes it possible to add various safety systems such as capacitors or fuses.

To develop this PCB, I used the EasyEDA software. It is directly connected to JLCPCB, the company that manufactures the PCBs. Thus, while drawing the motherboard, it is possible to directly integrate the components that will later be soldered onto it. Their footprints and electronic characteristics are available, which greatly simplifies the design.

Electrical schematic

The first step is drawing the electrical schematic. It represents how the components will be connected to each other. An addressing system is used for this so as not to have to draw each wire. That would make the drawing far too cluttered.

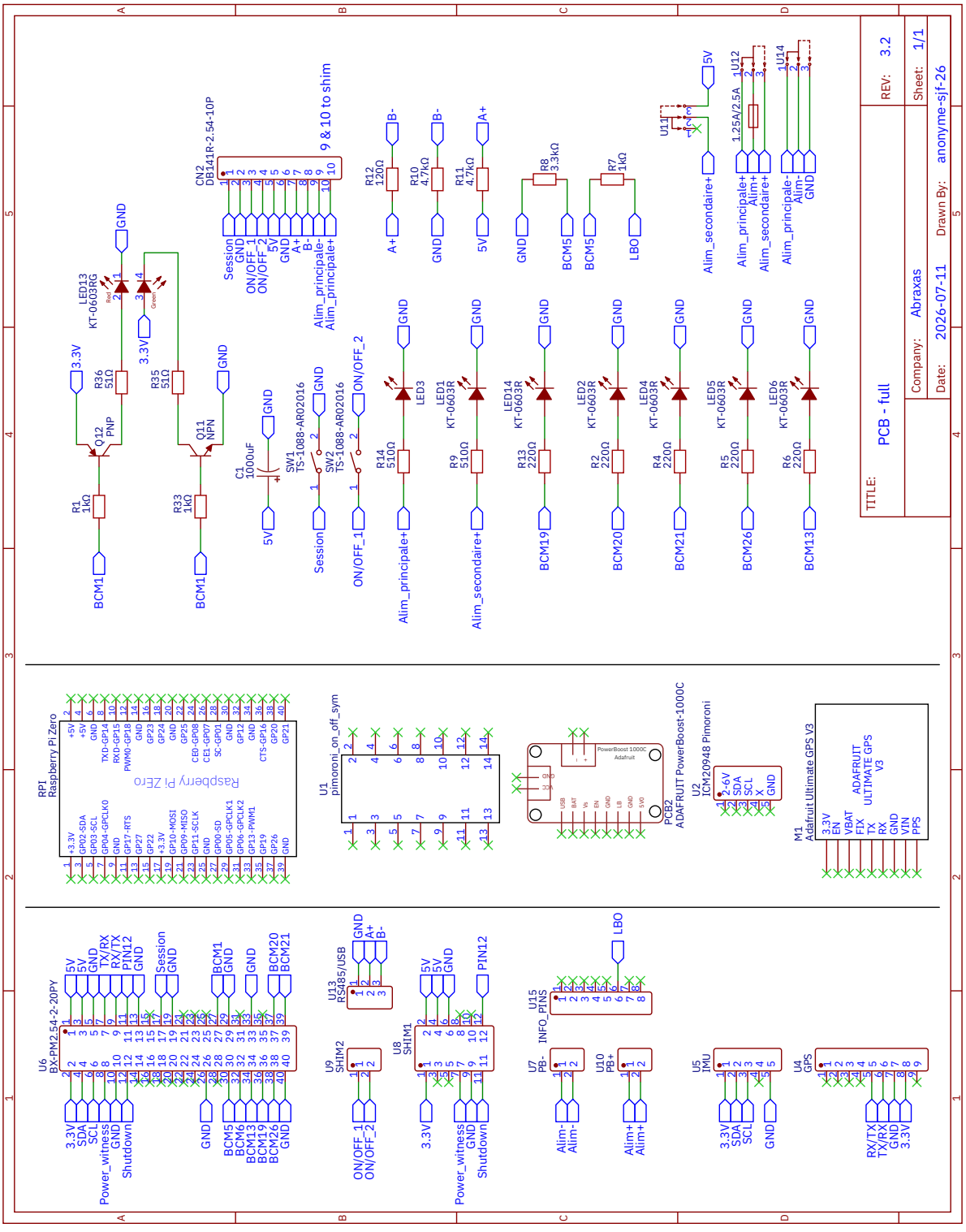


Figure 19: Electrical schematic of the PCB.
Personal schematic created with EasyEDA.

- Power supply

The power supply is divided into two separate circuits: the main power supply and the secondary power supply. The secondary one is intended as a backup plan in case the main one malfunctions.

For the main power supply, the path of the 5V is as follows:

```
diagram
Battery →(via cable)→ PowerBoost →(via PCB)→ terminal →(via cable)→ SHIM
→(via PCB)→ fuse →(via PCB)→ circuit
```

For the secondary power supply:

```
diagram
Battery →(via cable)→ PowerBoost →(via PCB)→ switch U11 →(via PCB)→ fuse
→(via PCB)→ circuit
```

The problem with the secondary power supply is that the device is switched on and off by hand. This is therefore no longer feasible once the box is sealed. With the main power supply, the 5V is cut by the SHIM, and therefore in a way that can be controlled from outside the box via a switch. The choice between the two power supplies is made by setting the position of switches U12 and U14.

As an additional safety measure, two pads have been provided underneath the PowerBoost, directly connected to the 5V-regulated + and -, at the first stage after the battery output, in case of total failure. It is in fact these pads that I use in my final prototype, because switch U12 is defective. I bridge it from the pads, so as to still use the SHIM via the main power supply.

- Raspberry

The Raspberry is plugged directly into header U6. Thus, all of its GPIO ports are accessible via the PCB.

- IMU and GPS

They are connected as indicated previously, directly to the Raspberry's GPIO. The IMU plugs into header U5, the GPS into U4.

- Anemometer

The breakout connects to header U13. The A+ and B- of the RS485 are indeed there. The USB link between the breakout and the Raspberry does not go through the PCB. Consequently, it is not shown. The anemometer itself plugs into terminal block CN2. The link to the breakout is made through the PCB.

To protect the RS485 against signal reflections, three resistors are needed: one on A+, one on B-, and one between the two. These are R11, R10 and R12 respectively.

- Camera

The CSI ribbon cable connecting the camera is independent of the PCB.

- Indicator lights

There are seven indicator lights. One for the main power supply, one for the secondary power supply, and five that can be freely assigned via GPIO BCM13, 19, 20, 21 and 26. Each of these LEDs is protected by a resistor.

- Inverting indicator light

There is a so-called inverting indicator light, i.e. two LEDs side by side, a red one and a green one, which can light up alternately. This inversion is controlled by a single GPIO, BCM1, and by two transistors, an NPN and a PNP (Q11 and Q12 respectively). When BCM1 is LOW, the PNP conducts and the NPN blocks: the red LED lights up. When BCM1 goes HIGH, the PNP blocks and the NPN conducts: the green LED lights up.

- Capacitor

A 1000µF capacitor (C1) has been placed as close as possible to the Raspberry's pins. Thus, the current peaks of more than 2A (the capacity at the battery output) that can occur when the WiFi is switched on will be absorbed without any problem.

- Voltage divider

Resistors R7 and R8 form a voltage divider. The LBO indicator of the Adafruit PowerBoost goes HIGH when the battery voltage drops below 3.2V, indicating a low battery. However, the voltage of this signal is 5V. It must be lowered to the Raspberry's 3.3V logic. To calculate their value, the voltage divider formula is used:

$$U_2 = U_1 * \frac{R_2}{R_1 + R_2}$$

- ON/OFF SHIM

The Pimoroni SHIM is connected to the PCB by two headers: U8 and U9. Its two terminals that short-circuit its button are connected to a switch. The base of the NPN, which must be set to LOW to trigger shutdown, is on BCM4. In addition, the SHIM's button is also connected to BCM17 so that the Raspberry can set BCM4 to LOW when the button is pressed (shutdown signal).

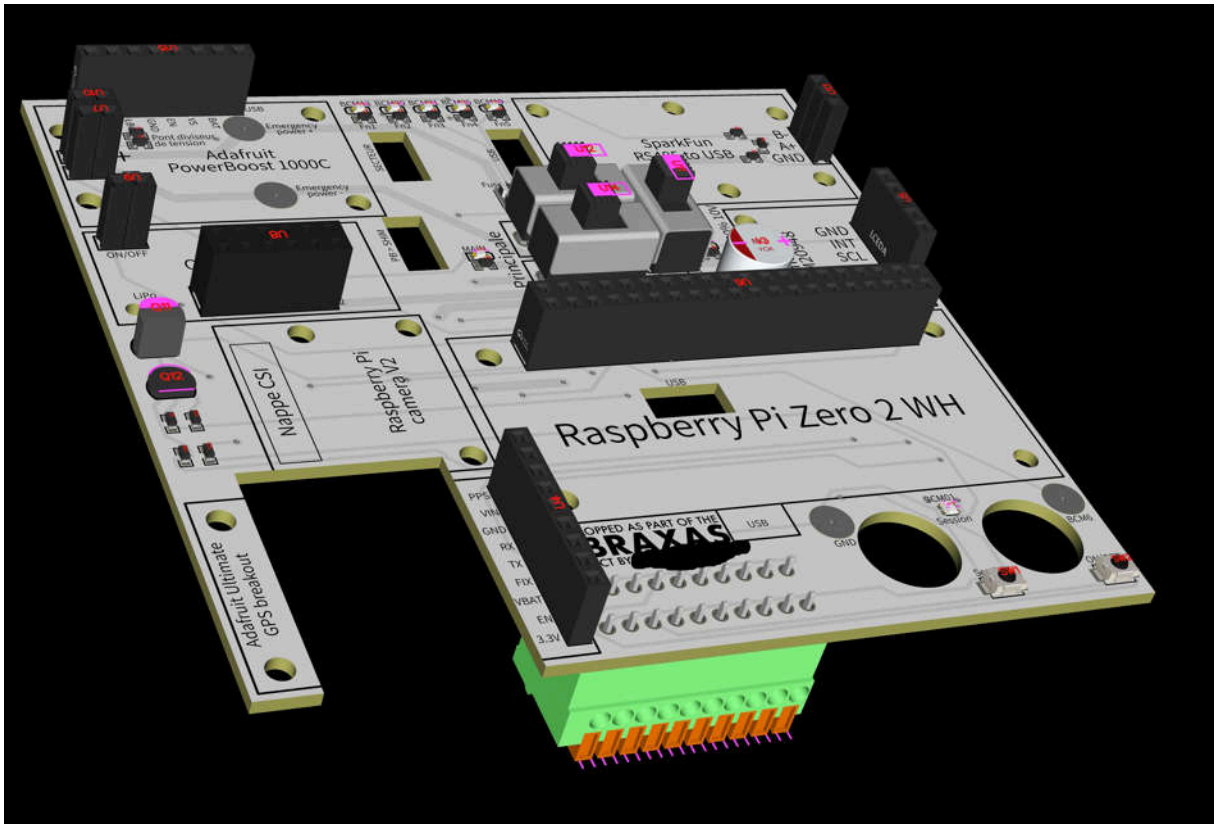


Figure 20: 3D model of the PCB with the components.
Personal representation created with EasyEDA.

- PowerBoost

The Adafruit PowerBoost is connected on U7, U10 and U15. The LBO indicator is located on U15.

- Switches

There are two switches: one to start and stop the session (short-circuiting GPIO17 and GND), and one to switch the device on and off (short-circuiting the two BTN terminals of the SHIM). They are both connected to terminal block CN2. For each of the two switches, two stand-ins (SW1 and SW2) are placed directly on the PCB so that the function of the switches can be tested before they are connected to the terminal block. The stand-ins bridge the same contacts as the main switches.

7.3 PCB footprint

After drawing the electrical schematic, these connections still have to be laid out concretely on the board that will become the PCB. It is possible to have connections (tracks) on each side of the board. To make them change sides, vias are used, whose number one tries to minimise for the sake of optimisation. The width of the tracks and vias depends on the voltage and current for which they are designed. I use tracks between 0.5 and 1.5mm wide, as well as vias with an inner diameter of 0.6mm and an outer diameter of 1.2mm. This provides a comfortable margin to prevent the whole from heating up[38].

The various breakouts are held on the PCB by means of standoffs. Holes are provided in the PCB to be able to attach them. When designing a motherboard, an excellent sense of spatial vision is needed to arrange all the components as well as possible in a minimum of space. Since compactness was an important constraint for me, the optimisation work was particularly crucial. Finding room for each element while limiting empty space, all the while leaving room for the various cables, was a real puzzle. This unexpected form of Tetris was particularly time-consuming.

Moreover, I had no experience in motherboard design, so until the moment of testing I was worried about whether I had made a critical mistake. That is why I multiplied the safety measures and the means of recovery. Fortunately, the result is satisfactory.

One representation is reproduced: the two sides superimposed, in black and white.

[38] “Calculateur de largeur de piste de circuit imprimé”, *DigiKey*, n.d., https://www.digikey.be/fr/resources/conversion-calculators/conversion-calculator-pcb-trace-width (accessed on 21.10.2026).

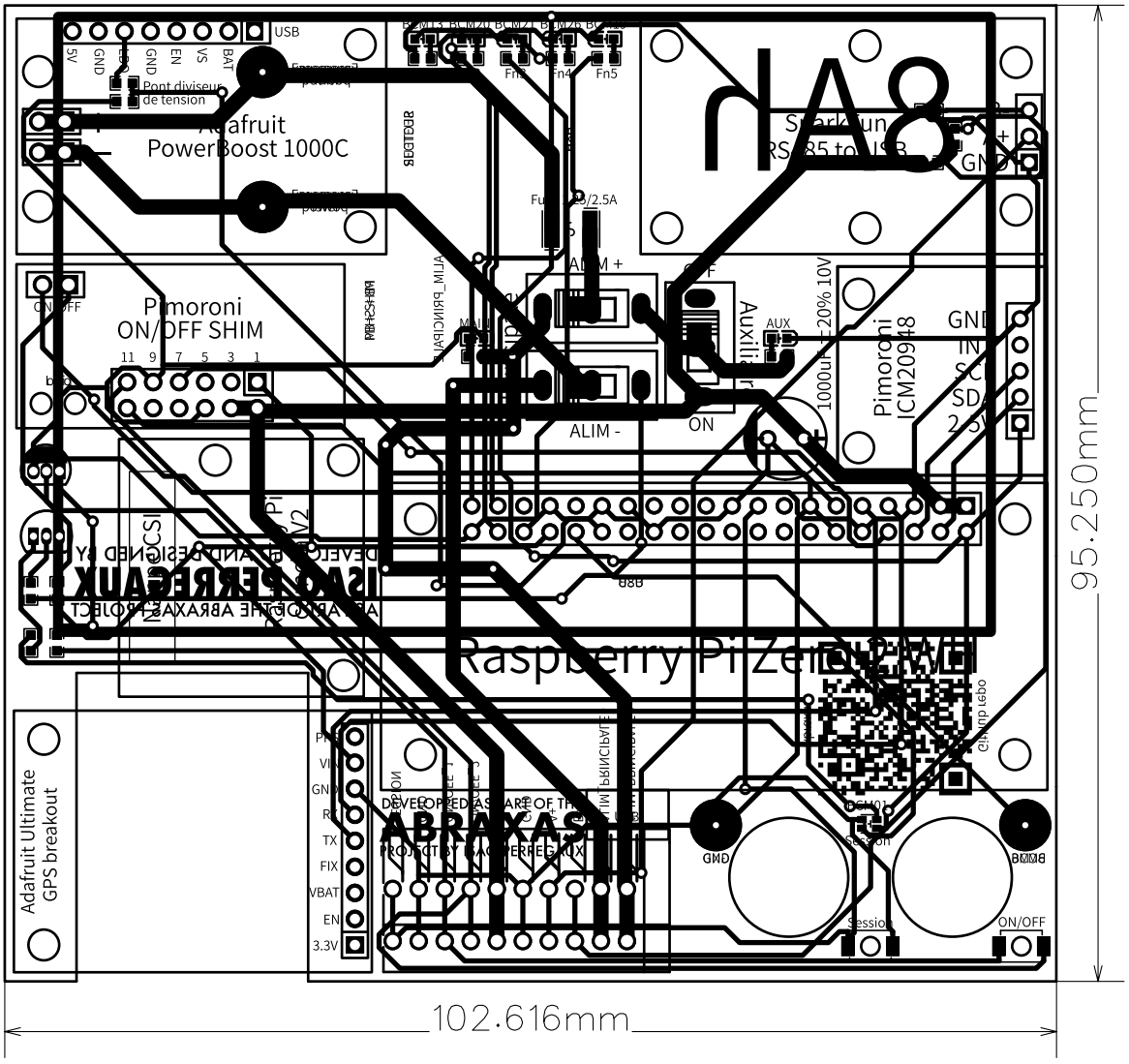


Figure 21: PCB footprint, both sides, black and white.
Personal representation created with EasyEDA.

8 3D WORK

In order to take the device out on the lake and use it on board a boat, a waterproof box was needed to house the electronic components. It had to meet several constraints:

- Be waterproof
- Have a transparent lid for the camera and the indicator lights
- Have two cable glands for the anemometer and the GPS antenna
- Have two buttons
- Have a USB-C port to recharge the device
- Allow the components to be firmly attached inside

To meet all these constraints, the simplest solution seemed to me to be to model this box in 3D and then print it.

8.1 Modelling

I had never worked with 3D software before, so I first had to learn the basics. The learning curve is not excessively steep, but I needed time to adapt. Even simply rotating the part in space was, for example, not obvious at first.

From a relatively simple box, I arrived at a complex model, with different systems of standoffs stacking on top of each other and of layers combining. The model file can be found in the appendix, in STL (`.stl`) and STEP (`.step`). The OnShape CAD (computer-aided design) model is also available in the appendix.

Box

Starting from the bottom of the case, there are the following layers:

1. 3mm: bottom of the box.
2. 14mm: battery surrounded by wedges, fixed standoffs on the side to support the PCB. Next to the battery is the terminal block from which the cables come out. It hangs below the PCB (cf. Figure 20).
3. 4mm: space reserved for the wiring coming out of the terminal block and going to the anemometer cable gland, the two buttons, the SHIM and the PowerBoost (cf. Section 7.2).
4. 11mm: PCB. It is supported by 18mm fixed standoffs, printed on the bottom of the box. Within this layer are the removable standoffs, either screwed onto a fixed standoff, sandwiching the PCB, or attached to the PCB from below.
5. 15mm: free space for the various components. Almost completely free of cables.

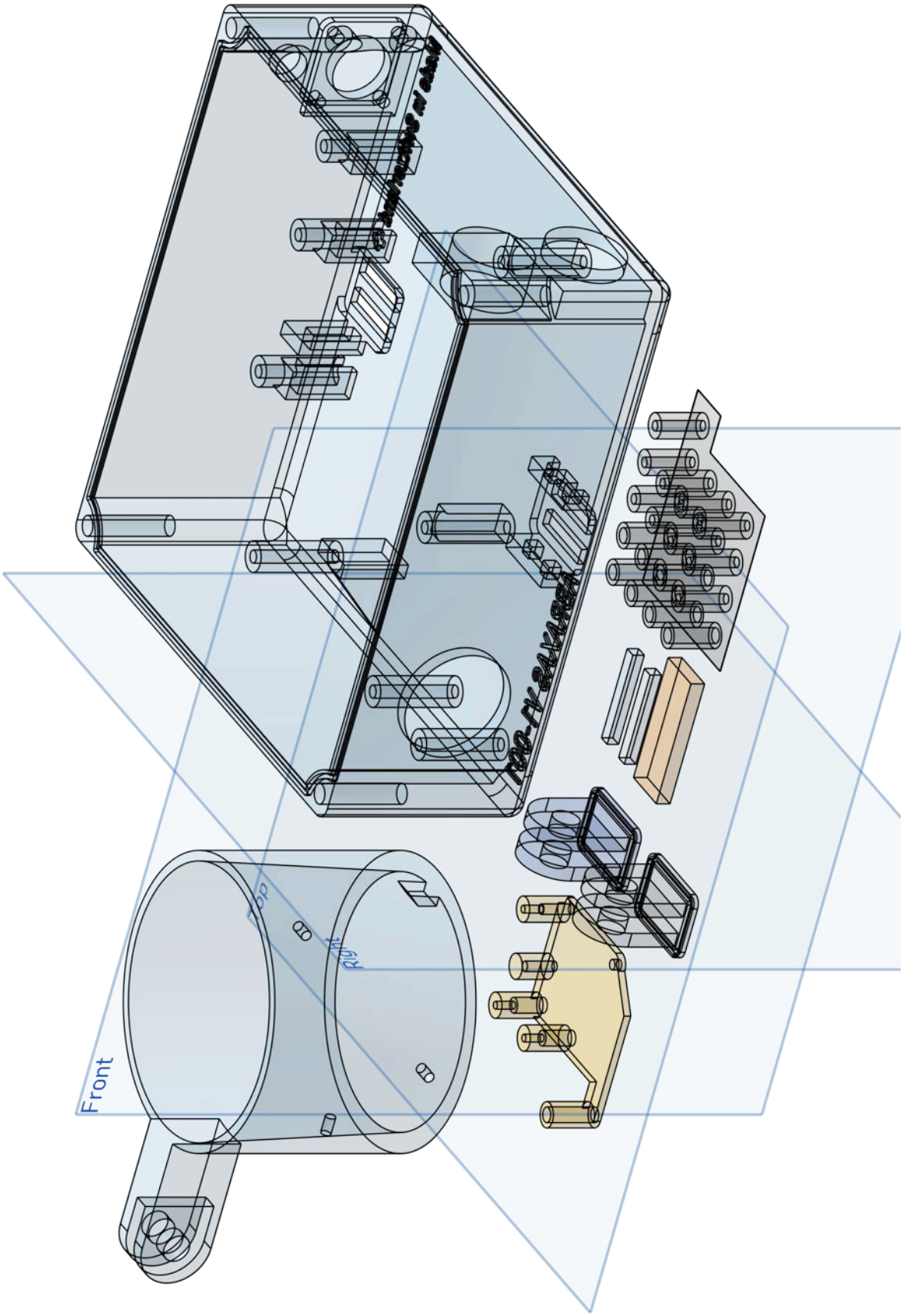


Figure 22: Overview of the CAD model in OnShape, translucent faces.
Personal representation created with OnShape.

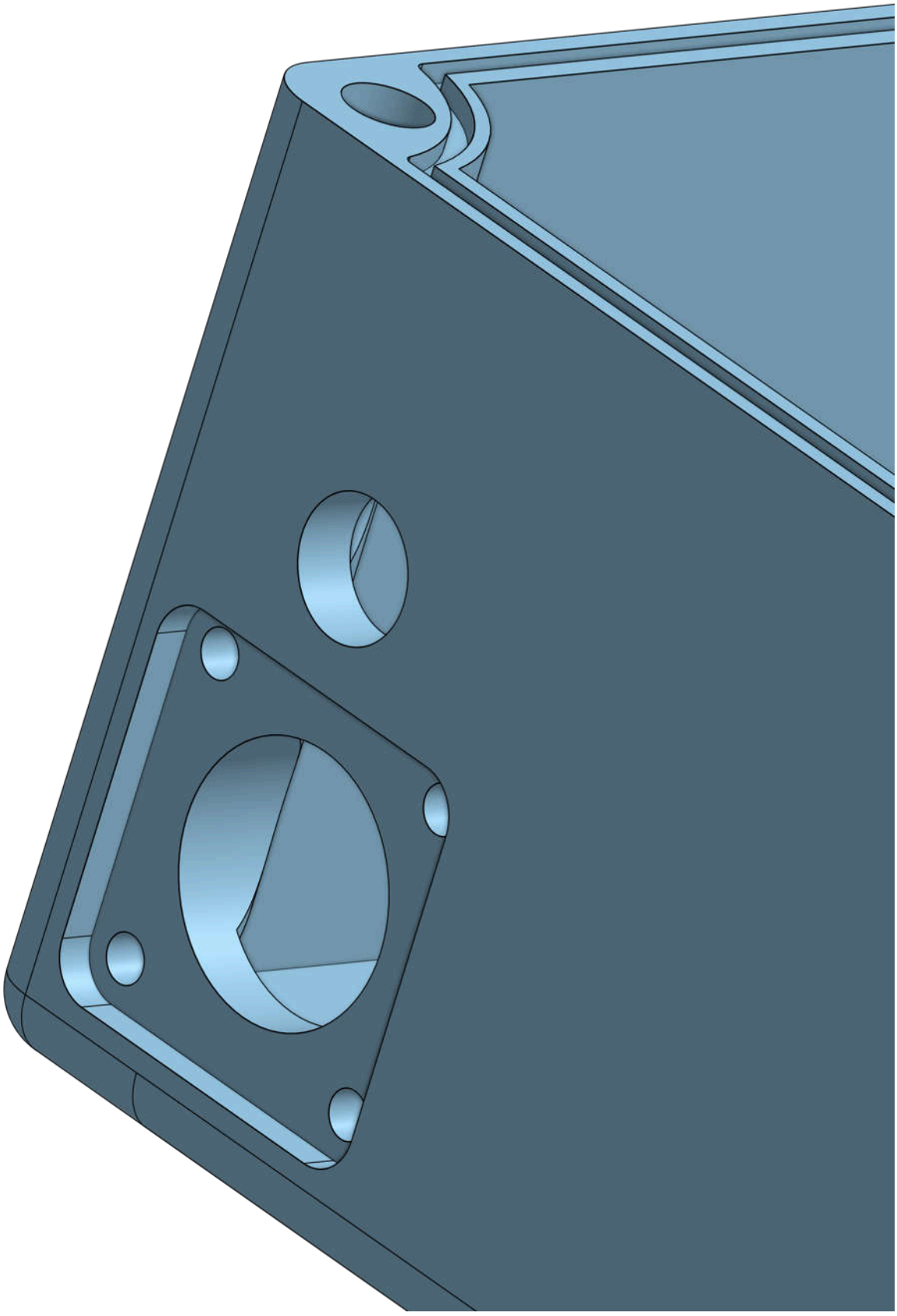


Figure 23: Recess for the anemometer and GPS cable glands, rear face.
Personal representation created with OnShape.

At the bottom of the box are two recesses designed for sliding in the GoPro-compatible mounts.

The walls are 3mm thick and 44mm high. The bottom measures 116.816mm by 109.45mm. Each corner has a reinforcement. On the top, there is a groove to place a silicone seal and a hole in each corner to screw on the lid. On the front face, there is a hole for the USB-C port. On the rear face, there is the port for the anemometer and the one for the GPS antenna. On the right face are the two buttons. One for shutting down, the other for starting and stopping the recording. On the underside, the inscription “Made in Switzerland +” can be seen, as well as the name of the model and its serial number.

Lid

The lid, made of transparent plexiglass, was laser cut. It is 3mm thick.

Removable standoffs

There are 18 removable standoffs of three different types. They are located between the PCB and the additional components and serve to hold the latter. 17 are “simple”, i.e. they take the form of a tube through which one or two screws pass. They can either be in line with a fixed standoff or offset. The model of the standoff differs depending on the case, because the screws used are not the same.

The last standoff, the one holding the camera, is more complex. Indeed, the camera had to be attached above a gap left in the PCB to let the USB-C socket through. The standoff therefore serves as a “bridge” over this gap.

GoPro-compatible mounts

In order to attach the device to the boat, it was decided to use GoPro-type mounts. Indeed, most clubs already have GoPro mounts on their boats. Thus, without any additional accessories, the case and the anemometer can be attached simply and benefit from the wide variety of existing mounts.

It was not possible to print the mounts as one piece with the box, because it would then no longer have had a base on which to rest during printing. They were therefore designed as separate inserts to be slid in from inside the box. Once in place, they are held by locking blocks slid over them.

Anemometer mount

The base of the anemometer is a slightly flared cylinder. The mount matches this shape and is printed as one piece with the GoPro-compatible mount. It includes three holes for screwing on the anemometer, as well as a locating pin matching a recess in the anemometer to guarantee its alignment with the mount.

8.2 Printing

For the modelling, I worked with the OnShape software, from which a `.stl` file can be exported. In order to print the model, it still has to be “sliced”. Indeed, an STL file is monolithic: it contains the solid shapes, whereas the printer needs a path for the nozzle, the details of the infill pattern of the shapes, the travel speeds, the type of filament to use, etc. To go from one to the other, CrealityPrint, the software provided with the printer, is used. The result is a `.gcode` file, usable by a specific printer. A `.3mf` file, the generic sliced model, can also be exported.

The total printing time of the latest version amounts to 3h15. The print uses 36.75m of filament and weighs 109.61g. The G-code and the 3MF can be found in the appendix.

To arrive at this latest version, the box had to be reprinted three times. The first two contained errors either of design or of tolerance for fitting the parts together. Indeed, at the time of the first print, I did not know that clearances had to be left between the parts. Moreover, judging them is sometimes difficult without experience.

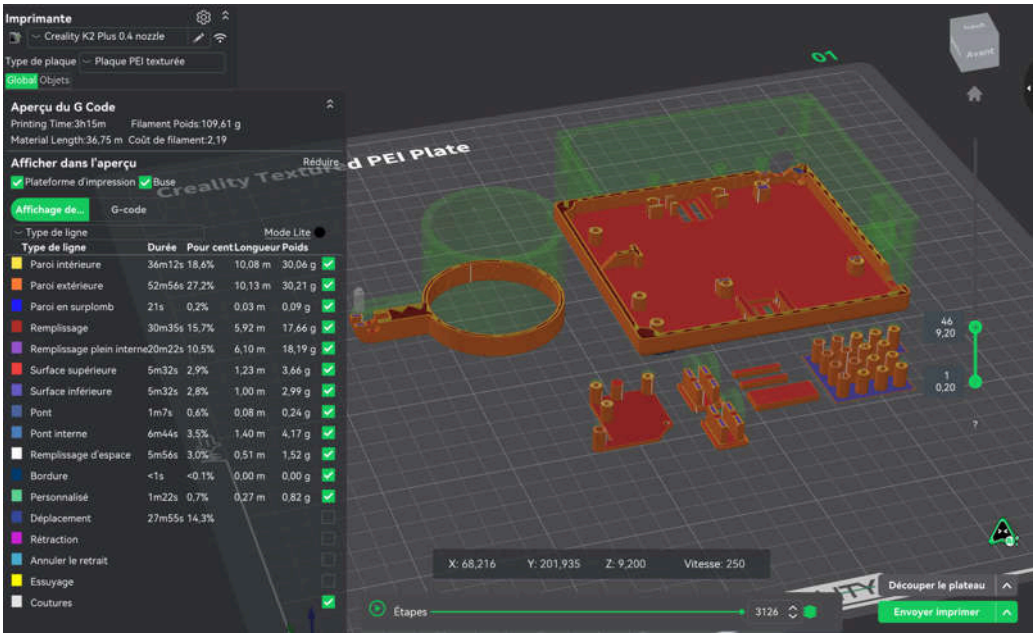


Figure 24: Detail of step 46 while printing the model, taken from the gcode.

8.3 Assembly

Additional parts for the assembly

For the assembly, in addition to the components of the prototype, parts specific to the box are needed. The references can be found in Section 5.1.

- Two IP67 buttons
- Two connectors with at least four contacts, IP67
- One IP67 connector for an active GPS antenna
- One IP67 active GPS antenna
- One IP67 USB-C port
- One flat USB cable, angled mini-B to flat micro-B
- Two bare micro-B USB plugs with two contacts

Soldering

We cut the cable coming out of the panel-mount USB socket as far as possible from the panel-mount block, extract the power wires from it, then strip them. Next, we solder:

- The wires we have just stripped to a micro-B USB plug.
- Each breakout to its pins. In order to be as precise as possible, the breakout-pin assembly is heated before bringing in the solder.
- The wires to the switches and to the micro-B USB plugs so that they can be connected to the terminal block.
- We solder the anemometer wires onto the female connector: 5V on contact 1, GND on 2, A+ on 3 and B- on 4. Contact 5 remains free. We also solder wires onto contacts 1 to 4 of the male connector.

All solder joints (except those on pins directly on a PCB) must be protected with heat-shrink tubing to prevent short circuits.

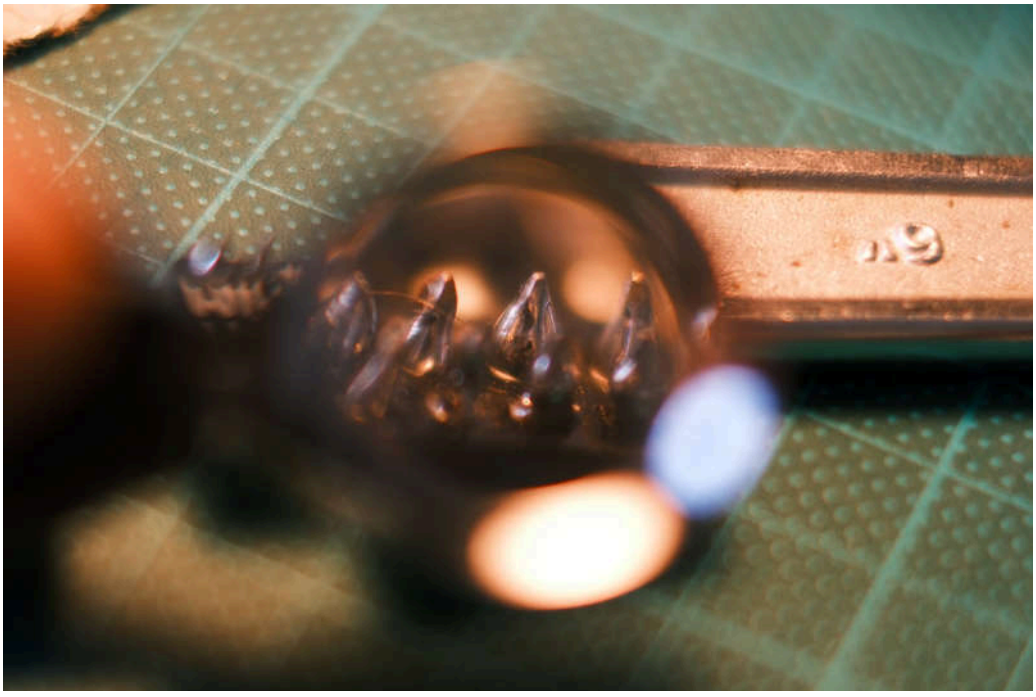


Figure 25: Hand soldering at 2.54mm pitch, seen under a magnifying glass

Wiring plan

- Legs of the upper switch to GND and to Session on the terminal block.
- Legs of the lower switch to ON/OFF_1 and ON/OFF_2 on the terminal block.
- Legs 1 to 4 of the connector to 5V, GND, A+ and B- respectively on the terminal block.
- Wires soldered to the PowerBoost's micro-B plug to the panel-mount USB-C block.
- Wires soldered to the SHIM's micro-B plug to ALIM_PRINCIPALE+ and ALIM_PRINCIPALE- on the terminal block.
- JST connector of the battery to the JST connector of the PowerBoost.
- CSI ribbon cable between the camera and the Raspberry.
- USB cable between the SparkFun USB/RS485 breakout and the USB port (not the one dedicated to power) of the Raspberry.

Assembly steps

1. Start by inserting the GoPro mounts at the bottom of the box, then secure them by sliding the locking blocks into their housings just above.
2. Screw the removable standoffs that are not above fixed standoffs to the PCB from its underside. Also screw on the special standoff for the camera.
3. Then position the breakouts and the Raspberry on the PCB, and screw them to the standoffs that have already been positioned. The CSI ribbon cable can be folded under the Raspberry so that it is not visible.
4. Then insert the lower button, the cable gland for the anemometer and the USB-C port.
5. Connect each cable to the terminal block located under the motherboard. Those connected to the upper button first go through its hole in the wall of the box, then through the circular hole on the bottom left of the PCB. Those connecting the PowerBoost to the USB-C port go through the rectangular hole in front of the PowerBoost. Those connecting the SHIM to the terminal block go through the rectangular hole in front of the SHIM.
6. The battery is placed in its housing at the bottom of the box. Its JST connector is connected to that of the PowerBoost.
7. Position the PCB on the fixed standoffs, taking care to arrange the cables correctly.
8. Position the rest of the removable standoffs, then secure the PCB by screwing it to the box with long screws through the breakouts, the removable standoffs and the fixed standoffs.
9. Insert the upper button and the cable gland for the GPS antenna. The GPS antenna cable is connected to the GPS. It can be routed under the components so that it is not visible.
10. Check that the slide switches ALIM+ and ALIM- are set to "PRINCIPALE", and that the slide switch "AUXILIAIRE" is set to OFF.
11. Apply a silicone seal in the groove on the top of the walls, then place the plexiglass lid before it dries to guarantee waterproofing. Screw on the lid.
12. Inject silicone into the holes of the anemometer cable gland to make it waterproof.
13. Finish by screwing the anemometer onto its mount. Since the plastic is very hard, it may be necessary to drill a hole in it beforehand.

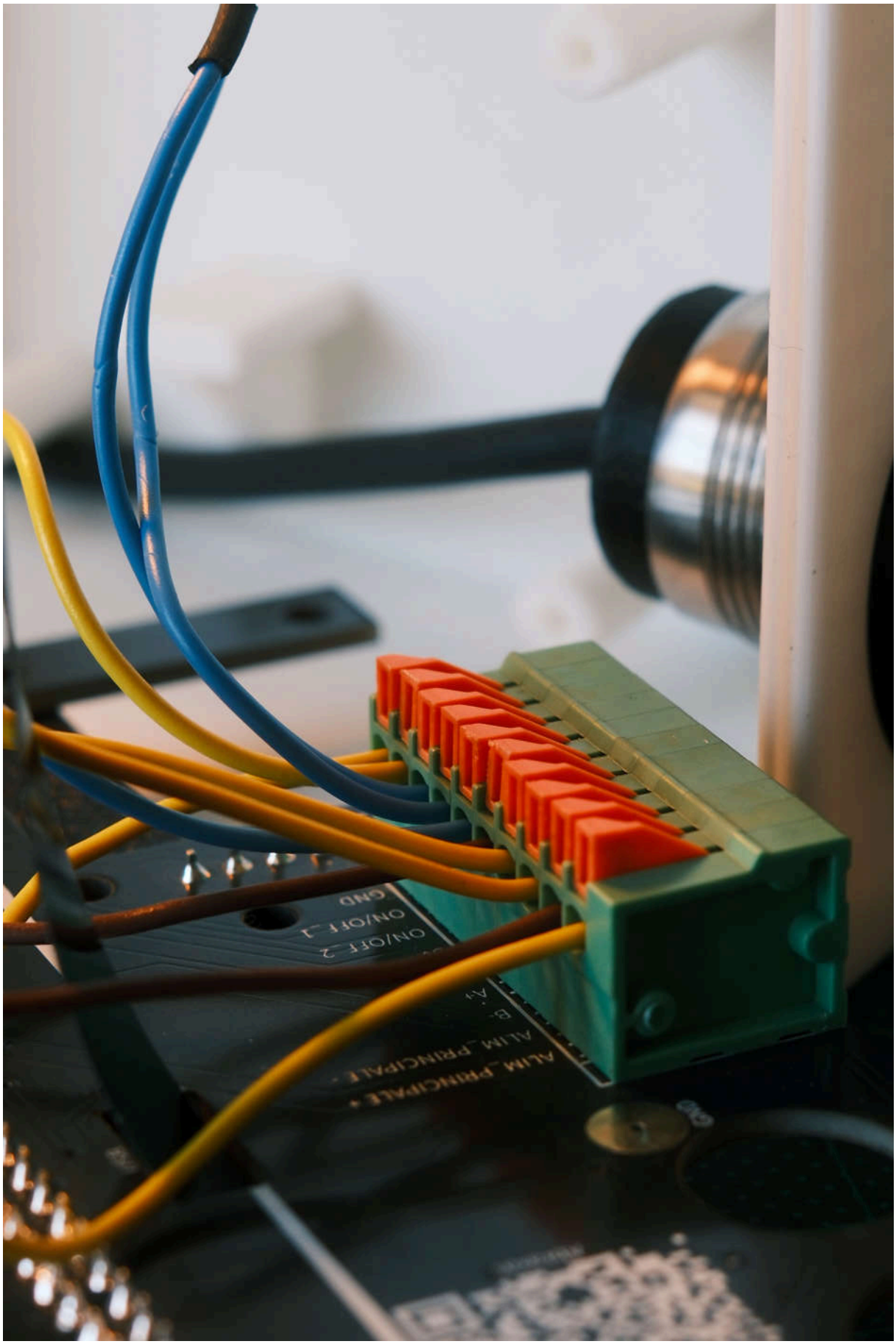


Figure 26: Terminal block at the end of step 5

9 SOFTWARE OF THE SECOND PROTOTYPE

At the end of the first stage, I had learned the basics of several new languages, notably bash, batch, JavaScript, expect and SQL. Drawing on my experience in web development for the website of a students' association, I am also able to use PHP, HTML, CSS, JS and JSON data structures for low-complexity applications. With this background, I was equipped to carry out the second part of the software.

The very verbose description of these first scripts taught me an enormous amount, because it forced me to think in detail about how they work and not simply to write code that works, but also code that is clean. To quote my supervising teacher, it was a matter of "practising my scales". I am now more fluent in writing my code, and no longer need to go through this step. Consequently, the code will be less commented: only the general architecture will be described.

9.1 Usage

Once fully assembled, the device works as follows:

Setting up the device

The case is calibrated to be installed vertically, for example on the mast, below the boom. It could, however, be installed differently. What matters is that it is firmly attached to the boat, so that the IMU takes reliable measurements.

The GPS antenna and the anemometer must be plugged in before switching on the device; otherwise, they are not recognised. For effective operation, the antenna should be placed pointing approximately towards the sky, in an open location. The anemometer should ideally be placed at the top of the mast. However, another open location on the boat where the airflow is not excessively disturbed is suitable and makes it possible to take measurements of acceptable reliability.

Switching the device on and off

The device is switched on and off by a short press on the upper button. It takes just under a minute for the system to be operational. The device should only be switched off when none of the indicators is blinking and when the two left-hand indicators are lit.

Indicators

The indicators are located at the top of the PCB. There are five of them. From left to right:

1. Interface: lights up when the device is ready to receive the inputs given by the buttons. Should light up about twenty seconds after the device has been switched on.
2. Server: lights up when the server is ready and available via the device's network. Should light up one minute after the device has been switched on.

3. Video saving: indicates, by blinking with a period of two seconds, that the video is being saved. Activates after the end of a recording. May remain active for up to about half the duration of the recording after it has been stopped. The fact that it is active does not prevent a second recording from being started.
4. Not assigned
5. Not assigned

There is one last indicator located at the bottom of the PCB: the recording indicator. If it is green, a recording phase is in progress. Otherwise it is red.

Starting and stopping a recording

Recordings are started and stopped by a short press on the lower button. Starting may take a few seconds.

It is advisable to wait until the video saving indicator has stopped blinking before stopping the recording. The saving will probably complete properly anyway, but tests have shown that this use case lacked consistency.

Displaying data in real time

To display the data in real time, connect to the open WiFi with the SSID “Abraxas”. Then, in any browser, type:

```
IP address and port
10.42.0.1:3000
```

A dashboard presenting the measurements taken by the device in real time is loaded. No noticeable latency has been observed.

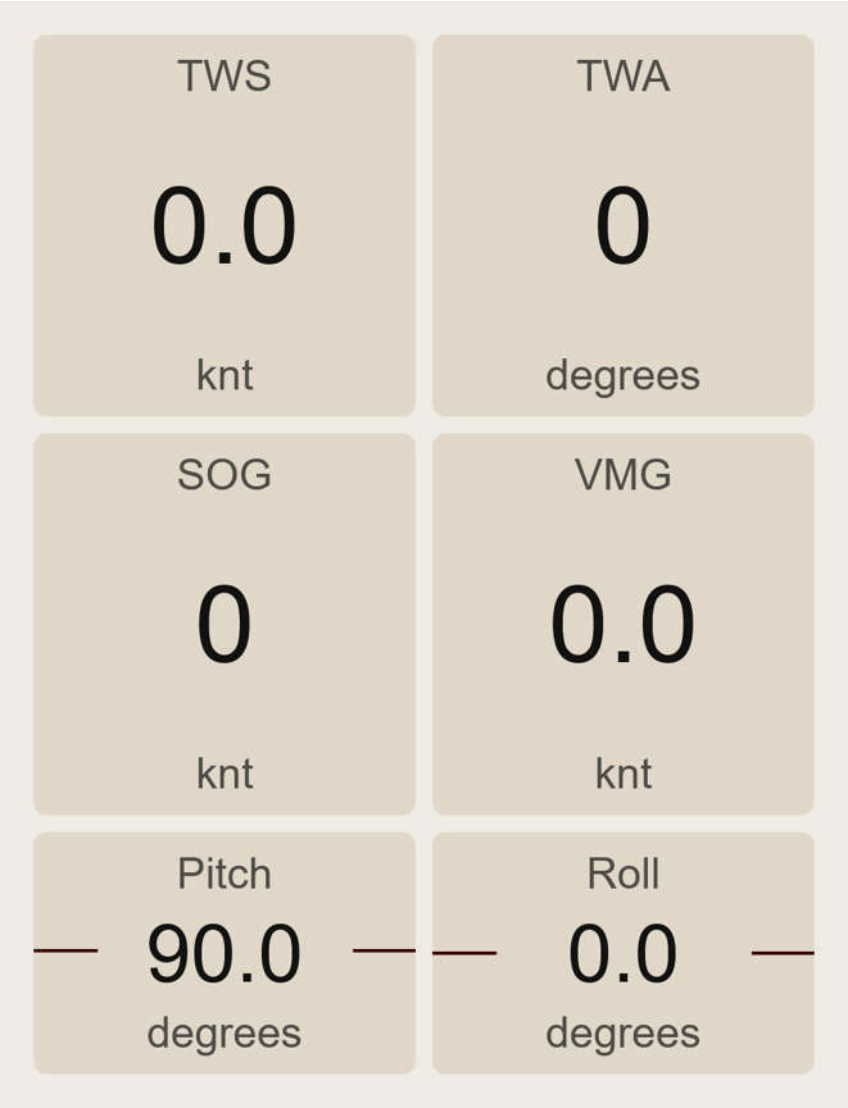


Figure 27: Dashboard presenting the data in real time

Accessing the debrief

Accessing the debrief is done in two steps:

1. Start by connecting to the open WiFi with the SSID “Abraxas”. Then go to a browser and open the following address:

```
IP address and port
10.42.0.1:3001
```

Download the database and the desired videos there.

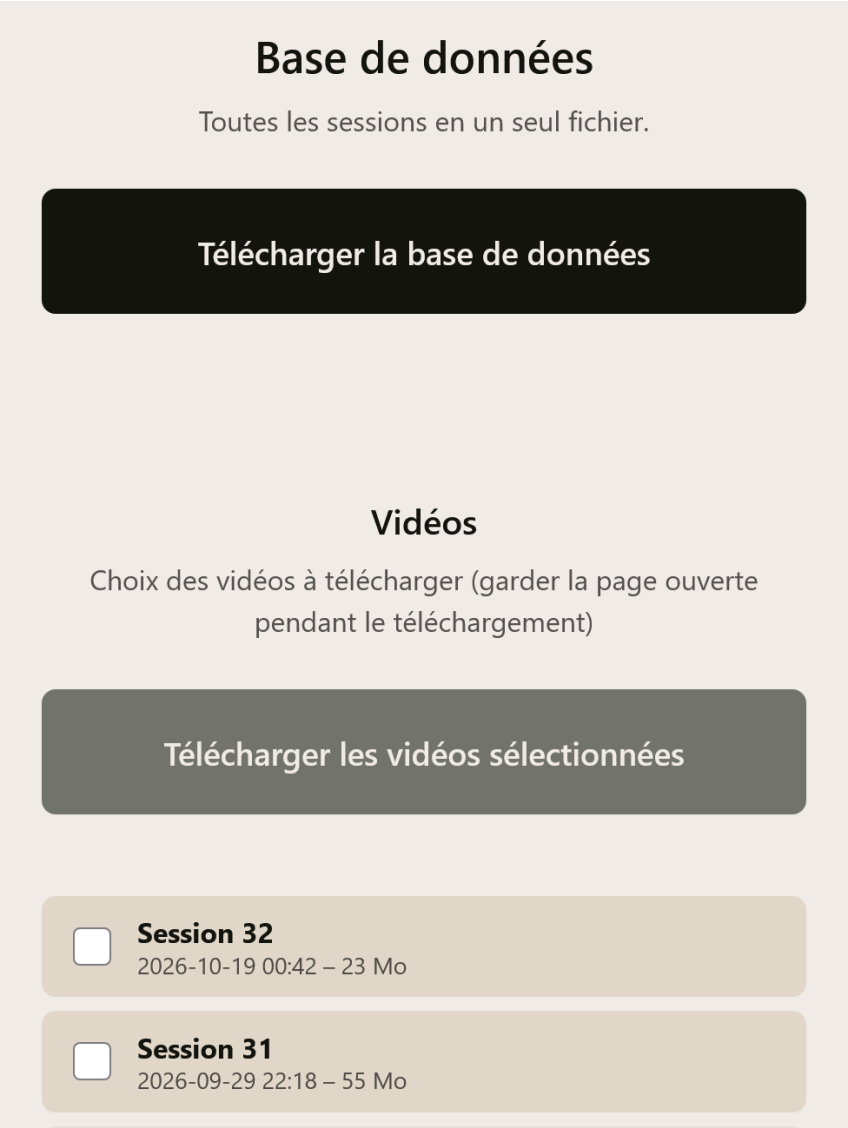


Figure 28: Download interface

2. Disconnect from the “Abraxas” WiFi and, over the mobile network or a regular WiFi, go to https://abraxas.example.org/upload. There, upload the videos and the previously downloaded database. Then, in the “Sessions” tab, the debriefs of the training sessions that have been uploaded can be found. (The cards showing an average can be flipped to display the median.) Actions on this server are protected by a password.

9.2 Automatic installation

The automatic installation proceeds exactly as described in Section 6.7. We run:

```
bash
cd
curl -LO https://raw.githubusercontent.com/anonyme-sjf-26/
                                          Abraxas/refs/heads/main/config.sh
```

```
sudo chmod +x config.sh
sudo ./config.sh
```

The progress bar appears, then, after a few minutes, the system is operational.

Several configuration files are used for this. Most of them have changed little since their first version. A more verbose description of their original version can be found in Section 6.8.

config.sh
It lists all the commands to be run in bash, loads the other configuration files and runs them. It is the conductor. It carries out most of the configuration.

config.sql
It creates the database for the recordings. The architecture of the database is similar to the one described in Section 5.13. The only difference lies in the addition of a table for the anemometer. In a completely analogous way to the others, it contains the fields `id`, `timestamp`, the data fields, namely `awa` and `aws`, as well as the foreign key `session_id`.

progress_bar.sh
No difference compared to the first version.

auto_install.exp
No difference compared to the first version.

9.3 Recording system

The architecture of the recording system has undergone a major change: previously, each script created a cursor for recording in the database and carried out the recordings itself. This turned out to be a major weakness during testing. Indeed, it can create conflicts and abruptly interrupt the execution of the scripts. Consequently, I moved all the recording into a single Python script.

MQTT
For data communication within the Raspberry, I set up a system based on MQTT via Mosquitto. The broker is local (at the loopback address `127.0.0.1`) and permanently enabled. The flow is organised into topics, classified by groups. Each script has a topic group and publishes on several topics: one for all of its data, intended for recording in the database, and one per value that will need to be displayed in real time on the server.

db_recording.py
This Python script subscribes to the topic containing all the data of each file. For each message received, it makes a record.

anemo.py, imu.py, gps.py
`anemo.py` works in the same way as the other two scripts: it collects the data with an acquisition function, parses it, then processes it. The processing of this data is common to

all three. They publish an MQTT message for each data input, following the topic organisation described above.

interface.py

It works in the same way as the first version. The changes made are the addition of two functions:

1. While it is running, it lights up indicator number 1 using the dedicated service.
2. When the shutdown button is pressed, it sets GPIO BCM4 to LOW, causing the SHIM's NPN to drop, which causes the PNP to drop, which in turn cuts the 5V power supply of the device.

master.py

It works in the same way as the first version. The only additions are the start-up of the anemometer script and the management of the blinking of indicator number 3 for video saving.

vid.py

No difference compared to the first version.

grp.py

It works in the same way as the first version. The only difference is the management of the blinking of indicator number 3.

classes.py

It works in the same way as the first version. The functions `emergency_reboot()` and `non_blocking()` have been adapted to the new power management standards (setting GPIO BCM4 to LOW to switch off the system).

gpio_daemon.py

The indicators are managed centrally. As I learned from the problems encountered with the database, it is important to separate the logic level from the interaction with services external to the code, especially if they are physical. The various scripts do not change the state of the indicators themselves. They pass the action to be carried out to the daemon via a socket, a lightweight and efficient way of communicating between different processes. The daemon reads this socket and changes the physical state of the GPIO accordingly.

The advantage of this architecture lies in two major points. First, it significantly increases robustness. There is no risk of conflict in changes to the state of the GPIO, because everything is centralised. Second, whatever the language of the script wanting to change the state of an indicator, the procedure remains the same. It does not carry out a complex physical interaction, but a simple write to a socket. A script in JavaScript, Python, bash, or even, should I ever use them in the future, Rust, PHP or C can interact with the indicators without risk of conflict and without using a different method. This is a huge gain in terms of scalability.

9.4 Hotspot

The hotspot is a new feature of the new prototype. The Raspberry becomes a server delivering two interfaces over the local network broadcast by the device. It works independently of any Internet connection. In order to keep the server lightweight for the Raspberry, whose capabilities remain limited, I chose not to use PHP, nor complex web servers such as Nginx or Apache. It is served by a JavaScript script run with Node.js.

server.js

It runs via Node.js directly on the Raspberry. Its role is to serve all the files needed to display the real-time data dashboard, i.e. `client.html`, `client.css`, `client.js` and `horizon.png`, on port 3000.

It listens for incoming MQTT messages on the specific topics (not those containing all the data), parses them and forwards them to each client via WebSocket. WebSocket was chosen because of its low latency, its ease of implementation, its lightweight execution and its robustness.

Its role is also to serve, on port 3001, `export.html`, `export.css` and `export.js`, the files making up the page used for downloading the data. When the database is downloaded, `server.js` does not transmit `sys.db` directly. Indeed, it could become corrupted because `db_recording.py` might be writing to it. It first makes a temporary copy of it, then sends it. The copy is deleted immediately afterwards. As for the videos, it modifies their metadata before sending them. It adds a precise timestamp to them (previously located in `start.txt`, next to the video in the directory). The videos are sent one after the other and sending is retried in the event of an error.

Since the logic contained in this code is more complex than in the rest of the project (notably queue management, robustness and Internet protocols), I relied more heavily on help from AI, through LLMs, to write the code. However, I still consider myself the main author of this code.

client.html

It contains the structure of the page. The data in the data display dashboard have specific attributes that can be retrieved via JavaScript so that the content of the tag can easily be replaced.

client.css

Contains the style of the data display dashboard.

horizon.png

This is the artificial horizon displayed by the dashboard to visually represent the roll and pitch values.

client.js

It is responsible for animating the artificial horizons and updating the DOM in `client.html`. To do this, it uses `querySelector` to retrieve the tags in the html according to the unique attributes given to them.

It must also calculate the values to be displayed from those received. From the SOG, the COG, the AWA and the AWS, it determines the VMG, the TWA and the TWS. These calculations have been moved to the client to relieve the Raspberry.

export.html

It contains the simple structure of the page.

export.css

It contains the style of the page. As it was of no particular coding interest but time-consuming to create properly, its creation was delegated to an AI.

export.js

It contains the logic for requesting the download of the videos and of the database, all with error handling. It also takes care of listing the videos, displaying their size, etc. Overall, the entire interactive part of the page relies on this file. Since this code works hand in hand with `server.js` (for the hotspot), part of that code's complex logic is also found here. As such, I also used LLMs to help me make these two pieces of code match. Once again, I nevertheless still consider myself the main author.

9.5 Remote server

The remote server is hosted on a subdomain of the website of a students' association, which its committee lends me. It is accessible at https://abraxas.example.org. The architecture designed to provide it with the training data stems from a significant technical constraint: it is generally impossible to have simultaneous access to the device's local WiFi and to the Internet. Even on devices with two network cards, where this would theoretically be possible, it is very complex to build something stable from there.

I therefore decided to use the memory of the phone or computer carrying out the transfer as a relay: the data (database and videos) are first stored in the device's downloads before being transferred again manually. This is a major drawback in terms of ergonomics, but to automate this transfer, it would have been necessary to go through an app. Indeed, web pages do not have the permissions needed for this type of automation.

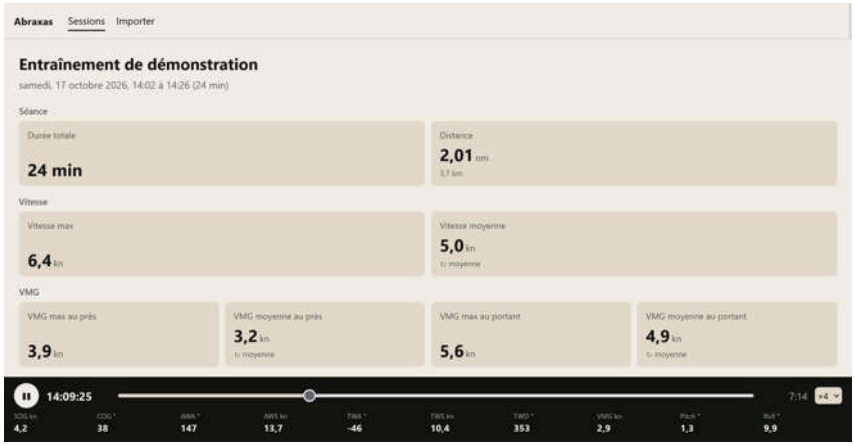


Figure 29: Beginning of the debrief of a mock training session

Server system

The server itself does not run on a standard Apache PHP stack, but on Node.js. I chose this system because of the excellent handling of asynchronous processes it offers. Indeed, each session must be analysed only once. Node.js is ideally suited to this type of application.

Operation and purpose of the server

For each uploaded training session, the server can produce a complete report. It goes beyond the simple reading of data and calculation of averages found in many solutions available on the market. It offers a synchronised visualisation of the wind, speed, position and heel data as well as the video. The various graphs and polars allow a detailed analysis of the data by the coach, while the AI summary allows the sailors to make sense of the recorded data by pinpointing the positive elements and those to be improved that they reveal.

The AI summary is generated once for each session using an API key stored as an environment variable in `.env`, at the root of the server. To interact with the server, I used SSH, as FTP is incompatible with Node.js.

The purpose of the server is not to replace the coach: although the data carry a great deal of information and make it possible to identify certain problems, they make it possible neither to analyse them in depth nor to determine their exact causes. This stems from the fact that sailing is a team sport; the crew dynamic cannot be neglected. The purpose of the AI debrief is to raise questions that the coach can answer and to help find one's way through a massive amount of data. The rest of the analyses are there to provide a tangible and easily analysable basis for the debrief given by the coach. The main thing here is not to forget that the AI debrief is partial, that it is only one element of a whole set of tools.

The exact content and structure of the debrief were developed following a discussion with Arnaud Psarofaghis[39], three-time world champion in Moth and GC32, skipper of the Swiss challenger Alinghi Red Bull Racing at the thirty-seventh America's Cup[40] and wing trimmer on the Swiss F50 at SailGP[41]. Accustomed to boats where electronics play a predominant role, he was able to tell me which data were the most useful while sailing or in a debrief and how to present them.

AI debrief

The AI debrief is generated with Claude Haiku 5.5. Its context window covers all the data collected during the training session, as well as the rest of the indicators calculated by the server. For reasons of privacy, energy efficiency and token economy, the video is not included in the context window.

The debrief is not generated freely: indeed, it follows a carefully considered and strict structure. Its purpose is not to make decisions about the approach to performance or about the type of debrief, but only to fill predefined boxes with text describing the measurements. The structure followed (provided in the prompt) is the following:

> 1. Executive summary
>
>   - the crew must understand in 30 seconds what happened and what deserves attention.
>
> 2. Conditions
>
>   - wind conditions: strength, direction, major shifts
>   - sea conditions: waves, swell if applicable
>
> 3. Speed and VMG analysis
>
>   - approximation of the boat's behaviour according to the wind angle
>   - small polar-curve-type analysis
>   - search for an operating zone of the boat
>   - analysis of the dispersion.
>
> 4. Analysis of the boat's stability
>
>   - mean and standard deviation of the roll;
>   - amplitude of the oscillations;
>   - frequency of the variations;
>   - correlation between roll and SOG;
>   - correlation between roll variation and speed variation;
>   - identification of periods when the boat becomes unstable;

[39] PSAROFAGHIS Arnaud, Professional racer, discussion of 22.09.2026.

[40] "Arnaud Psarofaghis", *Alinghi*, n.d., https://www.alinghi.com/crew/33_ArnaudPsarofaghis# (accessed on 21.10.2026).

[41] "Arnaud Psarofaghis", *SailGP*, n.d., https://sailgp.com/teams/switzerland/arnaud-psarofaghis/ (accessed on 21.10.2026).

- comparison between stable sailing and fast sailing.

5. Manoeuvre analysis
   - acceleration phases after a manoeuvre
   - speed before
   - minimum speed
   - maximum loss
   - time to recover 95 %
   - time to recover initial speed
   - TWA before
   - stabilised TWA after
6. Analysis of wind transitions
   - pressure variations
   - wind shifts
   - behaviour of the boat during these events
   - reaction of the crew
   - impact on performance
7. Consistency analysis
   - average speed
   - median speed
   - 10th / 25th / 75th / 90th percentile
   - dispersion
   - frequency of significant losses
   - time spent in a target performance zone
8. Summary
   - review of the important points

Role of the server within the project

For me, the server is not the centre of my project, quite the contrary. The debriefs and their display consist mainly of data analysis. What mattered most to me was the data collection, working with a microcomputer, working on Linux from the command line and, to a certain extent, the calculations directly related to the practice of sailing. These are the ones I carry out in the real-time data display dashboard, which I worked out by hand and detailed in Section 3.

Nevertheless, access to analyses of the recorded training sessions contributes greatly to achieving my initial goal, namely to enable real progress for the crews using this device. Moreover, it makes it possible to showcase the work I have done by offering a comfortable visualisation of it.

Server code

Faced with this situation – the need for an element not strictly part of my project, complex and long to develop – I decided, in agreement with my supervising teacher, to delegate its creation to an AI. I used Anthropic's Claude Opus 5.5 model to generate this code.

In this process, I nevertheless wanted to keep the greatest possible control over what was coded. Consequently, I left the LLM used almost no latitude in the chosen architecture, in the look of the UI and in the structure of the debrief, especially in the AI debrief section (each subsection was spelled out). Moreover, for the sake of security and rigour, I reread all the code provided to me, making sure I understood how it works. Thus, I am certain that, even though I am not the author of this code, I nevertheless remain in control of it.

Since I did not write this code myself, I will not describe it in full. It is accessible in the GitHub repository Abraxas_server. I do not consider it part of my Matura thesis. On the other hand, the result it creates, the tool it constitutes, is an essential building block in the system I am developing.

9.6 Abraxas OS

An image of the operating system after configuration was made on a Raspberry Pi Zero and on a Raspberry Pi Zero 2. It contains the modified version of *Raspberry Pi OS* that runs the device of the Abraxas project. It is called Abraxas OS. Instead of going through an automatic installation system on a blank OS, one can directly download the image of the desired version of Abraxas OS (Raspberry Pi Zero or Raspberry Pi Zero 2) and flash it onto an SD card. The SD card is inserted into the Raspberry, and the system is operational from the first boot, without any additional configuration.

10 TRIAL

The training report is accessible at https://abraxas.example.org/session?id=33.

A trial in real conditions was carried out on Wednesday 21 October 2026 at the Yacht Club de Genève. It took place between about 2pm and 3:30pm, on an Open 5.70 (light twin-rudder keelboat), in a variable southerly wind, with a strength oscillating between 12 and 20knt. The lake was fairly calm, the cloud cover heavy, becoming moderate, and the temperature was about 10 degrees. Moderate rain, becoming light, began during the training session. Two members of the racing team were trimming the headsails and the mainsail; I helmed the boat during the training session.

10.1 Installing the device

The anemometer was installed with a GoPro suction-cup mount directly on the hull. It held well throughout the training session. Attaching the box turned out to be more complex: it is designed to be attached to the mast, but the Open's rotating mast did not lend itself to this. We finally attached it with a suction cup as well, which turned out to be very stable and rigid. The installation took us about fifteen minutes, but, now that we know how to proceed, we would probably be much quicker. In the end, the various cables did not bother us at all and, overall, we forgot about the device while sailing. It worked well while sailing: one press on the button was enough to start the session, and another to stop it.

10.2 Course of the training session

We had planned to string manoeuvres together in order to test the analysis capabilities of the system. After leaving the Yacht Club, we started by sailing up to the level of the Eaux-Vives beach. The first upwind leg went well; the system required no particular attention, allowing us to concentrate on sailing. During this upwind leg, I checked the dashboard presenting the live measurements; they seemed consistent. The wind gradually increased as we approached Geneva, which is usual in this type of wind regime. During this first upwind leg, we had a failure in the steering system (at time 04:07:55 in the dashboard): the port ball joint of the tiller cross-bar broke. This part connects the boat's two rudders. Without it, it is still possible to sail, but the boat is much harder to manoeuvre. Both tillers must be held by hand.



Figure 30: Abraxas attached to the deck with a GoPro suction cup.



Figure 31: View of the report at the moment the ball joint broke.

Once we reached the edge of the sailing area, we hoisted the spinnaker and began the downwind leg. The manoeuvre presented an unexpected difficulty: part of the sail had wrapped around the pole. This problem was quickly solved. During this downwind leg, the rain set in and grew heavier. The waterproofing of the case was put to the test. The wind dropped as we moved away from Geneva, as expected. We did not go much further down than the Yacht Club.

During the second upwind leg, we sailed in a group with the rest of the team. This was an opportunity to test our manoeuvres and our speed against other boats. During this upwind leg, an incident occurred that put an end to the training session (at time 05:06:30 in the dashboard). The port tiller gave way off the Yacht Club. This critical failure of the steering gear removes all means of controlling the boat's course. The priority was to drop the spinnaker in order to bring ourselves and the boat to safety. After that, the recording was stopped and we were towed back to the harbour by a RIB.

10.3 Technical operation of the device

The operation of the device itself turned out to be largely in line with expectations. The video quality is good, and the position, speed and orientation measurements are present and accurate. The only problem relates to the wind data: the anemometer recorded almost all of the sailing as being downwind. It detected almost no upwind sailing. The fact that a few TWA measurements are nevertheless positive shows that the problem does not come from a calculation error that would fold all the values back into the negative range. My hypothesis is that the positioning of the anemometer is the cause: on the hull, the airflow it picks up is highly disturbed and deflected by the sails. To remedy this problem, my first attempt would be to place the anemometer at the top of the mast, where the air is undisturbed.

Overall, I consider that if my device withstood this training session, it will be able to withstand all the others. On its very first outing, it went through rain, wind, a broach, a gear failure and an emergency tow, and does not seem to have suffered any particular damage.

10.4 Quality of the analysis and of the visualisation

Without analysable wind data, the whole part relating to the VMG and to the polar curves, one of the most interesting, is lost. As for the analysis of the manoeuvres, there are many false positives and false negatives because of the polluted wind data. The analyses of those that were correctly detected are relevant. All the analyses relating to speed and orientation are not only correct, but also particularly precise. The visualisation of the position is good. Overall, the synchronisation of the different visualisations is good (once corrected) and the interface works properly on phones as well as on computers.

At the end of the training session, I started the data transfer while we were getting changed. It was finished and the interface was ready when we came out of the changing rooms, which is a perfectly acceptable time, since it would have allowed a debrief to be held without waiting.

10.5 Drawbacks of using the device for the crew

The only two drawbacks for the crew during this training session were the set-up time at the beginning of the session and the constraint of having to remember to bring an additional device to training and to make sure it is charged. I consider these two drawbacks easily surmountable, because the set-up, once known and with the appropriate mounts in place, will be much quicker. As for having to carry a charged electronic device, it is analogous to what everyone does every day with their mobile phone.

10.6 Benefits of using the device for the crew

In the context of this particular training session, marked by gear failures and considerably shortened, it was not possible to carry out any real constructive performance analysis. It was, however, possible to reconstruct exactly the two incidents that occurred, to understand them better and to explain to our coach precisely where, when and how these incidents had happened. This offers a very promising outlook for performance analysis. Even at this stage, without perfectly usable information on the wind, we can see that the map, the key points at the beginning of the report and the video are extremely useful and pleasant elements. The synchronisation between the different visualisations is a real plus compared to the other manual combinations of tools we had been able to try. Overall, I am pleased with the possibilities I can see with this system for our crew.

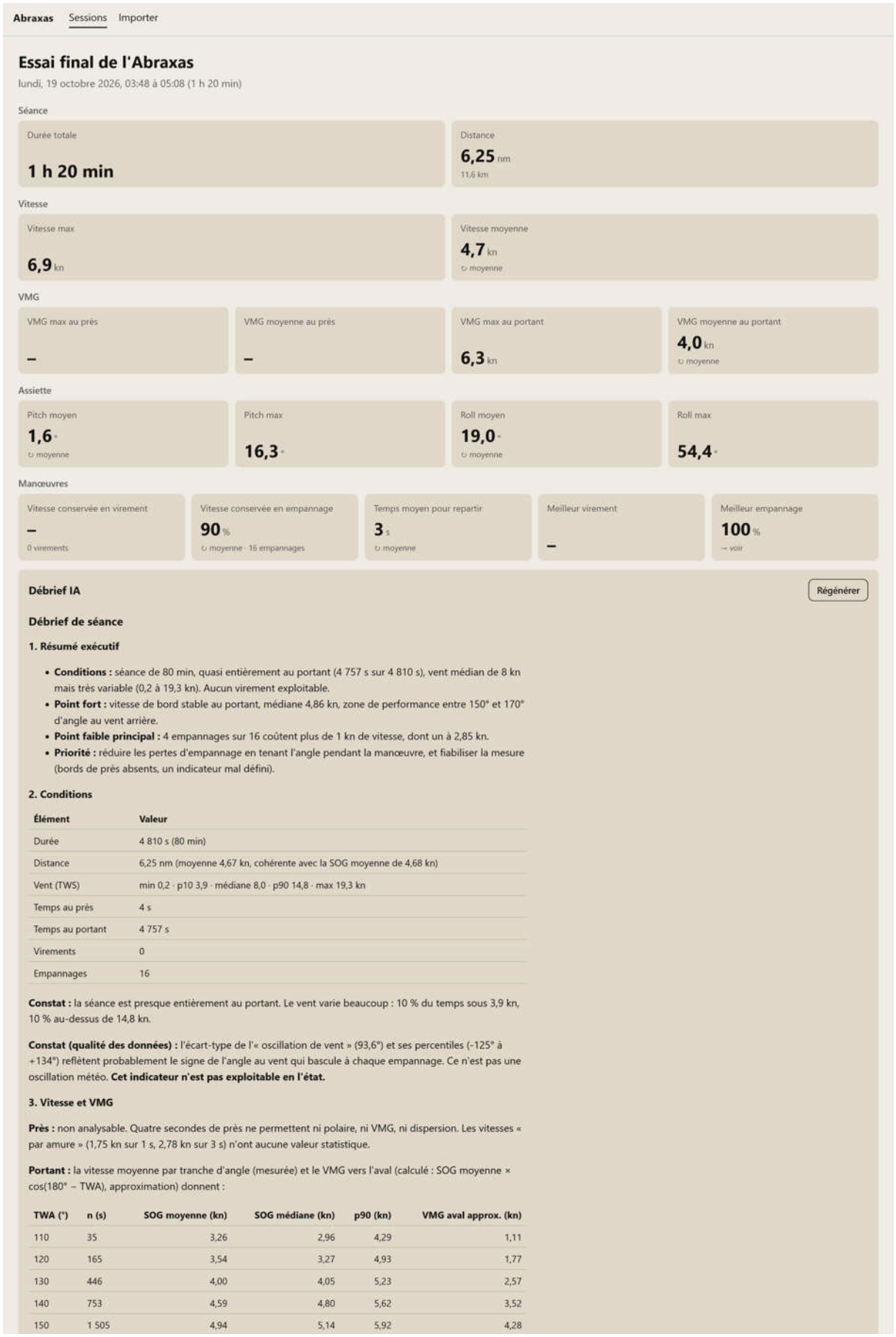


Figure 32: View of the debrief page, at 05:01:26, during an upwind leg (part 1).

150	1 505	4,94	5,14	5,92	4,28
160	**951**	**4,98**	**5,15**	**5,94**	**4,68**
170	460	4,75	4,87	5,73	4,68
180	442	4,63	4,66	5,77	4,63

Constat :

- La vitesse monte régulièrement de 110° à 160°, puis recule légèrement à 170° et 180°.
- Le VMG aval forme un plateau entre 160° et 180° (≈4,6–4,7 kn). Il chute nettement sous 140°.
- 61 % du temps (2 916 s sur 4 757) est passé entre 150° et 170°.
- VMG aval moyen 4,02 kn, médian 4,20 kn, max 6,34 kn.

Hypothèse : au-delà de 160°, le gain de VMG vers l'aval devient marginal. Une partie du temps passé à 150–160° pourrait être gagnée en route directe, mais cela reste à confirmer par la mesure du cap et de la route.

Dispersion (SOG) : p10 3,18 · p25 4,03 · p50 4,86 · p75 5,42 · p90 5,79 kn. L'écart interquartile est de 1,39 kn, ce qui est large : la vitesse dépend fortement des conditions ou de la conduite.

4. Stabilité

Indicateur	Valeur
Gîte absolue moyenne	19°
Écart-type de la gîte	12,4°
Amplitude p5–p95	37,6°
Tangage moyen	1,6°
Écart-type du tangage	3,1°
Corrélation gîte / vitesse (« près »)	0,96

Constat : la gîte est forte et variable (amplitude de 38°). Le tangage est faible et régulier. Les périodes instables « au près » ne sont pas calculables (aucune fenêtre listée, 4 s de près).

Constat (incohérence) : la corrélation de 0,96 est étiquetée « près », alors qu'il n'y a que 4 s de près. Elle a donc probablement été calculée sur toute la séance. **Ne pas l'interpréter comme une relation gîte-vitesse au près.**

Hypothèse : gîte et vitesse montent ensemble avec la pression de vent (rafales). Une corrélation élevée ne prouve pas que la gîte fait gagner ou perdre de la vitesse.

Constat (empannages) : les gîtes les plus fortes avant empannage (38,4° ; 42,5° ; 39,9°) n'ont pas produit de perte importante (0,29 ; 0,71 ; 0,02 kn). À l'inverse, des pertes de 1 kn ou plus se sont produites à 5–11° de gîte. **La gîte avant manœuvre ne prédit pas la perte.**

5. Manœuvres

Virements : aucun. Rien à analyser.

Empannages (16) :

Indicateur	Valeur
Rétention médiane (v_min / v_entrée)	0,951
Perte médiane (calculée)	≈ 0,22 kn
Perte maximale	2,85 kn (t = 1420 s)
Empannages avec perte > 1 kn	4 / 16
Empannages avec perte ≥ 0,5 kn	6 / 16
Reprise à 95 % mesurée (non nulle)	6 cas : 3, 4, 7, 13, 19, 20 s

Empannages les plus coûteux :

t (s)	Vitesse entrée → min (kn)	Perte (kn)	TWA avant → après (°)	Reprise 95 % (s)
1420	4,63 → 1,78	2,85	157 → 119	19
1400	5,51 → 4,40	1,12	141 → 157	0 (incohérent, voir ci-dessous)
1307	5,54 → 4,47	1,07	159 → 143	13
3101	6,28 → 5,22	1,06	158 → 147	20

Constat :

- Le groupe 1307–1449 s concentre 4 empannages en 142 s, dont 3 avec perte. La vitesse descend à 1,78 kn à 1420 s. Le 1449 s est entré à 1,86 kn, sans perte mesurée car déjà bas.
- Les événements avec forte perte s'accompagnent d'un changement d'angle important (11 à 38°). Les empannages « propres » (2 à 4° de variation) coûtent peu, voire rien.
- Les 3 empannages des 3924 s, 4501 s et 4290 s ont peu coûté, malgré une gîte élevée ou une entrée à vitesse moyenne.

Hypothèse : les pertes importantes viennent d'une sortie de cap ou d'une conduite hésitante pendant la manœuvre, plus que de la manœuvre elle-même. L'épisode 1400–1449 s pourrait être lié à une phase de vent faible, mais aucune transition de vent n'est enregistrée à cet instant.

Constat (incohérence) : l'empannage à 1400 s a une perte de 1,12 kn mais une reprise de 0 s. Soit la définition de « reprise » est différente, soit la donnée est fausse. **À vérifier avant de la comparer.**

Recommandation : pour chaque empannage, relever l'angle au vent et la vitesse en 3 instants (avant, pendant, après) et conserver la même fenêtre de mesure pour toutes les manœuvres.

Figure 33: View of the debrief page, at 05:01:26, during an upwind leg (part 2).

6. Transitions de vent

Douze transitions sont enregistrées, toutes entre 16 s et 457 s. **Aucune transition n'est détectée après 457 s** (couverture limitée à ~10 % de la séance).

Type	n	Δ vent moyen (kn)	Δ SOG moyen (kn)	Δ gîte moyen (°)
Risées	4	+3,5	+0,34	+1,4
Molles	8	–4,6	–0,03	+2,4

Constat :

- Les risées (+3,2 à +3,8 kn) font monter la SOG en moyenne de 0,34 kn, de façon inégale (3 hausses sur 4).
- Les molles (–3,1 à –7,7 kn) ne font presque pas baisser la SOG en moyenne. Elles s'accompagnent de hausses et de baisses.
- La réaction maximale observée est de +0,69 kn et –0,65 kn.

Hypothèse : la réponse de vitesse est modeste, mais la fenêtre de mesure de la SOG après chaque transition n'est pas précisée. Un retard de réaction ou une mesure trop courte peut masquer l'effet. La molle à 457 s (–7,7 kn) n'a produit aucune baisse visible.

7. Régularité

Indicateur (SOG, kn)	Valeur
Moyenne	4,68
Médiane	4,86
p10 / p25 / p75 / p90	3,18 / 4,03 / 5,42 / 5,79
Part du temps à ± 0,25 kn de la médiane	non calculable (le champ n'est renseigné que pour le près, 0 s)

Constat : la médiane (4,86) est supérieure à la moyenne (4,68). La distribution est étirée vers le bas : les 10 % les plus lents descendent sous 3,18 kn, ce qui correspond aux phases de vent faible et aux empannages.

Constat : la part du temps dans une zone cible n'est pas calculable faute de zone cible définie pour le portant. On peut seulement dire que 61 % du temps est passé entre 150° et 170° d'angle au vent arrière.

8. Synthèse

Forces

- Vitesse portante stable, médiane 4,86 kn.
- Angle de performance clair : VMG aval en plateau entre 160° et 180°, meilleure vitesse à 160°.
- Bonne conduite générale : la plupart des empannages coûtent moins de 0,4 kn.

Faiblesses

- 4 empannages sur 16 coûtent plus de 1 kn, dont un à 2,85 kn.
- Vitesse basse et dispersée (p10 à 3,18 kn).
- Aucune donnée de près : performance au près inconnue.

Points à investiguer

- Causes des empannages du groupe 1307–1449 s (conduite, vent, trajectoire).
- Phase de vent faible vers 1400–1450 s.
- Définition exacte de la « reprise » et des corrélations (incohérence à 1400 s et sur la corrélation 0,96).
- Signification de l'indicateur d'oscillation de vent (signe de l'angle).

3 priorités

1. **Empannages :** tenir l'angle au vent pendant l'empannage (variation cible < 10°) et éviter les séquences de plusieurs empannages rapprochés en conditions faibles.
2. **Angle de vent arrière :** se maintenir entre 150° et 170°, avec un VMG aval proche du plateau, plutôt que de descendre à 180° ou de monter sous 140°.
3. **Mesure :** enregistrer des virements (au moins 20 pour une analyse exploitable), corriger le TWA signé, couvrir toute la séance par la mesure du vent, et définir précisément les indicateurs de reprise.

Objectifs quantifiés pour la prochaine séance (à comparer à cette séance)

Indicateur	Actuel	Objectif
Empannages avec perte > 1 kn	4 / 16	≤ 1
Empannages avec perte ≥ 0,5 kn	6 / 16	≤ 2
Rétention médiane des empannages	0,951	≥ 0,97
Part du temps entre 150° et 170°	61 %	≥ 70 %
SOG médiane au portant	4,86 kn	≥ 4,86 kn (sans baisse)
Virements analysables	0	≥ 20

Indicateurs à suivre à chaque séance

- Perte de vitesse et temps de reprise (95 %) par empannage, avec la variation d'angle associée.
- Distribution de l'angle au vent arrière (part par tranche de 10°).
- SOG médiane et p10, en distinguant vent faible (< 6 kn) et vent établi.
- Réaction de la SOG aux risées et aux molles, sur une fenêtre fixe et documentée.
- Gîte moyenne et amplitude, mais sans conclusion sur la vitesse tant que la corrélation n'est pas confirmée sur plusieurs séances.

Généré par claude-haiku-5-5 le 21.10.2026 17:20:55. Basé sur cette seule séance : les causes proposées sont des hypothèses.

Figure 34: View of the debrief page, at 05:01:26, during an upwind leg (part 3).

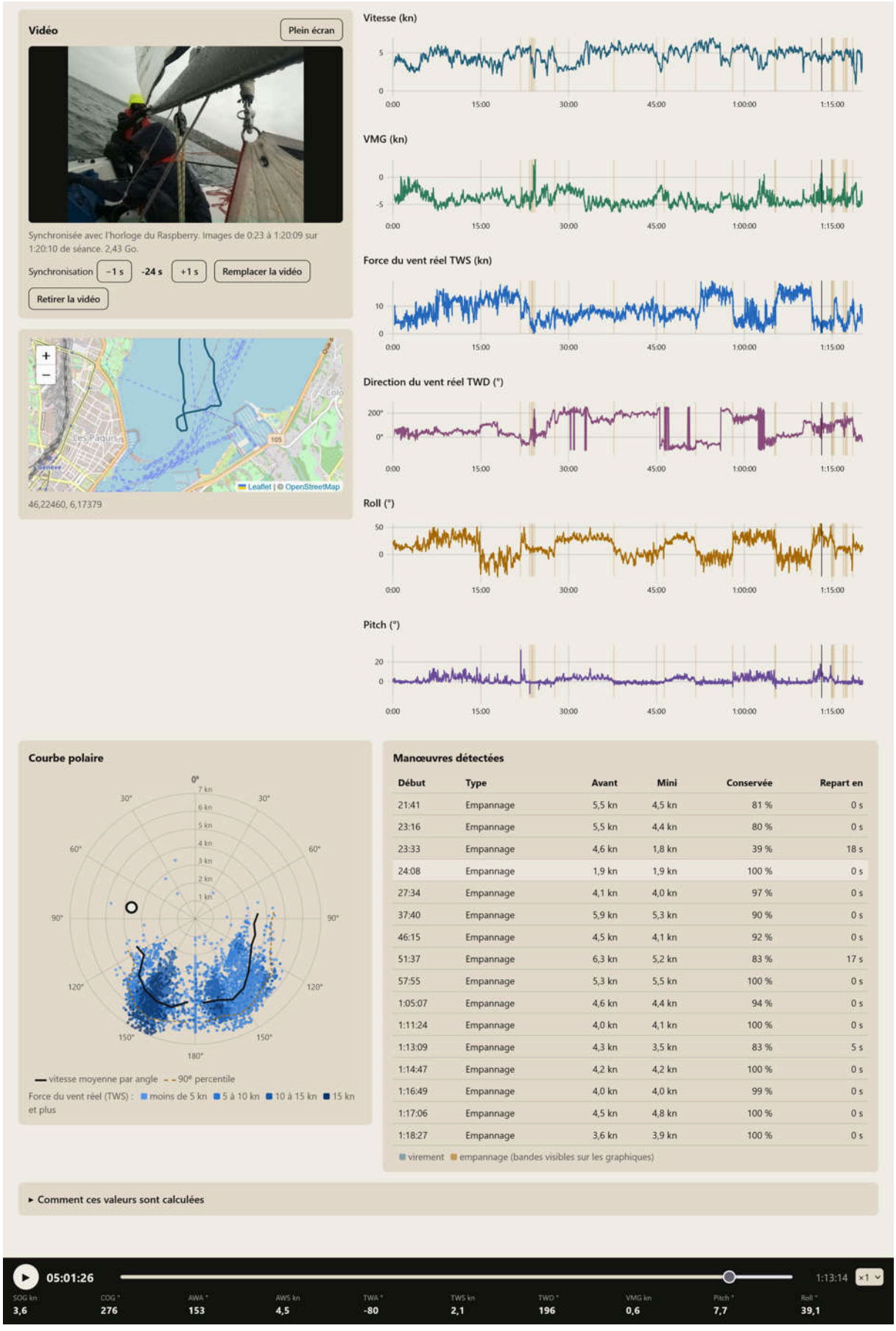


Figure 35: View of the debrief page, at 05:01:26, during an upwind leg (part 4).

11 CONCLUSION

A little over a year after the 2025 Critérium Longtze, we won the Belle Étoile 2026[42]. During the two seasons separating these two events, our training and debriefing methods and our approach to tactics were thoroughly reworked. The differences in both speed and angles are observable and quantifiable. Nevertheless, technique is not the only thing that has changed. The crew has also learned to know each other better and to work better as a group. Part of the difference also comes from the new dynamic that has taken hold. Performance in sailing is an alchemy, and only part of what it is made of can be quantified.

11.1 Summary

The ambition of this project was to create a device for recording data during an inshore sailing training session in order to provide tangible support for the debriefing and, in doing so, to give the team using it the opportunity to progress.

To do this, I first had to define which data to collect, then design a device capable of doing so, and finally build it. My project was organised according to the following structure: identify the problem, give oneself the means to remedy it, implement a solution. In doing so, I kept in mind the five requirements I had set myself at the beginning of my work:

- Affordability: while the solutions on the market have price ranges from 600 to 1100CHF (for the direct equivalents, Orca and Vakaros), I estimate the manufacturing cost of a second model identical to the first at about 300 to 400CHF. This price is difficult to estimate, because many of the components I used were ordered separately, which artificially increased their price because of the multiple deliveries. Moreover, if real production were launched, this price could be drastically reduced.

- Adaptability to clubs: the device can easily be mounted and removed on different types of boats (mounting tested on Longtze, Skeeta, Viper F16, Nacra F16 and Open 5.70). Using it for the racing team in no way complicates the use of the boat for other lessons. The only notable drawback is the presence of the anemometer cable, as described below.

- Open Source: the project is placed in its entirety under the MIT license, including this report. The entire work is available in the repositories GitHub Abraxas, Abraxas - server, Abraxas - 3D files and Abraxas - PCB.

- No-nonsense: the results are clear, direct and easily usable. Only the break in ergonomics in the display of the debrief, presented below, is to be regretted.

[42] First place in real time on the short course, first place in corrected time on the short course, first place in TCF3. “Classement de la Belle Étoile 2026”, *Manage2Sail*, 25.08.2026, https://chk.me/tlAsHST (accessed on 21.10.2026).

- Multifactorial approach: all the factors mentioned in the specifications are measured. It would, however, be possible and interesting to widen this window to other parameters, as we will see later.

The assessment of the work itself can be made on two main levels:

On the technical level

The work itself is a success: the prototype works as expected. It could be argued that the duration of the video saving and of the device start-up could be optimised. Moreover, the time used by the Raspberry is not correct, because its clock stops when it is switched off. However, these problems do not hinder the proper functioning of the device.

On the level of the benefit for the team

The goal of data collection and of the possibility of analysis has been achieved. The debrief display panel is functional; the debrief itself can rely on solid data and makes it possible to gain clarity during a discussion about the sailing. However, regular use of the device is limited by two significant ergonomic problems. First, installing the device involves connecting the anemometer to the case with a cable. This is an additional element to be careful of while sailing, which cannot be asked of a team in regular training. Moreover, the fact that, to access the debrief, one must first download the files to one's phone before transferring them to the server is a break in ergonomics that may discourage regular use.

Despite these two possible points of improvement, I consider my initial goal achieved, in the sense that I have provided an answer to my research question: I identified how it was possible, using an electronic device, to collect data in order to give a team the opportunity to improve. I then built such a device and verified that it actually represented an advantage for the team using it, as could be seen during the training session described in Section 10.

11.2 Improvements and outlook

Resolving these two breaks in ergonomics would be a major improvement for the system. As for removing the cable, the architecture would not have to be modified. Indeed, I already use MQTT for data transfer, and I already have an open network to which it is possible to connect. Thus, it would be enough to connect the anemometer to a second Raspberry and to pass the MQTT messages over WiFi, which is natively supported. As for downloading the files locally, setting up an automatic transfer system would require developing an app, because web pages do not support managing the choice of network, nor the automations needed to transfer these files. Remedying this problem represents a development that goes beyond the scope of this project.

At the end of this project, I note that some data would have been at least as interesting, if not more so, to measure. Indeed, the tension in the sheets, in the cunningham, in the vang and in the outhaul[43] are extremely important indicators for understanding the boat's behaviour and for the reproducibility of performance. It would also have been interesting to film the sails themselves in order to keep a record of their shape at a given instant t. It would be interesting to create a system of accounts on the data analysis server so that several different teams could access their own debrief privately.

11.3 Final thoughts

Finally, I would like to point out that this system does not aim to replace the coach in a team. I believe that sailing is and will remain, despite the appearance of ever more electronics on boats, a fundamentally human sport. There is one dimension of performance that cannot be measured: the crew dynamic. There comes a moment when, once knowledge of each other and mutual trust within the crew are total, a glance is enough to make a decision. Actions follow one another naturally, coordination becomes self-evident, because you know your teammate as well as yourself and none of their actions comes as a surprise. In situations where the boat is fast, the wind strong and the sea rough, adrenaline, and sometimes even fear, are among the elements that must be dealt with to achieve optimal performance. Confidence in oneself and in the other is just as essential as a successful gybe. The work of a coach also lies there, and I am convinced that no computer system, not even the most modern AI, can replace that.

[43] Different settings affecting the shape of the sail.



Figure 36: Downwind leg during the Belle Étoile 2026. We are on the Longtze Osteo 7/7.
Untitled, 21.08.2026, private collection,
with the photographer’s permission.

12 ACKNOWLEDGEMENTS

I would particularly like to thank my supervising teacher for this work, for his availability, his time and his advice, both technical and concerning the approach and structure of the project. More generally, I would like to thank him for the sincere interest he took in my work, and for sharing some of his own projects with me. I found in him not just an adviser, but someone to collaborate with and to exchange technical ideas with.

Thanks also to my CAD teacher, head of the film photography laboratory and friend, for making the school's 3D printer available to me and for guiding and advising me in the design of my case. More broadly, my thanks go to all the teachers who brought me curiosity, method and knowledge during my years at the school.

Thanks to my coach, for his sailing and technical advice, and above all for his unfailing friendship. I would also like to thank the base manager of my club, for the interest he took in my work.

Thanks to Arnaud Psarofaghis and to the philosopher consulted on the use of AI, specialists in their respective fields, for their enlightened advice.

Thanks to my parents for their support, their interest in my project, the courage they showed in proofreading the most technical parts and their patience when I was completely absorbed in my work.

Thanks to my teammate, thanks to whom I know that sailing is, in racing as elsewhere, above all a matter of trust and friendship. He taught me that no sensor can capture what a crew truly is, on the water as on land.

Thanks to my two female teammates for being there during the turbulent first hours of testing of the project, and, above all, for having brightened them with your laughter. You showed me that it is when no problem can wipe away a smile that one is ready to set sail.

Thanks to my other teammates and to the members of the club. I have sailed with you, and without you sailing would not mean the same thing to me.

Finally, thank you to all of you whom I could not name, but without whom this work would not be the same.



Figure 37: Photograph taken during SailGP 2025 in Geneva.

13 BIBLIOGRAPHY

13.1 Books

- HESSE Hermann, *Demian. Die Geschichte von Emil Sinclairs Jugend*, Berlin, Suhrkamp, 2012.

13.2 Interviews and oral sources

- Philosopher specialising in AI-related issues, discussion of 09.06.2026.
- Coach of the racing team of the Yacht Club de Genève, *Theory lessons, briefing and debriefing*, unpublished oral knowledge.
- PSAROFAGHIS Arnaud, Professional racer, discussion of 22.09.2026.

13.3 Existing systems

- “Wearables smartwatches”, *Garmin*, n.d., https://www.garmin.com/fr-CH/c/wearables-smartwatches/ (accessed on 11.10.2026).
- “Get Orca”, *Orca*, n.d., https://getorca.com/product-configuration/ (accessed on 11.10.2026).
- “Atlas 2”, *Vakaros*, n.d., https://www.vakaros.com/en-eu/collections/atlas-family/products/atlas2 (accessed on 11.10.2026).

13.4 Technical documentation and code repositories

- HERRADA Eva, “Adafruit Ultimate GPS”, *Adafruit*, 20.08.2026 (last updated), https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps.pdf (accessed on 21.10.2026).
- “Adafruit Powerboost 1000C”, *Adafruit*, 08.03.2024, https://learn.adafruit.com/adafruit-powerboost-1000c-load-share-usb-charge-boost (accessed on 21.10.2026).
- “Camera software”, *Raspberry Pi*, n.d., https://www.raspberrypi.com/documentation/computers/camera_software.html (accessed on 21.10.2026).
- HERRADA Eva and LEBLANC Williams, “Running CircuitPython Code without CircuitPython”, *Adafruit*, 05.07.2023 (last major update), https://chk.me/O2xfxk0 (accessed on 21.10.2026).
- HERRADA Eva and LEBLANC Williams, “Installing Blinka on Raspberry Pi”, *Adafruit*, 05.07.2023 (last major update), https://chk.me/Y40lkS7 (accessed on 21.10.2026).
- “OnOff SHIM”, *Pimoroni*, n.d., https://shop.pimoroni.com/en-ch/products/onoff-shim (accessed on 21.10.2026).
- HERRADA Eva *et al.*, “Adafruit_CircuitPython_GPS”, *GitHub*, 09.09.2026 (last commit), https://github.com/adafruit/Adafruit_CircuitPython_GPS (accessed on 21.10.2026).
- HOWARD Philip *et al.*, “icm20948-python”, *GitHub*, 28.06.2024 (last commit), https://github.com/pimoroni/icm20948-python (accessed on 21.10.2026).

- “User Manual”, *Hongyuv*, 30.04.2024, https://www.hongyuv.com/download/WDC2E_Manual.pdf (accessed on 21.10.2026).
- “Calculateur de largeur de piste de circuit imprimé”, *DigiKey*, n.d., https://www.digikey.be/fr/resources/conversion-calculators/conversion-calculator-pcb-trace-width (accessed on 21.10.2026).

13.5 Articles and web pages

- “Anémomètre” in *Wikipedia*, n.d., https://chk.me/XaOHDIC (accessed on 21.10.2026).
- AVONTURE Christophe, “Linux - Using a progression bar in your script”, *avonture.be*, 07.09.2024, https://www.avonture.be/blog/bash-progression-bar/ (accessed on 21.10.2026).
- EYGAY Erwan, “JSON ou SQL : Quel format choisir pour gérer vos données ?”, *DataBird*, 19.03.2026 (last updated), https://www.data-bird.co/blog/json-ou-sql (accessed on 21.10.2026).
- FAULKNER Andrew and BLANC Florian, “Avantages et faiblesses du protocole SSH”, *Hakin9*, n°2, 2008, https://chk.me/qm9L8t2 (accessed on 21.10.2026).
- “Arnaud Psarofaghis”, *Alinghi*, n.d., https://www.alinghi.com/crew/33_ArnaudPsarofaghis# (accessed on 21.10.2026).
- “Arnaud Psarofaghis”, *SailGP*, n.d., https://sailgp.com/teams/switzerland/arnaud-psarofaghis/ (accessed on 21.10.2026).

13.6 Regatta results

- “Classement de la Belle Étoile 2026”, *Manage2Sail*, 25.08.2026, https://chk.me/tlAsHST (accessed on 21.10.2026).
- “Classement du Criterium Longtze YCG 2025”, *Manage2Sail*, 04.05.2025, https://www.manage2sail.com/fr/event/Longtze2025 (accessed on 21.10.2026).

14 APPENDICES

14.1 Photos of the prototype



Figure 38: Mounting the anemometer on the deck of the Longtze with a suction cup. Untitled, 22.10.2026, private collection, with the photographer's permission.

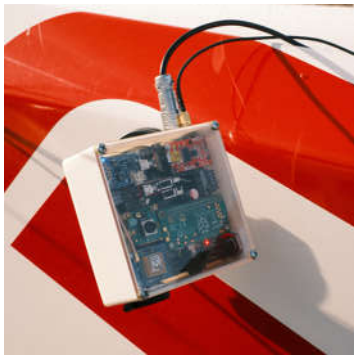


Figure 39: Abraxas on the bow of a Viper F16.

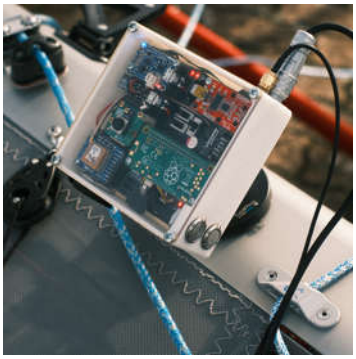


Figure 40: Abraxas on the beam of a Nacra F16.



Figure 41: Abraxas on the mast of a Longtze.

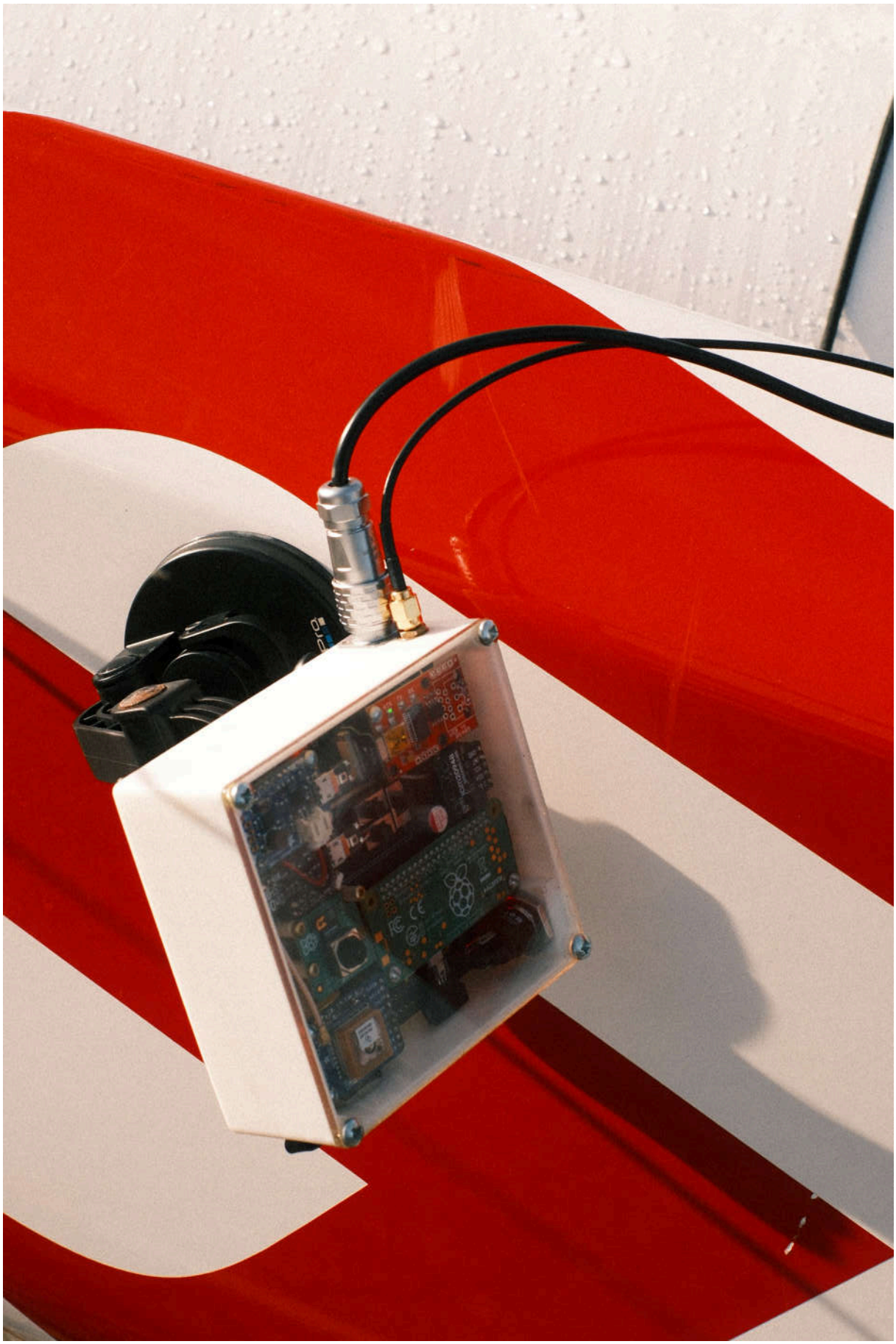


Figure 42: Abraxas on the bow of a Viper F16.



Figure 43: Abraxas on the bow of a Viper F16.

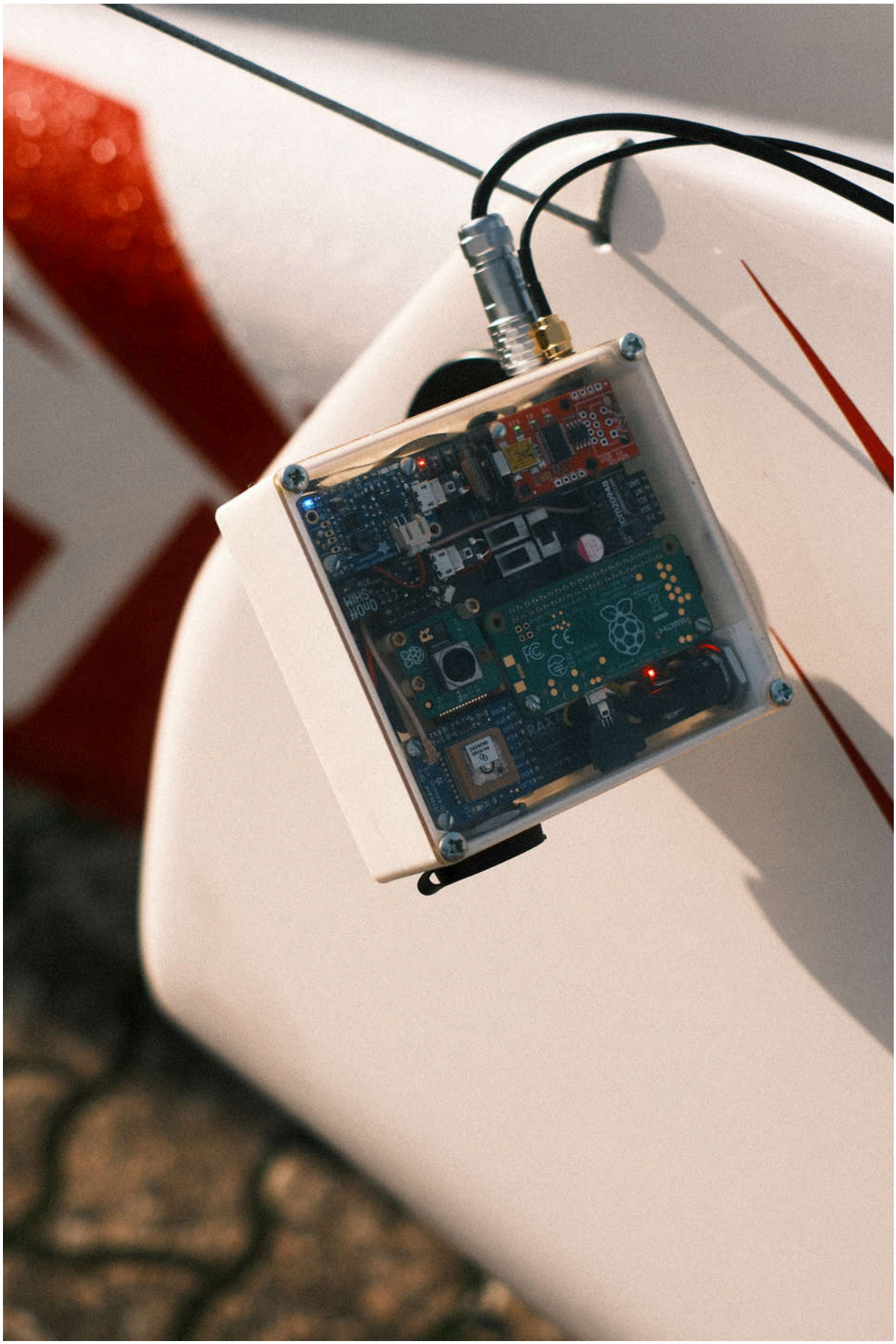


Figure 44: Abraxas on the bow of a Viper F16.

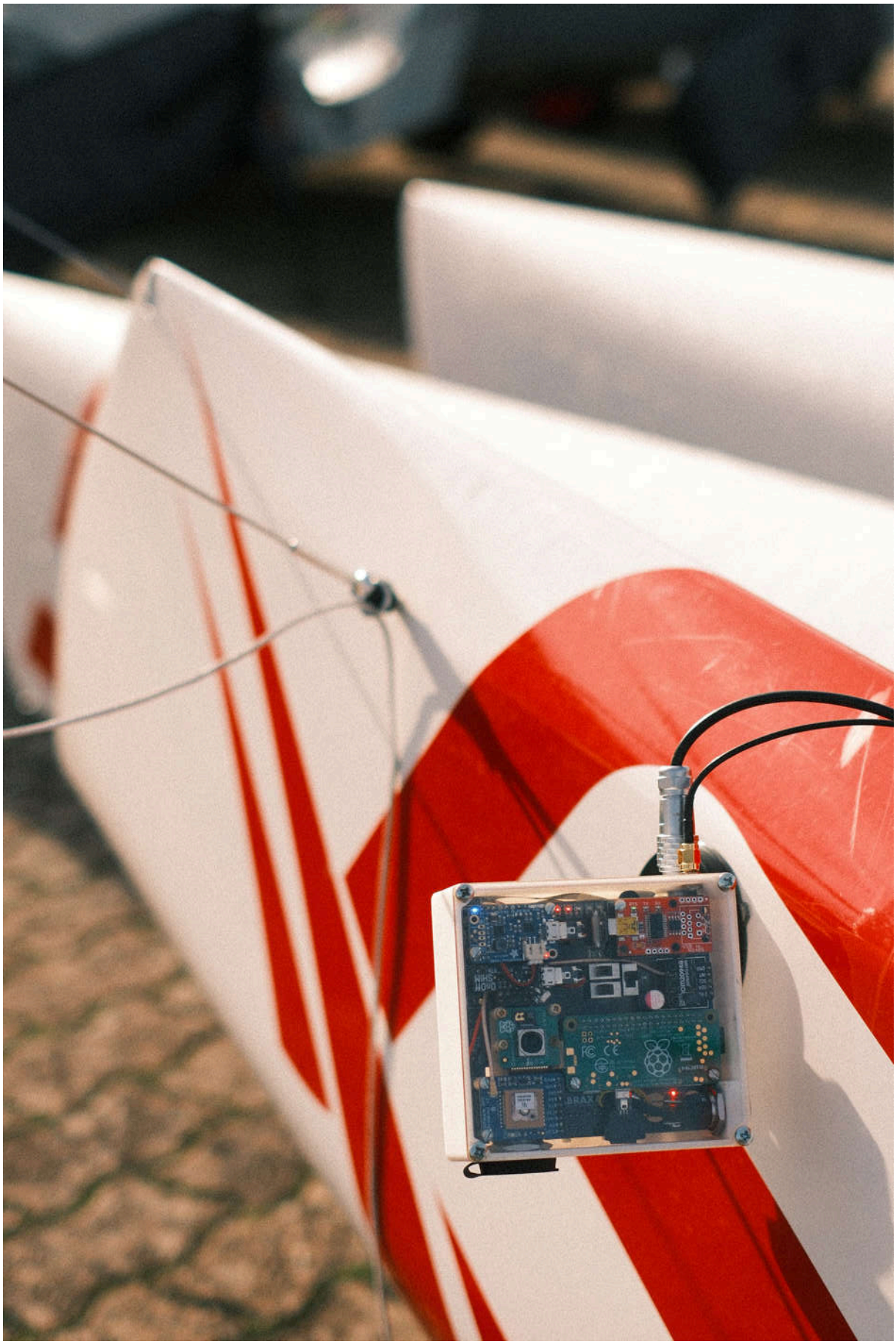


Figure 45: Abraxas on the bow of a Viper F16.



Figure 46: Anemometer on the bow of a Viper F16.



Figure 47: Anemometer on the beam of a Nacra F16.

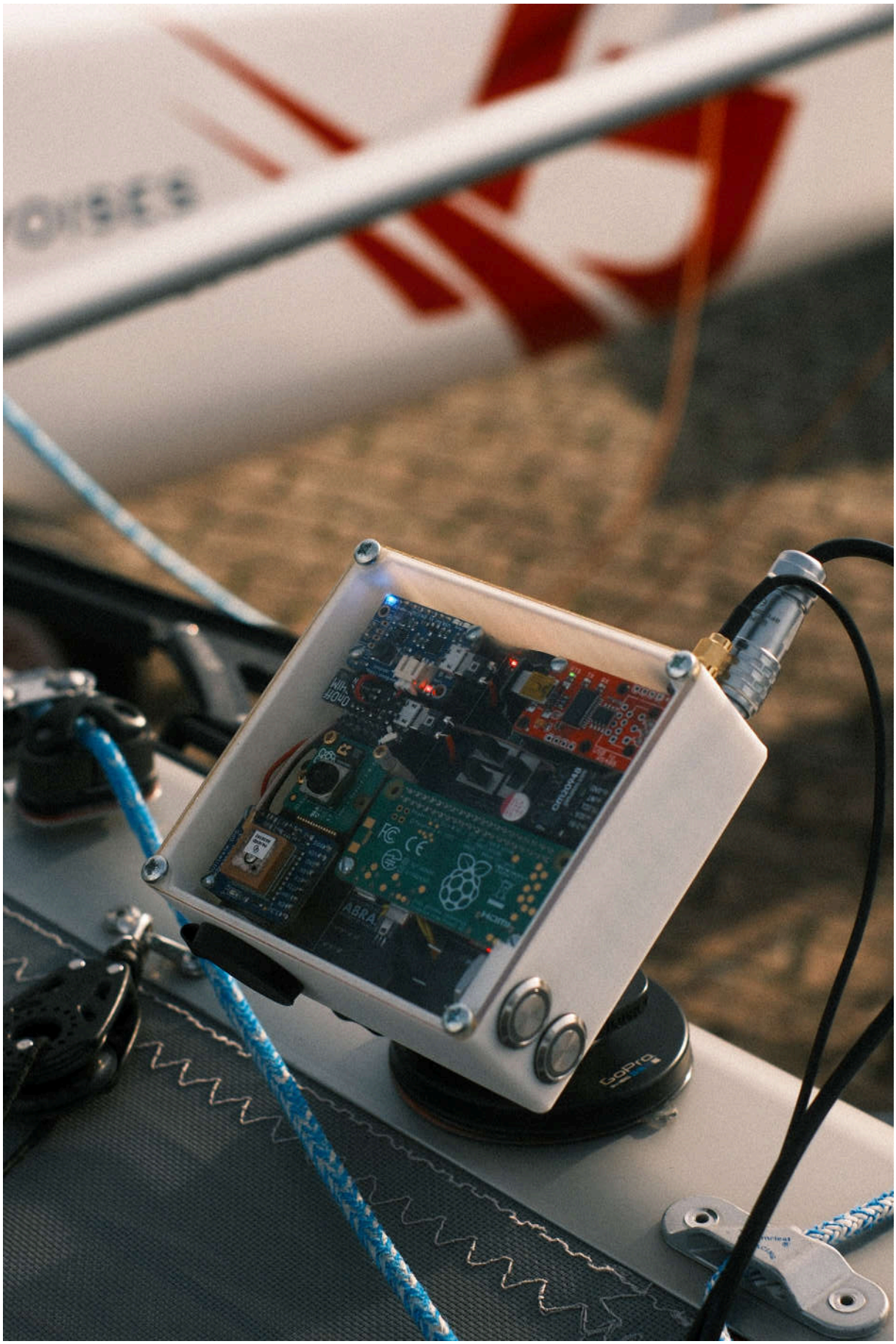


Figure 48: Abraxas on the beam of a Nacra F16.

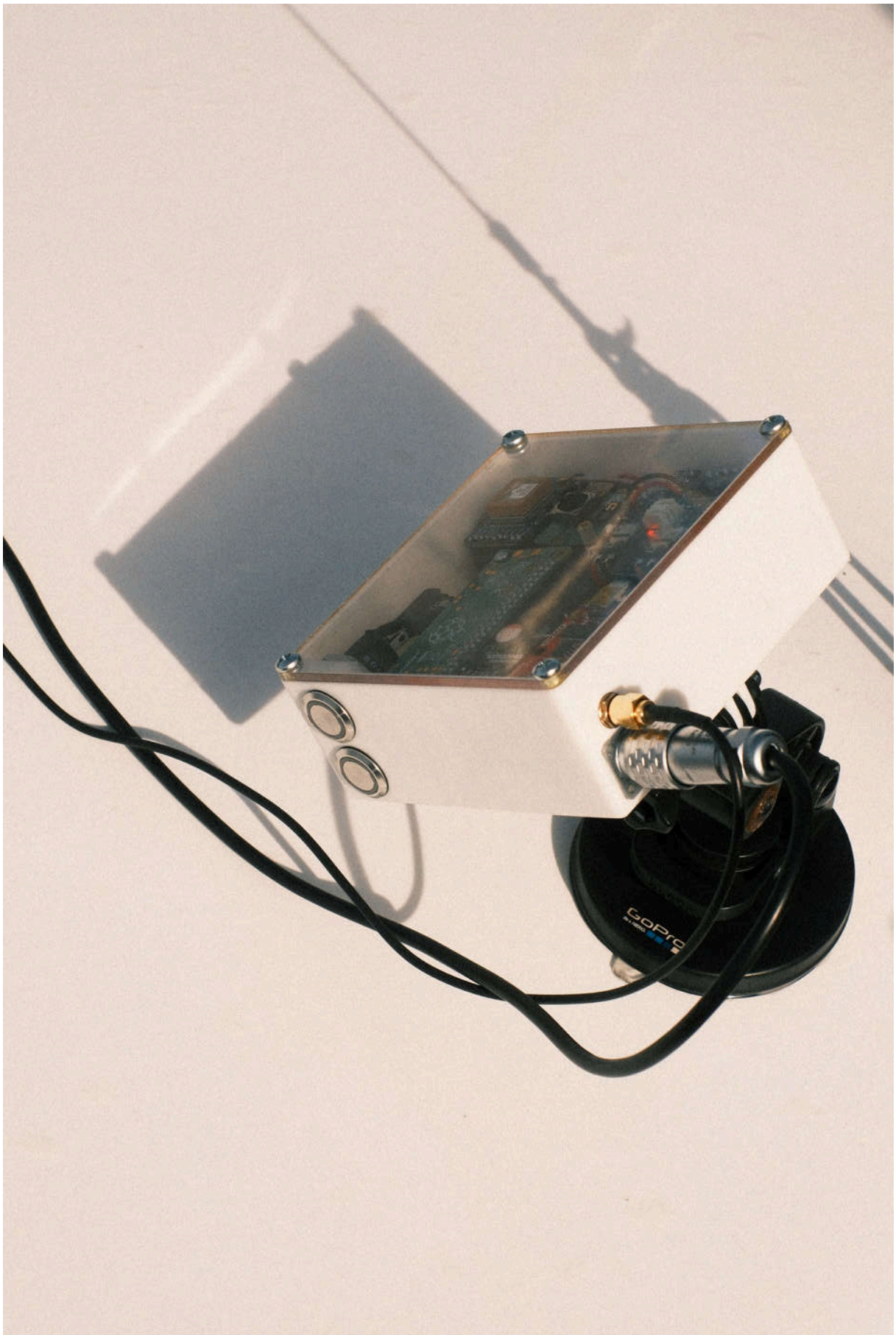


Figure 49: Abraxas on the foredeck of a Skeeta.



Figure 50: Anemometer on the foredeck of a Skeeta.



Figure 51: Anemometer on the foredeck of a Longtze.



Figure 52: Anemometer on the foredeck of a Longtze.



Figure 53: Anemometer on the bow of a Longtze.

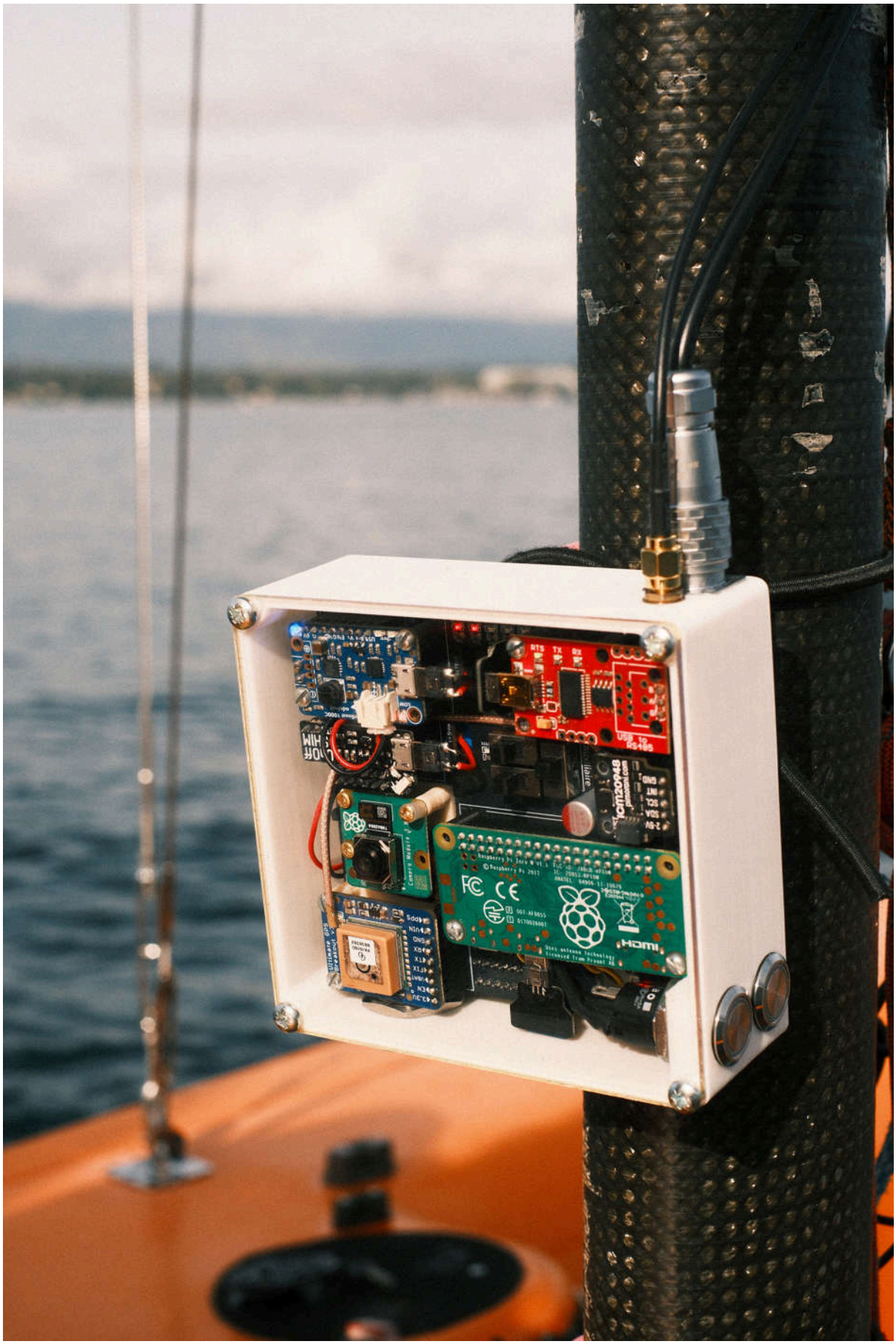


Figure 54: Abraxas on the mast of a Longtze.

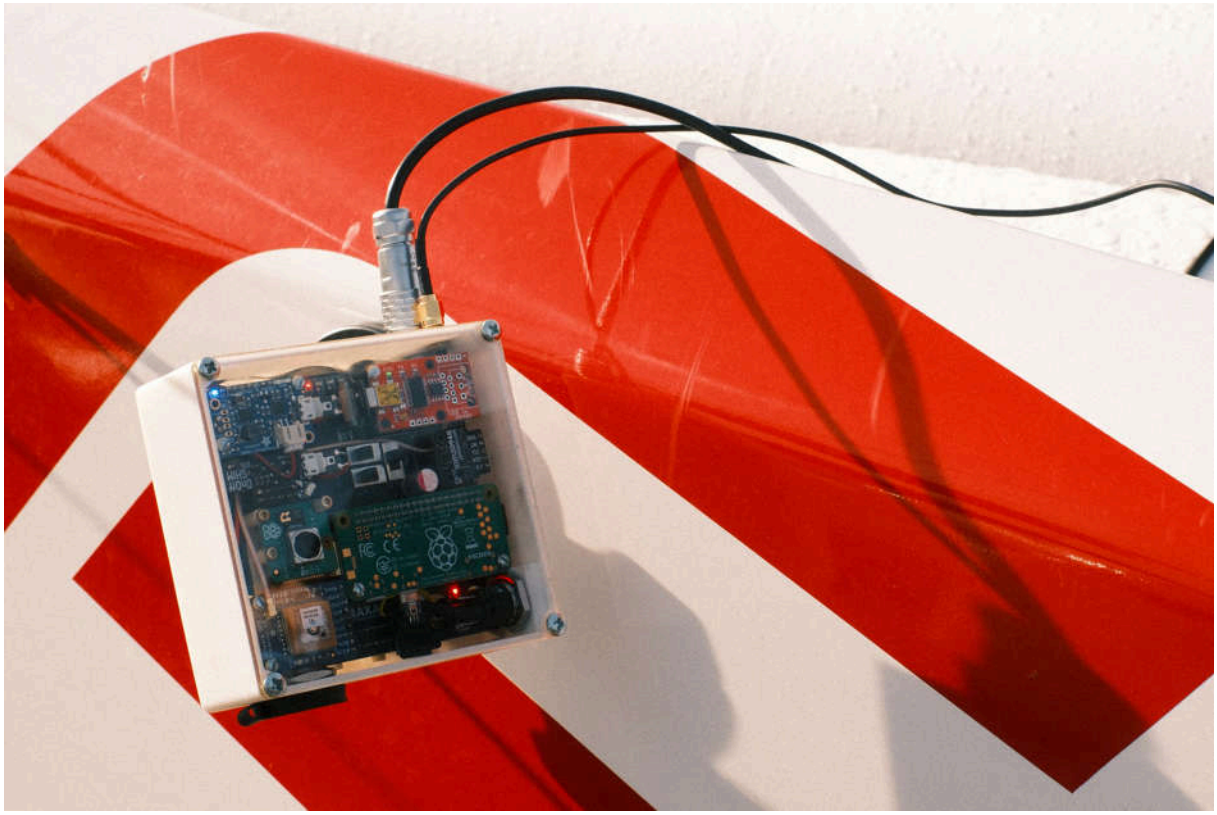


Figure 55: Abraxas on the bow of a Viper F16.

14.2 Access to the report PDF and the GitHub repositories

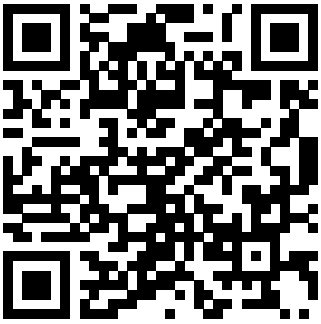
Report PDF

Abraxas (code)

Abraxas release


3D files


3D files release

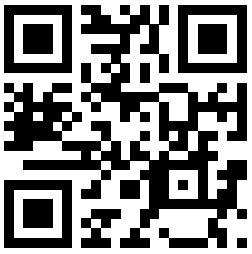

Server


Server release


PCB


PCB release

14.3 Access to the CAD model


OnShape model

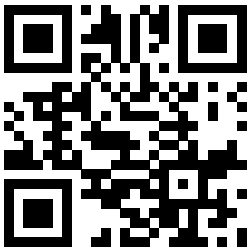
14.4 Access to the server


Analysis server


Data upload


Debrief of the test training session

14.5 Access to the hotspot



Live data visualisation



Data export

14.6 Websites of the existing solutions



Garmin



Vakaros Atlas 2



Orca

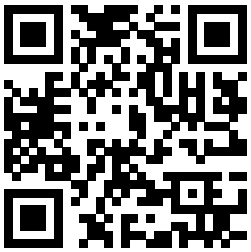
14.7 Hardware



IMU ICM20948 (Pimoroni)



Raspberry Pi Zero 2 WH



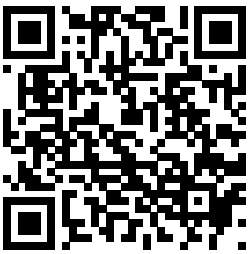
GPS MTK3339 (Adafruit)



Raspberry Pi V2 camera



PowerBoost 1000C



LiPo battery 8000 mAh



CSI ribbon cable

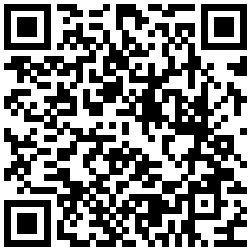


HY-WDC2E anemometer


Panel-mount micro-USB socket


Panel-mount USB-A socket


ON/OFF SHIM


USB/RS485 WaveShare


USB/RS485 SparkFun


Male connector


Female connector


IP67 button

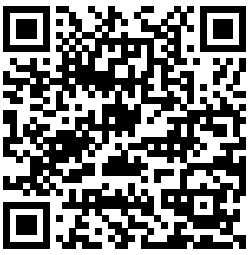

Zero4U USB hub


Active GPS antenna


Antenna connector

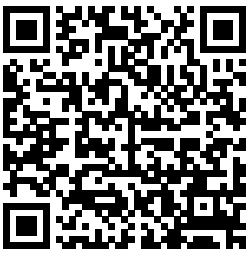

Angled USB cable


Micro-B plugs

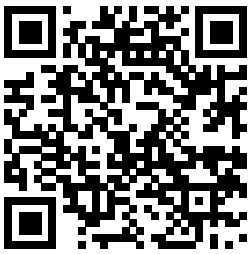

SparkFun USB/ RS485 converter

14.8 Technical documentation


Adafruit Ultimate GPS (PDF)


Adafruit PowerBoost 1000C

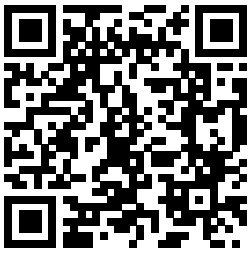

Pimoroni OnOff SHIM


Anemometer manual


Camera software (Raspberry Pi)


CircuitPython without CircuitPython


Installing Blinka


Adafruit CircuitPython GPS repository


icm20948-python repository


Trace width calculator (DigiKey)

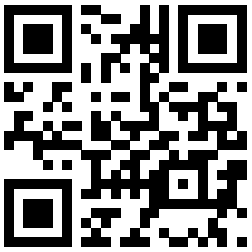
14.9 Articles and web pages


Advantages and weaknesses of the SSH protocol


Anemometer (Wikipedia)


JSON or SQL (DataBird)


Progress bar in bash


MyIP


Critérium Longtze 2025


Belle Étoile 2026


Alinghi page


SailGP page

14.10 Access to the code

The code can be accessed via the GitHub repositories.

Printed at Trajets on 22 October 2026

Trajets is a social enterprise that has been active since 1979 in the social and professional integration of people living with mental health difficulties